

Resumo de IA

Escrito por Diogo Silva

**adaptando os slides e imagens fornecidos para a
disciplina e informação do livro Artificial
Intelligence, A Modern Approach by Stuart
Russel & Peter Norvig**

Index

Introdução à Inteligencia Artificial	3
Agentes Inteligentes.....	5
Lógica Propsicional.....	24
Lógica de Primeira Ordem	37
Linguagem KIF.....	48
Ontologia Geral.....	53
Redes Semánticas.....	62
Redes de Bayes.....	74
Introdução à Pesquisa Clássica	85
Algoritmos de Pesquisa Clássica	93
Técnicas de Pesquisa Local.....	114
Técnicas de Pesquisa com Restrições.....	123
Planeamento.....	133

Introdução à Inteligência Artificial

I. Dimensões da Inteligência Artificial

Podemos definir Inteligência Artificial de diversas formas, mas historicamente, são quatro as dimensões utilizadas como padrões para a definição do que realmente é IA.

Estas dimensões dividem-se em termos da procura de fidelidade comparativamente com o **humano VS** com o ser **ideal** (i.e racional), e em termos da preocupação com o processo de **pensamento** e raciocinamento **VS** o **comportamento** e ação

As quatro dimensões são então:

- **Pensar Humanamente**
 - Comparação entre o processo de resolução (i.e thought process) de um problema entre um computador e um ser humano
 - Utilizado pelo campo da Ciencia Cognitiva
- **Pensar Racionalmente**
 - Resolução de problemas através de lógica
 - Problemas surgem do facto que é difícil inferir de forma lógica e formal problemas que provem de informação informal na qual o conhecimento pode não ser 100% correto. Por outro lado existe uma diferença entre resolver um problema teoricamente re
 - solve-lo na prática
- **Agir Humanamente**
 - Associado com o Teste de Turing
 - Um agente para pensar humanamente deve ser capaz de:
 - **Processamento de linguagem natural**

- **Representação de conhecimento**
- **Automação de raciocínio**
- **Machine learning.**
- Podemos ainda adicionar campos relativos à **visão de computador** (i.e percepção de objetos) e **robotica** (i.e capacidade de interagir com o mundo real)
- **Agir Racionalmente**
 - Um agente racional é aquele que age de forma a atingir o melhor outcome ou, quando não tem a certeza de qual é que este poderá ser, o melhor outcome esperado (**best expected outcome**)
 - Note-se que, fazer inferências corretas (i.e pensar racionalmente) faz parte de um agente racional devido ao facto de que agir racionalmente, envolve raciocinar logicamente (se bem que existem também formas de agir racionalmente que não envolvam inferências)
 - Por outro lado, as skills necessárias para passar o teste de Turing também permite a um agente agir racionalmente.
 - Esta dimensão possui vantagens sobre as outras:
 - É mais geral que as “leis do conhecimento” utilizadas pelo Pensamento Racional, visto que esta é apenas uma das formas que um agente tem de atingir racionalidade
 - É mais benéfica para o desenvolvimento científico do que as aproximações baseadas em comportamento humano, devido ao facto de que a racionalidade é um padrão matematicamente bem definido e completamente geral (ao contrário da complexidade do comportamento humano)

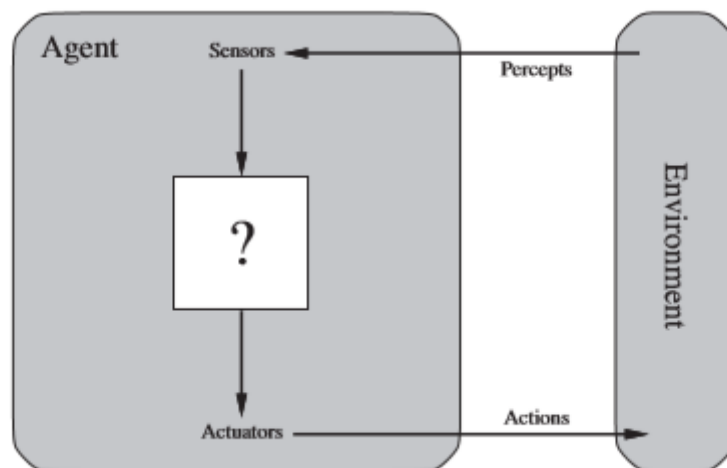
Agentes Inteligentes

I. Definição – Agente Inteligente

Podemos definir um agente como sendo “*tudo o que tenha a capacidade de visualizar o seu **ambiente** através de **sensores** e agir segundo esse ambiente através de **actuators**”.*

Aos inputs perceptuais recebidos pelo agente a cada instante chamamos **Percept**. A **Percept Sequence** é então o histórico de tudo o que um agente visualizou.

Em geral, podemos dizer que as ações de um agente a cada instante dependem na **percept sequence** observada até esse instante, mas não em nada que ainda não tenha observado.



Img 1.1 - Um agente recebe Percepts do ambiente através dos seus sensores, processa-os de alguma forma, e realiza uma ação sobre esse ambiente, dependendo do processamento da informação recebida, através dos seus Actuators

II. Definição – Agente Racional

Uma definição inicial de agente racional pode ser algo como: “*Um **agente racional** é um tipo específico de agente inteligente que faz sempre a **ação correta***”

Para que esta definição faça qualquer sentido convém definirmos o que realmente é uma **ação correta**. Podemos dizer que um agente realizou as ações corretas se “*a sequencia de ações que optou por realizar são desejáveis comparativamente com uma **medida de performance** que a avalia relativamente à sequencia de **environment states** fornecidos*”

Note-se que a medida de performance avalia segundo os estados do ambiente e não os do agente. Caso definissemos que a sequencia de ações tomadas foi boa segundo a opinião do próprio agente, este poderia atingir perfeita racionalidade convencendo-se de tal (i.e “Acho que o estou a fazer é bom, portanto sou perfeito, mesmo que o que esteja a fazer seja uma valente merda”).

Note-se também que definir uma boa **medida de performance** não é tão simples quanto isso. P.ex, no caso de um agente de limpeza, se definissemos bom performance segundo a quantidade de sujidade limpa, o agente poderia simplesmente optar por limpar o chão, suja-lo e voltar a limpar o mesmo chão, sendo que isto estaria de acordo com a medida de performance definida. Como rule of thumb a sugestão é “*definir medidas de performance de acordo com o que **nos queremos** e não de acordo com a forma de como o agente se deve comportar*”

Podemos agora completar a nossa **definição de agente racional** como: “*Um agente racional é aquele que, para cada possível **percept sequence** seleciona a ação que é expectavel de maximizar a **medida de performance**, dadas as provas definidas pela percept sequence e qualquer conhecimento a-priori que o agente possua*”

III. Racionalidade vs. Omnisciencia

Tendo em conta a definição de racionalidade que demos na secção anterior, convém também fazer a distinção entre um agente racional e um **agente omnisciente**.

Este ultimo sabe qual o **exato outcome** das suas ações, conseguindo agir de acordo. Efetivamente, omnisciencia é quase impossivel pelo que não tende a ser procurada.

É importante frisar que **racionalidade != perfeição**. Um agente racional **maximiza o expected performance**, um agente omnisciente/perfeito **maximiza actual performance**.

IV. Racionalidade – Recolha de informação, Aprendizagem e Autonomia

Agentes que utilizam um conjunto reduzido de **informação** para escolher as suas ações não podem ser considerados racionais racionais! P.ex, não podemos dizer que um agente que atravessasse a estrada sem olhar seja racional pois, por muito que esteja a agir de acordo com toda a sua **percept sequence**, esta não possui todas as informações necessarias para que pudesse atravessar a estrada de forma segura.

Para que um agente seja considerado racional, este deve portanto realizar ações de **information gathering** para que este possa adquirir uma **percept sequence** que lhe permita realizar ações adequadas

Para além de recolher informação, um agente racional deve tambem **aprender** o mais quanto possivel sobre o ambiente que o rodeia. Mesmo nos casos em que o agente possui conhecimento a-priori do seu ambiente, este deve na mesma realizar o esforço de aprendizagem continua, de forma a encobrir os casos em que o ambiente seja alterado de forma não expectavel

Caso um agente se guie apenas por conhecimento previamente adquirido, ao invés de utilizar os seus próprios **percepts**, podemos considerar que não possui **autonomia**.

Needless to say que um agente racional deve ser autônomo, possuindo a capacidade de aprender tudo o que possa para compensar por erros parciais ou totais do conhecimento que lhe foi previamente imbutido.

V. Task Environment – Definição

De uma forma bastante geral, os “**Task Environments** referem-se aos problemas dos quais os agentes racionais são as soluções”

Podemos descrever um Task Environment segundo o **Performance Measure, Environment, Actuators e Sensors – PEAS**

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Img 5.1 – Exemplo da descrição PEAS do task environment de um condutor de taxis

Performance Measure

- Qualidades/Objetivos desejáveis que o nosso agente deve procurar cumprir
- Por vezes alguns destes objetivos entram em conflito uns com os outros, pelo que devem ser considerados tradeoffs

Environment

- Ambiente que o nosso agente vai enfrentar
- Contem todos os obstaculos, meios, objetos, etc com os quais o agente irá interagir
- O quão mais restringido o ambiente for, o mais facil se tornará desenhar o problema

Actuators

- Métodos que o agente possui de interagir com o ambiente.
- Sistemas de output à disposição do agente

Sensors

- Sensores que o agente possui para perceber o ambiente que o rodeia
- Sistemas de input à disposição do agente

Tipo de agente	Percepção	Ação	Objectivos	Ambiente
Sistema de diagnóstico médico	Sintomas, respostas do paciente	Perguntas, testes, tratamentos	Saúde do paciente, custo mínimo	Paciente, hospital
Sistema de análise de imagens de satélite	Imagem	Devolver uma categorização da cena	Categorização correcta	Imagens de um satélite em órbita
Braço robótico para em embalagem	Imagem, sinal de força	Colocar peças em caixas	Colocar as peças na posição correcta	Alimentador de peças, caixas
Controlador de refinaria	Temperatura, pressão	Abrir e fechar válvulas; ajustar temperatura	Pureza, segurança	Refinaria
Tutor de inglês interactivo	Palavras introduzidas	Propôr exercícios, corrigi-los, dar sugestões	Maximizar o resultado dos alunos num teste	Conjunto de alunos

Img 5.2 – Outros exemplos da descrição PEAS para diferentes tipos de agentes

VI. Task Environment – Propriedades

Podemos categorizar task environments segundo certas dimensões:

- **Fully Observable vs Partially Observable**

- Caso os sensores de um agente lhe forneçam um estado completo do ambiente a cada instante de tempo, podemos dizer que o ambiente é **Fully observable**.

- Note-se que para ser considerado **fully observable** os sensores não precisam de detetar **todos os aspetos do ambiente** mas sim apenas os **relevantes para a escolha da ação**

- Podemos ter ambientes **Partially observable** devido a ruído ou sensores imprecisos, ou devido a não possuímos os sensores necessários para captar todo o ambiente.

- No caso do agente não possuir sensores nenhuns, o ambiente é **Unobservable**

- A vantagem que os ambientes **fully observable** possuem é o facto de que o agente não tem de manter um estado interno to keep track of the world.

- **Single Agent vs Multi Agent**

- Um task environment é **Single agent** se possuir apenas um agente, caso contrário é **Multi agent**

- Devemos considerar uma entidade um outro agente caso o seu comportamento pode ser descrito como *“procurando maximizar performance measures cujos valores dependem do agente inicial”*

- Com esta definição podemos subcategorizar os ambientes multiagentes como **competitivos** - *caso a maximização do performance measure de um agente B vá minimizar os do agente A*

– ou **cooperativos** - caso a maximização do performance measure de um agente *B* maximize também as do *A*

- **Deterministic vs Stochastic**

- Caso o próximo estado do ambiente seja completamente determinado pelo estado atual e pela ação executada pelo agente, podemos dizer que o ambiente é **Deterministic**. Caso contrário, este é considerado **Stochastic**

- Nos ambientes **partially observed** ou em situações complexas reais (nas quais é impossível dar keep track de todos os aspetos não observados) o ambiente deve ser tratado como **stochastic**.

- Adicionalmente podemos dizer que um ambiente é **uncertain** caso seja **não fully observable** OU **não determinístico**

- **Episodic vs Sequential**

- Num task environment **Episodic** a experiencia do agente é dividida em episódios atomicos, em cada um dos quais o agente recebe um percept e realiza uma unica ação. O facto de serem atomicos implicam que o proximo episodio não dependa das ações tomadas nos anteriores

- Em task environment **Sequential**, as decisões tomadas poderão afetar todas as futuras decisões

- **Static vs Dynamic**

- Se o ambiente puder sofrer alterações enquanto o agente está a deliberar, dizemos que o ambiente é **Dynamic** para aquele agente. Caso contrário é **Static**.

- Comparativamente, ambientes **static** são faceis de lidar dado que o agente não tem de continuamente olhar para o mundo enquanto está a decidir que ação tomar. Por outro lado ambientes **dynamic** continuamente perguntam ao agente o que é que este

quer fazer, caso este ainda não tenha tomado uma decisão, responderá com “do nothing”.

- Temos ainda que, se um ambiente não mudar com a passagem do tempo, mas o **performance do agente** mudar, consideramos o task environment **SemiDynamic** (um bom exemplo disto é o xadrez caso seja jogado com um relógio e a pontuação do agente dependa de alguma forma do quão rápido ou lento o agente for)

- **Discrete vs Continuous**

- Esta distinção é aplicada ao state do ambiente (segundo a forma de como o tempo é tratado) e aos percepts e ações do agente.

- Como exemplo temos o xadrez, que consiste num numero finito de **Distinct states** e um **Discrete Set** de percepts e actions

- Conduzir um carro porém é um problema **Continuous** em state e tempo.

- **Known vs Unknown**

- Esta categorização não se refere ao ambiente em si, mas sim ao **estado de conhecimento do agente** sobre as leis do ambiente.

- Num **Known** environment, os outcomes para todas as ações são dados. Caso o ambiente seja **Unknown** o agente terá que aprender como é que este funciona de forma a conseguir realizar boas ações.

- Note-se que isto nada tem a ver com a **observabilidade** do ambiente. Podemos ter um ambiente **known** ser **partially observable**, p.ex, um jogo de cartas (tipo solitário, aka you lmao gottem). Um outro exemplo são os video jogos, nos quais o ambiente é (ou poderá ser) **fully observable**, visto que o ecrã mostra todo o estado do jogo, mas **unknown** caso o agente não seja fornecido com as ações que cada botão realiza.

NOTA: Os ambientes mais difíceis de tratar são *partially observable, multiagent, stochastic, sequential, dynamic, continuous e unknown*

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Img 6.1 – Exemplo da categorização dos Task Environments descritos na secção anterior

VII. Tipos de Agentes

O objetivo da IA é o de criar um **agent program** que implemente a **agent function** – o mapeamento entre os percepts e as ações (note-se que o agent program recebe como input apenas o percept atual enquanto que a agent function pode olhar para toda a percept sequence)

Podemos ainda assumir que dito programa irá correr dentro de um qualquer computing device com sensores e atuadores físicos, a isto chamamos **arquitetura**.

Temos então que: **agent = architecture + program**

Nesta secção vamos-nos focar inteiramente no **agent program**.

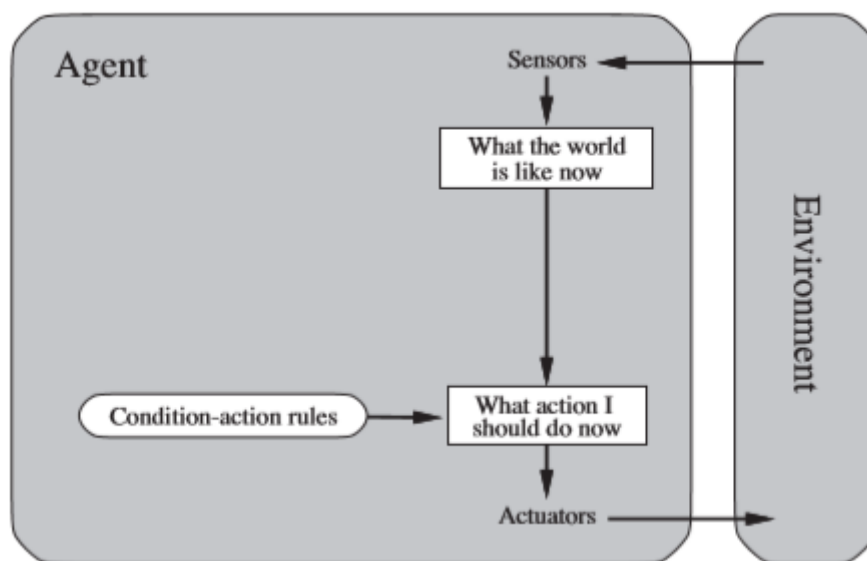
Note-se que todos os **agent programs** que vamos apresentar possuem em comum o facto de receberem como input o **percept atual** (dos sensores), retornando uma **ação** (a ser executada pelos actuators).

VII.I Simple Reflex Agents – Agente Reativo Simples

Estes agents limitam-se a seleccionar ações dependendo apenas no **percept atual**, ignorando toda a restante percept history.

Baseia-se na ideia de conexões **condition-action rule** (a.k.a **if-then rules** ou **regras de condição (ou situação)-ação**).

Um exemplo de comportamento **simple reflex** seria o de um condutor que vê que o carro da frente começou a parar (recebeu como percept, p.ex, as luzes de travagem do carro da frente), e insitntivamente começa também a parar – **if** car-in-front-is-breaking **then** initiate-breaking



Img 7.1.1 –Modo como um Simple Reflex Agent age.

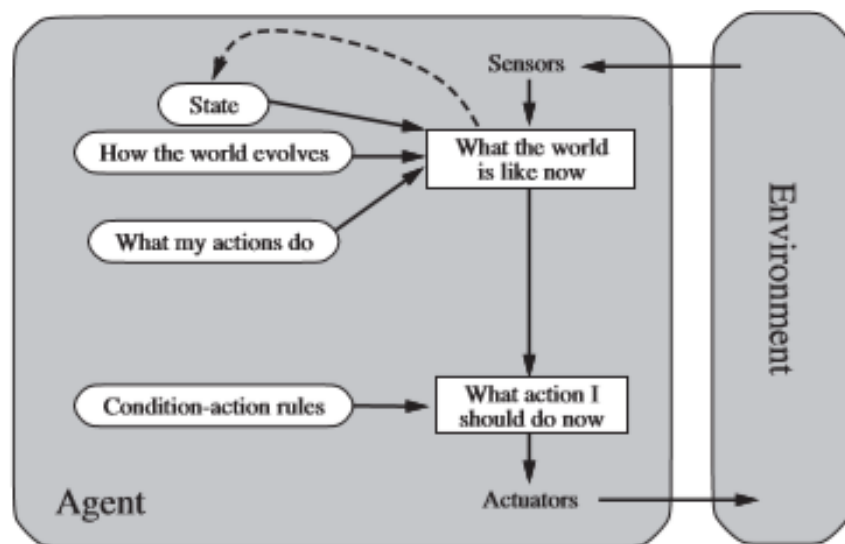
Este tipo de agentes são incrivelmente simples, mas possuem uma **inteligência** bastante **limitada** – *funcionam apenas se a decisão correta puder realmente ser feita apenas através do percept atual*

Note-se também que estes agentes só funcionam caso o ambiente seja **fully observable**

VII.II Model-Based Reflex Agents – Agente Reativo Com Estado Interno

Podemos expandir a ideia dos Simple Reflex Agents introduzindo um **internal state** qualquer que dependa da **percept history**.

Com isto o agente vai ser capaz de tratar task environments com **partial observability** visto que o estado interno vai ser capaz de guardar a parte do mundo que não consegue ver atualmente. Ou seja, o *“internal state deve refletir pelo menos em parte, alguns dos aspetos que não puderem ser observados no estado atual”*



Img 7.2.1 –Modo como um Model-Based Reflex Agent age.

Note-se “que de forma a atualizar a informação do estado interno vamos ter que possuir informação de **como é que o mundo evolui independentemente do agente e de que forma é que as ações do agente afetam o mundo**”

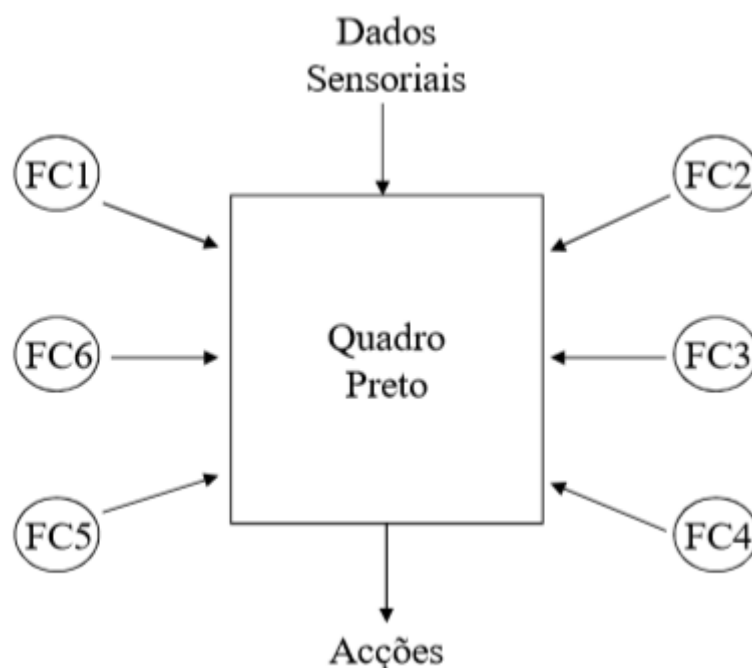
É também importante frizar que pode não ser possível ao agente determinar o estado atual de um partially observable environment de forma exata. A caixa “What the world sees now” refere-se portanto à melhor guess que o agente pode inferir.

VII.III Blackboard System – Sistemas de Quadro Preto

Podem ser vistos como uma elaboração dos Model-Based Reflex Agents.

A ideia base consiste em termos uma base de conhecimento – **blackboard** – que vai sendo iterativamente atualizada por um diverso grupo de specialist **knowledge sources**, começando pela especificação do problema e terminando na solução

Cada knowledge source atualiza o blackboard com uma solução parcial quando as suas constraints internas derem match com o estado atual do blackboard.



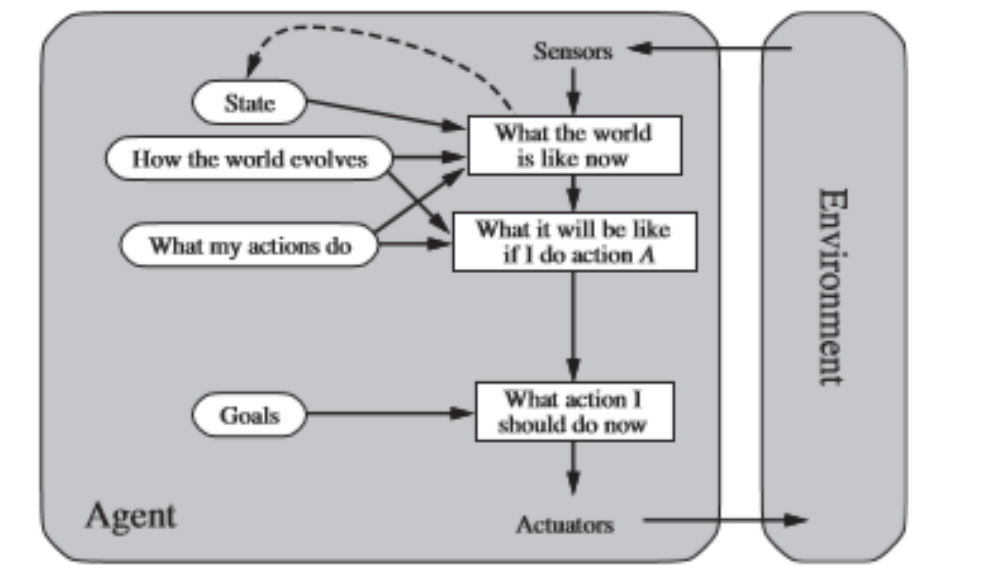
Img 7.3.1 – Visualização de como as varias fontes de conhecimento – FC – vão contribuir para o Quadro preto

Uma Knowledge Source pode ser vista como um especialista num dado dominio. Note-se também que cada Knowledge Source rege-se por um conjunto de **regras de condition-action**

VII.IV Goal-Based Agents – Agente Deliberativo orientado por objetivos

Possuir informação sobre o atual estado do ambiente pode não ser suficiente para se chegar a uma ação que faça sentido. Para estes casos, o agente pode precisar de um **objetivo – goal information** – que descreva situações que sejam desejáveis.

Por exemplo, um condutor num cruzamento pode virar à esquerda, direita, ou continuar em frente, mas qual destas ações deve tomar? Isso depende de qual for o destino pretendido – Informação sobre qual o destino seria então a **goal information** necessária.



Img 7.4.1 –Modo como um Goal-Based Agent age.

Note-se a diferença entre o **processo de decision-making** entre um reflex agent (que use **condition-action rules**) e um goal-based agent. Este primeiro pergunta – “O que é que está a acontecer e como é que devo reagir?”, o outro pergunta – “Como é que devo reagir ao que está a acontecer de forma a ser feliz no final?”. Existe portanto uma consideração pelo futuro.

Esta flexibilidade de pensamento envolve maior computação, mas torna o agente mais apto visto que o conhecimento que

suporta as suas decisões é representado explicitamente e pode ser modificado.

Comparando uma ultima vez o thought process deste agente com agentes reflexivos, voltemos ao exemplo da travagem. Um agente reflexivo recebe como percept as luzes de travagem do carro da frente que se ligaram, e, posusindo a condição **if** car-in-front-is-breaking **then** initiate-breaking, vai começar a parar. Por outro lado um agente que se baseie em objetivos vê que o carro da frente começa a parar e vai pensar, se o carro da frente ligou as luzes de travagem é porque vai começar a abrandar, pela forma como o mundo evolui, a unica ação que posso tomar que me leve ao objetivo de “não bater noutros carros” é começar a abrandar.

VII.V Utility-Based Agents – Agente Deliberativo orientado por função de utilidade

Por vezes objetivos não bastam para gerar comportamento de alta-qualidade em alguns ambientes – P.ex, existem varios caminhos que levarão um condutor ao seu destino, porém alguns são mais rapidos, outros mais seguros, outros mais baratos, etc, etc

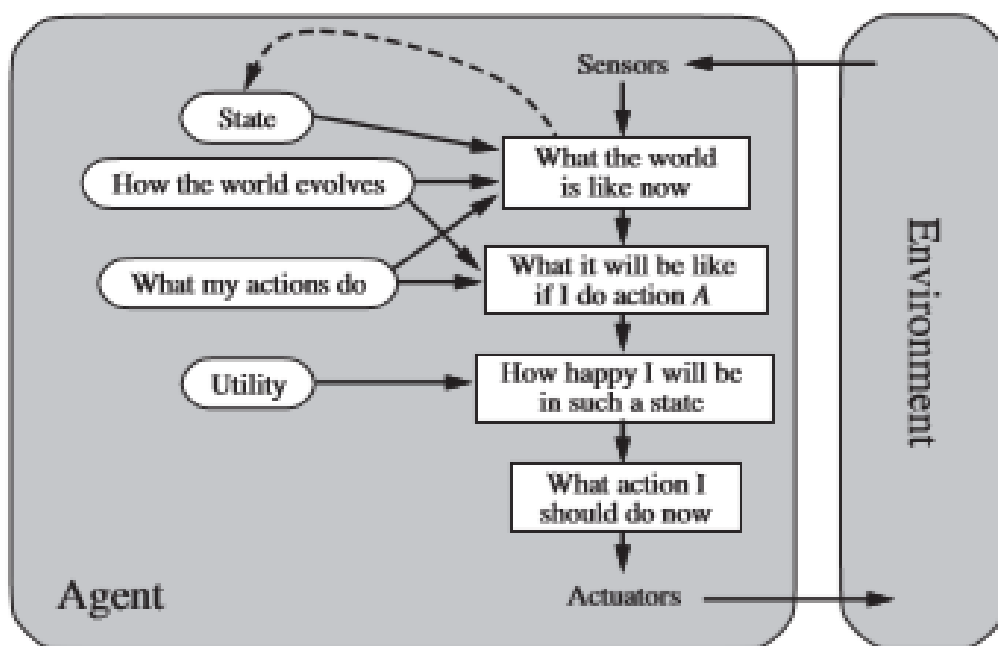
Devemos ter uma forma de comparar diferentes world states de acordo com o quão “felizes” farão o agente. A isto chamamos **Utility**

Já sabemos que a **performance measure** atribui uma pontuação a qualquer dada sequencia de environment states de forma a conseguir distinguir caminhos mais ou menos desejaveis. A **utility function** de um agente é, essencialmente, uma internalização da preformance measure.

Caso a internal utility function e o external performance measure estejam de acordo, um agente orientado por função de utiidade escolherá ações que maximizem a sua utility (garantindo portanto a maximização da performance measure, tornando-o num agente racional automaticamente).

É conveniente dizer que os outros agentes podem também ser racionais – existem várias maneiras de atingir racionalidade como já vimos – mas os utility-based agents possuem vantagens em termos de flexibilidade e aprendizagem.

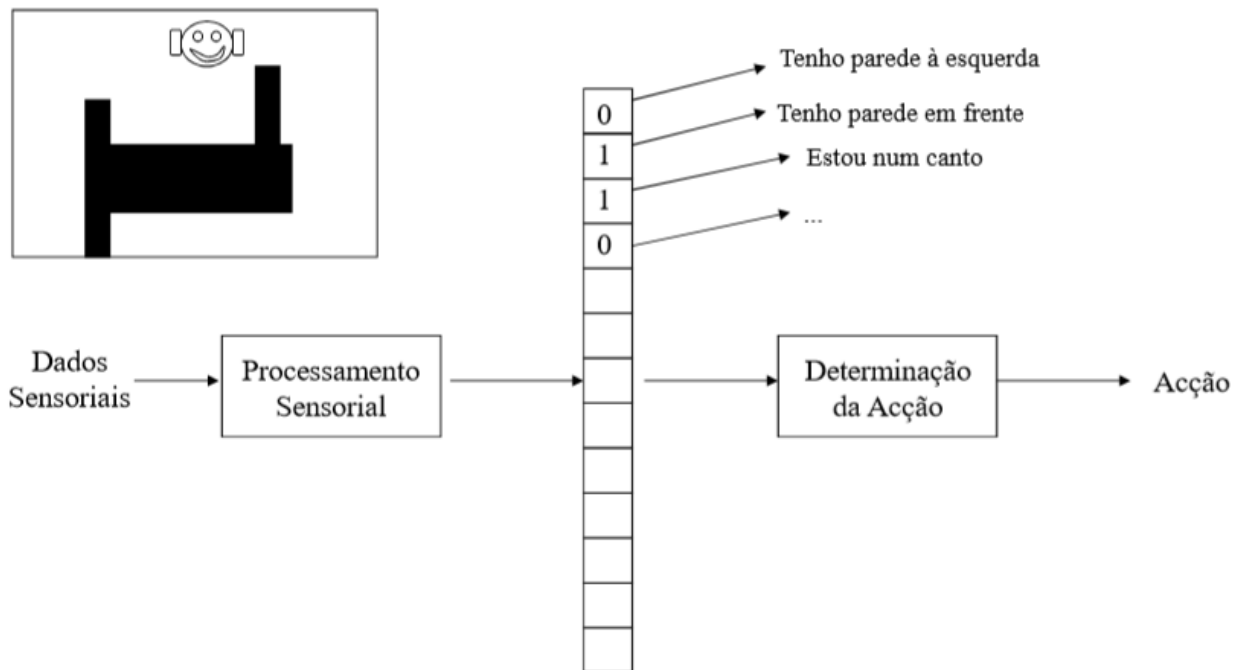
Por outro lado há casos em que os goals não são adequados – Quando entram em conflitos, ou quando existem vários goals e não sabemos se conseguimos atingir qualquer um que seja com certeza. Nestes casos devemos obviamente optar pelos Utility Based Agents



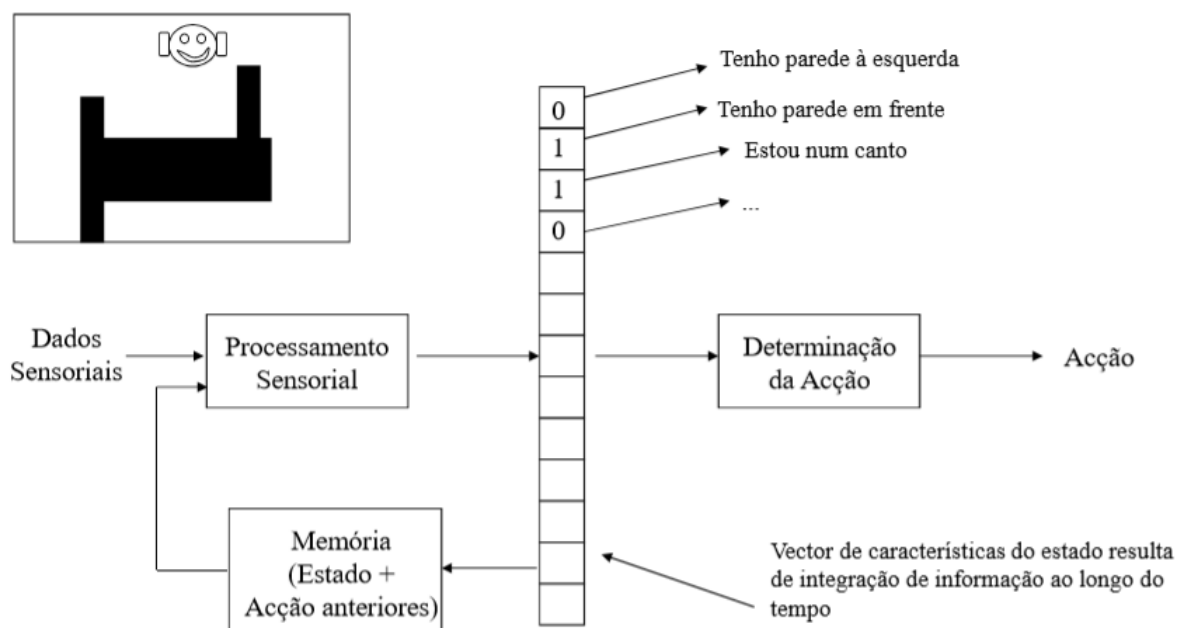
Img 7.5.1 –Modo como um Utility-Based Agent age.

Convém também dizer que um rational utility-based agent escolhe a ação que maximize a **expected utility** (e não a actual utility, pois para isto o agente teria de ser perfeito, e não apenas racional, mesmo sob ambientes **parcialmente observáveis** e/ou **estocásticos**)

VIII. Representação do ambiente/percepção do agente



Img 8.1 – Representação através de um vetor de características



Img 8.2 – Representação através de um vetor de características com um estado interno

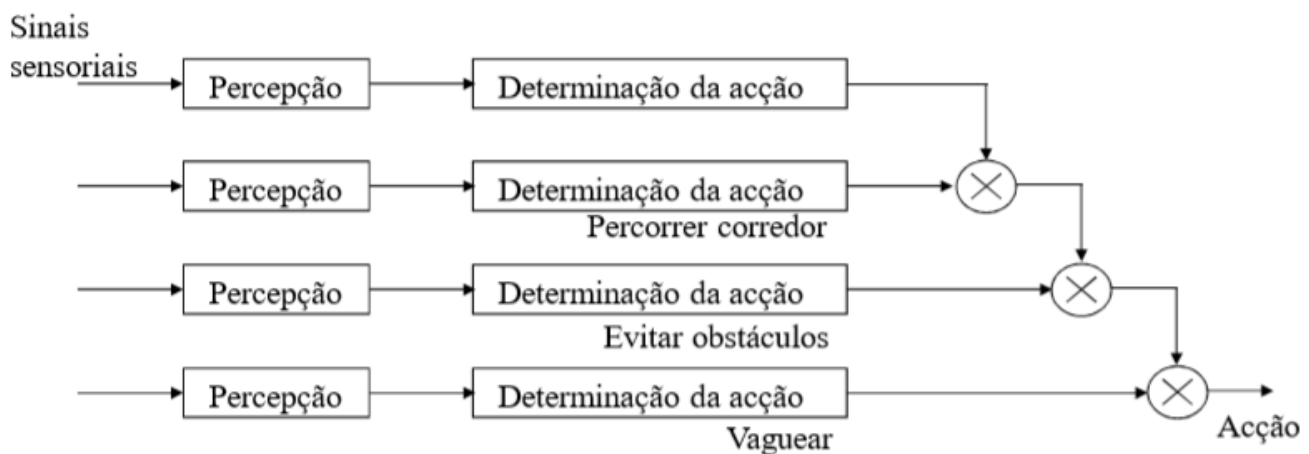
IX. Arquiteturas de Agentes

Já vimos que um agente é formado pelo *agent program* em conjunto com a sua arquitetura, i.e, *o meio no qual o agent program vai correr e que deve **possuir sensores e atuadores***.

Vamos agora dar exemplos de algumas arquiteturas.

IX.I Subsumption Architecture - Subsunção

A arquitetura de **subsunção** procura “estabelecer a ligação entre a percepção e a ação a **varios niveis**, resultando numa **organização por camadas**”.

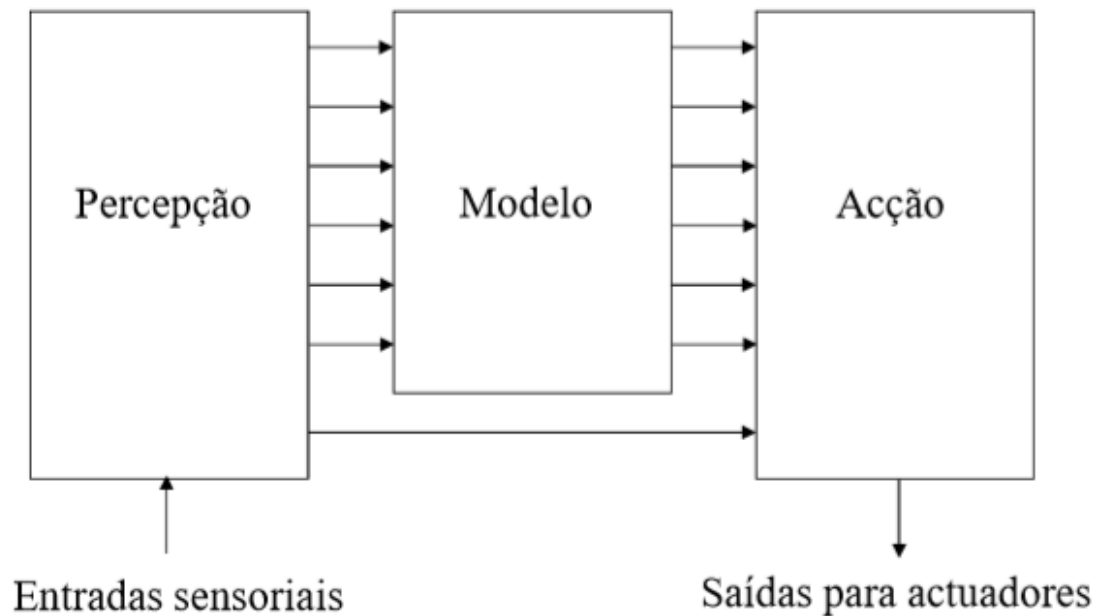


Img 9.1.1 – Desenho da Arquitetura de Subsunção

Note-se que a **camada mais baixa é a mais reativa**

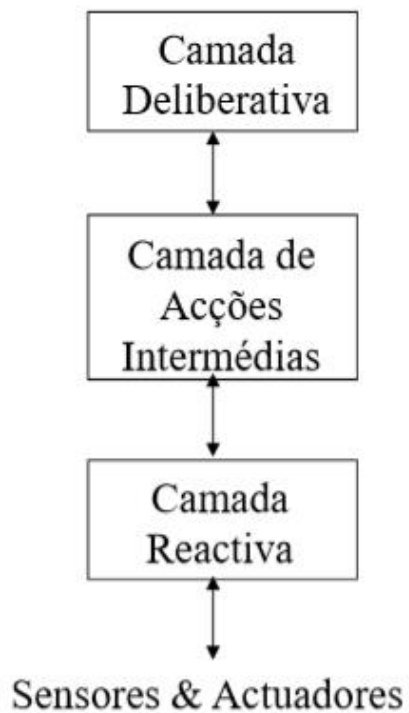
Diga-se também que **o peso da componente deliberativa aumenta à medida que se sobe na estrutura de camadas** (i.e, decisões tomadas em camadas superiores têm prioridade sobre as tomadas nas camadas inferiores)

IX.II Triple Tower Architecture – Três Torres



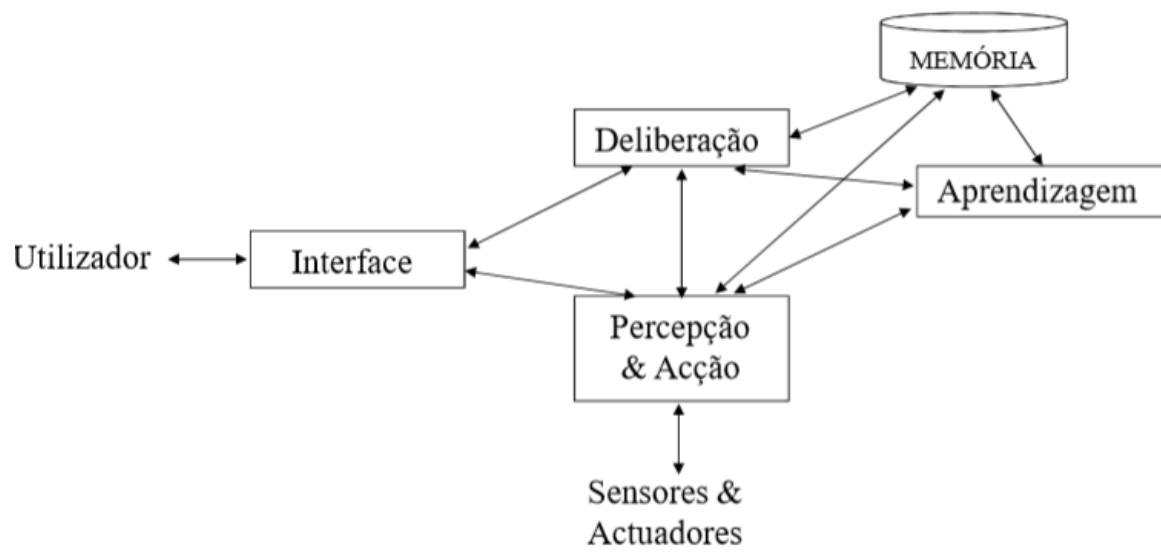
Img 9.2.1 – Desenho da Arquitetura de Tres Torres

IX.III Three Layer Architecture – Três Camadas



Img 9.3.1 – Desenho da Arquitetura de Tres Camadas

IX.IV CARL Architecture



Img 9.4.1 – Desenho da Arquitetura de CARL

Lógica Proposicional

I. Introdução à Lógica

Em muitas situações somos obrigados a representar uma determinada **knowledge base** – conjunto de **frases** que representam uma afirmação sobre o mundo (caso a frase seja aceite como verdadeira sem ter que derivar do conhecimento de outras frases disemos que é um **axioma**).

A **lógica** trata desse mesmo problema de representação através da definição de:

Sintaxe

- Conjunto de frases ou formulas que é possível escrever
- Chamamos a estas frases **formulas bem formadas** (ou WFF)

Semântica

- Processo que vai estabelecer a relação entre as frases, escritas segundo a sintaxe previamente definida, e os factos que estas representam
- Basicamente é aquilo que vai dar significado às frases, definindo se são verdade num dado mundo
- No exemplo da **Lógica Proposicional** que vamos abordar neste capítulo, podemos definir a semântica através de **tabelas de verdade**

Regras de Inferência

- Regras que permitem manipular as frases gerando umas a partir de outras
- São estas a base do processo de raciocínio lógico

A ideia de **modelo** vai ser explicada melhor mais à frente na secção IV, mas basicamente podemos pensar no modelo como sendo um “possível mundo”. Por exemplo, imaginando a frase “ $x + y = 4$ ” é verdadeira no modelo $m1$ no qual $x=2$ e $y=2$, modelo $m2$ no qual $x=3$ e $y=1$ mas falso no modelo $m3$ no qual $x=3$ e $y=3$. De foram simplifacada, um modelo não é nada mais do que uma possível atribuição de variáveis que ditam se certas frases são verdadeiras ou falsas

II. Consequências Lógicas (Entailment) e Provas

É das **Regras de Inferência** que nasce a ideia de **entailment**, ou **consequência lógica** entre frases, ou seja, a ideia que uma frase logicamente procede de outra. Formalmente temos que , a frase B procede de A se e só se para cada **modelo** no qual A é true, B também é true

$$\alpha \models \beta$$

Img 2.1 – Notação matemática para “Alfa entails Beta” ou “Beta procede de Alfa”

Uma exemplo muito básico de inferencia seria o seguinte:

- Tendo as frases $A - x = 0$ e $B - x*y = 0$
- Sabendo que $A \models B$
- Então sabemos que sempre que A for verdadeiro, B também será verdadeira (obrigatoriamente), ou seja, sempre que x for 0, $x*y$ será também 0

Portanto, e rephraseando, temos que, se $A \models B$, então, B tem de ser verdadeiro em todos os modelos nos quais A é verdadeiro

Uma sequencia de formulas $\{A_1, A_2, \dots A_n\}$, é uma **Prova** (ou **dedução**) de A_n a partir de um conjunto de fórmulas X se cada uma das formulas, A_j está em X ou poder ser inferida a partir das formulas $A_1 \dots A_{j-1}$.

Nesse caso a prova escreve-se $X \vdash A_n$

Dizemos que um conjunto de regras de inferencia é **sound** ou **truth preserving** ou **correto** se todas as formulas que gera são consequencias lógicas, ou seja, se só derivar entailed sentences.

Por outro lado, o conjunto de regras de inferencia é **complete** se permitir gerar todas as consequencias lógicas, ou seja, se conseguir derivar qualquer frase que for entailed.

*Um **sistema de inferência correto e completo permite tirar consequencias lógicas sem ter que analisar caso a caso as várias interpretações!***

III. Lógica Proposicional - Sintaxe

Lógica proposicional é um tipo de lógica bastante simples baseada em **proposições** – frases declarativas elementares que podem ser verdadeiras ou falsas

Exemplos de proposições incluem:

- “A neve é branca”
- “O açúcar é um hidrocarbono”
- “Está a chover”

Para além de **proposições** temos:

- **Frases Atómicas**
 - Frases formadas por uma unica proposição
 - As proposições podem ser representadas por simbolos que comecem com uma letra maiuscula
 - Ex:
 - *Branca*
 - Frase atómica que consiste apenas no simbolo proposicional Branca, que aponta para a proposição “A neve é branca”
 - *C*
 - Frase atómica com simbolo prop. C que aponta para a proposição “Esta a chover”

- **Frases Complexas ou Fórmulas**

- Frase composta por uma ou mais **frases atômica** interligadas por **conectivas lógicas**
- Ex:
 - $C \wedge \text{Branca}$
 - Frase complexa que pode ser lida como “Está a chover e a neve é branca”
 - $\neg \text{Branca}$
 - Frase complexa que pode ser lida como “A neve não é branca”

- **Conectivas Lógicas**

- Conectores utilizado para interligar frases simples:
- $\wedge$ - Conjunção (**e**)
 - $C \wedge \text{Branca}$ – Está a chover e a neve é branca
- $\vee$ - Disjunção (**ou**)
 - $C \vee \text{Branca}$ – Está a chover ou a neve é branca
- $\Rightarrow$ - Implicação (**se...então...**)
 - $C \Rightarrow \text{Branca}$ – Se está a chover então a neve é branca
- $\Leftrightarrow$ - Bicondicional (**...se e só se...**)
 - $C \Leftrightarrow \text{Branca}$ – A neve é branca se e só se estiver a chover
- $\neg$ – Negação (**não**)
 - $\text{NOT } C$ – Não está a chover

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Img 3.1 – Tabelas de verdade para os conectores lógicos de lógica proposicional

Temos então a gramática de lógica proposicional:

$$\begin{aligned} \textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\ \textit{AtomicSentence} &\rightarrow \textit{True} \mid \textit{False} \mid P \mid Q \mid R \mid \dots \\ \textit{ComplexSentence} &\rightarrow (\textit{Sentence}) \mid [\textit{Sentence}] \\ &\mid \neg \textit{Sentence} \\ &\mid \textit{Sentence} \wedge \textit{Sentence} \\ &\mid \textit{Sentence} \vee \textit{Sentence} \\ &\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\ &\mid \textit{Sentence} \Leftrightarrow \textit{Sentence} \end{aligned}$$

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Img 3.2 – Gramática da lógica proposicional

IV. Interpretação em Lógica Proposicional

Já explicamos que um **modelo de uma fórmula** é uma **interpretação que satisfaz certa fórmula**, por exemplo, dada a fórmula $x^2=4$, os únicos modelos possíveis são aqueles em que $x=2$.

Mas ainda não explicamos o que é uma **interpretação!** (even though its fucking obvious what it is)

Em lógica proposicional, uma **interpretação de uma fórmula** é uma **atribuição de valores de verdade ou falsidade às várias proposições que nela ocorrem**. Por exemplo, a fórmula $A \wedge B$ tem 4 interpretações possíveis – A:True, B:True ; A:True, B:False ; A:False, B:True ; A:False, B:False

Acrescentamos ainda os conceitos de Satisfatibilidade e Tautologia:

- **Satisfatibilidade**

- Uma interpretação satisfaz uma fórmula se a **fórmula for verdadeira para essa interpretação**
- No exemplo dado, a unica interpretação que satisfaz a fórmula é A:True, B:True
- Visto que um **modelo** é uma interpretação que satisfaz a formula, o unico modelo para a nossa fórmula é, claramente, A:True, B:True

- **Tautologia**

- É uma fórmula cujo valor é verdadeiro em qualquer interpretação
- Por exemplo, a fórmula $A \vee \neg A$

V. Regras de Substituição

Em lógica temos certas frases que são **logicamente equivalentes** a outras. Dai nascem as regras de substituição que nos permite transformar certas frases noutras. Em logica proposicional temos então:

Leis de DeMorgan:

- $\neg (A \wedge B) \equiv \neg A \vee \neg B$
- $\neg (A \vee B) \equiv \neg A \wedge \neg B$
- $A \wedge B \equiv \neg (\neg A \vee \neg B)$
- $A \vee B \equiv \neg (\neg A \wedge \neg B)$

Associação:

- $A \wedge B \wedge C \equiv A \wedge (B \wedge C)$
- $A \vee B \vee C \equiv A \vee (B \vee C)$

Distribuição:

- $A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$
- $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$

Definição da Bicondicional:

- $A \Leftrightarrow B \equiv (A \Rightarrow B) \vee (B \Rightarrow A)$

Definição da implicação:

- $A \Rightarrow B \equiv \neg A \vee B$

Comutação:

- $A \wedge B \equiv B \wedge A$
- $A \vee B \equiv B \vee A$

Transposição:

- $A \Rightarrow B \equiv \neg B \Rightarrow \neg A$

Dupla negação:

- $\neg \neg A \equiv A$

VI. Regras de Inferência

Para além das regras de substituição temos também as seguintes regras de inferência:

Modus Ponens: $\{ A, A \Rightarrow B \} \vdash B$

Modus Tolens: $\{ \neg B, A \Rightarrow B \} \vdash \neg A$

Silogismo hipotético: $\{ A \Rightarrow B, B \Rightarrow C \} \vdash A \Rightarrow C$

Conjunção: $\{ A, B \} \vdash A \wedge B$

Eliminação da conjunção: $\{ A \wedge B \} \vdash A$

Disjunção: $\{ A, B \} \vdash A \vee B$

Silogismo disjuntivo (ou resolução unitária):

$$\{ A \vee B, \neg B \} \vdash A$$

Resolução: $\{ A \vee B, \neg B \vee C \} \vdash A \vee C$

Dilema construtivo:

$$\{ (A \Rightarrow B) \wedge (C \Rightarrow D), A \vee C \} \vdash B \vee D$$

Dilema destrutivo:

$$\{ (A \Rightarrow B) \wedge (C \Rightarrow D), \neg B \vee \neg D \} \vdash \neg A \vee \neg C$$

Img 6.1 – Regras de Inferencia em Logica Proposicional

Para além destas regras temos também os seguintes

MetaTeoremas:

- **Teorema da Dedução**

- Se $\{ A_1, A_2, \dots, A_n \} \models B$, então $A_1 \wedge A_2 \wedge \dots \wedge A_n \Rightarrow B$
 - E vice-versa

- **Redução ao absurdo**

- Se o conjunto de formulas X é satisfativo (logo tem pelo menos um modelo) e $X \cup \{ \neg A \}$ não é satisfativo, então $X \models A$

Nenhuma das regras anteriormente apresentadas são completas por si só. Existe, porém uma regra entre as apresentadas que merece atenção especial – A **Resolução**, $\{A \vee B, \neg B \vee C\} \vdash A \vee C$

Convém explicar que a **Resolução por si só não é completa!** É correta porque gera fórmulas necessariamente verdadeiras, MAS existe certas consequencias lógicas que não vai conseguir derivar!

Um exemplo de uma consequencia lógica que a resolução não consegue derivar é o seguinte: $\{A \wedge B\} \models A \vee B$

O interessante da **resolução** é o facto de formar a base para uma familia de **complete inference procedures**. “Um resolution-based theorem prover consegue, para qualquer frase A e B em logica propusicional, decidir se $A \models B$ ”

Quando junta a qualquer search algorithm que seja **complete**, a resolution consegue-nos levar a um **complete inference algorithm**

VII. Forma Normal Conjuntiva - CNF

Antes de passarmos para a **refutação por resolução** convem primeiro explicarmos quatro novos conceitos: Literais, Clausulas, Forma Clausal e Forma Normal Conjuntiva

- **Literal**
 - Podem ser **Literais Positivos** ou **Literais Negativos**
 - **Literais Positivos** – Frase (ou fórmula) atómica
 - Ex: C ; Branca ; A
 - **Literais Negativos** – Frase complexa que consiste na negação de uma frase atómica
 - Ex: $\neg C$; $\neg \text{Branca}$; $\neg A$
- **Clausula**
 - Frase complexa que consiste na **disjunção ($\vee$) de literais**
 - Ex: $C \vee \neg \text{Branca}$; $\neg C \vee A$; $A \vee \text{Branca} \vee C$

- **Forma Normal Conjuntiva – CNF (Conjunctive Normal Form)**
 - Frase complexa que consiste na **conjunção ($\wedge$) de cláusulas**
 - Ex: $(C \vee \neg \text{Branca}) \wedge (\neg C \vee A)$
- **Forma Clausal**
 - Representação de uma fórmula CNF através do conjunto das respetivas cláusulas
 - Ex:
 - $(C \vee \neg \text{Branca}) \wedge (\neg C \vee A)$ – CNF
 - $\{ C \vee \neg \text{Branca}, \neg C \vee A \}$ – Forma Clausal

Podemos facilmente converter qualquer fórmula/frase complexa proposicional para a CNF:

1. Remover implicações ($\Rightarrow$) e Bidirecionais ($\Leftrightarrow$)
2. Reduzir âmbito de aplicação das negações
3. Realizar substituições por Associação e Distribuição até obter a CNF

Ex: Passe a formula $A \Leftrightarrow (B \vee C)$ para a CNF

$$A \Leftrightarrow B \vee C$$

$$\equiv (A \Rightarrow (B \vee C)) \wedge ((B \vee C) \Rightarrow A) \text{ Definição da Bicondicional}$$

$$\equiv (\neg A \vee (B \vee C)) \wedge (\neg(B \vee C) \vee A) \text{ Definição da Implicação}$$

$$\equiv (\neg A \vee B \vee C) \wedge ((\neg B \wedge \neg C) \vee A) \text{ Leis de DeMorgan}$$

$$\equiv (\neg A \vee B \vee C) \wedge ((\neg B \vee A) \wedge (\neg C \vee A)) \text{ Distribuição}$$

$$\equiv (\neg A \vee B \vee C) \wedge (\neg B \vee A) \wedge (\neg C \vee A) \text{ CNF - Vitória :)}$$

$$\text{Forma Clausal} = \{ (\neg A \vee B \vee C), (\neg B \vee A), (\neg C \vee A) \}$$

VIII. Refutação por Resolução

A **refutação por resolução** é um *mecanismo de inferência completo* que envolve usar a **resolução** para **provar** que a **negação da consequência lógica é inconsistente com a premissa** (metateorema de redução ao absurdo)

No exemplo dado anteriormente - $\{A \wedge B\} \models A \vee B$ – vamos provar que $A \wedge B$ e $\neg(A \vee B)$ é **inconsistente**. Para tal, **basta mostrar que é possível derivar a fórmula “Falso”**

Relembre-mos a regra da resolução - $\{A \vee B, \neg B \vee C\} \vdash A \vee C$

Os passos para a refutação são:

1. Converter a premissa e a negação da consequência lógica para um conjunto de clausulas (Forma Normal Conjuntiva)
2. Aplicar a resolução até obter a **clausula** (disjunção de literais) vazia (equivalente a False)

Ex: Sabendo que $(A \Leftrightarrow (B \vee C)) \wedge \neg A$, prove $\neg B$

Primeiro vamos começar por converter o que sabemos numa CNF:

$$\begin{aligned} & (A \Leftrightarrow (B \vee C)) \wedge \neg A \\ \equiv & ((A \Rightarrow (B \vee C)) \wedge ((B \vee C) \Rightarrow A)) \wedge \neg A \text{ Definição da Bicondicional} \\ \equiv & ((\neg A \vee (B \vee C)) \wedge (\neg(B \vee C) \vee A)) \wedge \neg A \text{ Definição da Implicação} \\ \equiv & ((\neg A \vee B \vee C) \wedge ((\neg B \wedge \neg C) \vee A)) \wedge \neg A \text{ Leis de DeMorgan} \\ \equiv & ((\neg A \vee B \vee C) \wedge ((\neg B \vee A) \wedge (\neg C \vee A))) \wedge \neg A \text{ Distribuição} \\ \equiv & (\neg A \vee B \vee C) \wedge (\neg B \vee A) \wedge (\neg C \vee A) \wedge (\neg A) \text{ CNF} \\ \text{Forma Clausal} = & \{ (\neg A \vee B \vee C), (\neg B \vee A), (\neg C \vee A), (\neg A) \} \end{aligned}$$

Queremos então provar que:

$$(\neg A \vee B \vee C) \wedge (\neg B \vee A) \wedge (\neg C \vee A) \wedge (\neg A) \models \neg B$$

Para tal temos que mostrar que:

$$(\neg A \vee B \vee C) \wedge (\neg B \vee A) \wedge (\neg C \vee A) \wedge (\neg A) \text{ e } \neg \neg B (\equiv B)$$

são inconsistentes

Para tal basta verificarmos se com a **CNF** e a **Resolução**

$$(\neg A \vee B \vee C) \wedge (\neg B \vee A) \wedge (\neg C \vee A) \wedge (\neg A) \wedge \neg \neg B$$

Conseguimos chegar a alguma clausula vazia

Derivando usando a **Resolução**, $\{A \vee B, \neg B \vee C\} \vdash A \vee C$, à nossa forma causal, $\{(\neg A \vee B \vee C), (\neg B \vee A), (\neg C \vee A), \neg A, B\}$ conseguimos derivar:

$$\{(\neg B \vee A), \neg A\} \vdash \neg B$$

Note,se que $\neg A$ é equivalente a $\neg A \vee \emptyset$

Ficamos então com a CNF:

$$(\neg A \vee B \vee C) \wedge (\neg C \vee A) \wedge \neg B \wedge B$$

Aplicando outra vez a resolução de clausulas à conjunção de clausulas $\neg B \wedge B$, temos:

$$\{(\neg B \vee \emptyset), (B \vee \emptyset)\} \vdash \emptyset$$

Visto que conseguimos inferir a clausula vazia $\emptyset$ podemos concluir que a afirmação $(\neg A \vee B \vee C) \wedge (\neg B \vee A) \wedge (\neg C \vee A) \wedge (\neg A) \wedge \neg \neg B$ é

inconsistente, pelo que, por redução ao absurdo $(A \leftrightarrow (B \vee C)) \wedge \neg A \models \neg B$, ou seja, acabamos de provar que com o nosso conhecimento, $\neg B$ é necessariamente verdadeiro (sempre que o nosso conhecimento for verdadeiro)

IX. Clausulas de Horn

O facto da prova baseado na refutação por resolução ser um mecanismo completo e correto é bastante nice, mas merda é o facto de ser um bocado ineficiente – é NP-Complete (ou seja, é um problema de decisão cuja solução pode ser verificada em tempo **polinomial**)

Muitas vezes não precisamos do poder completo que a resolução nos dá. Muitas knowledge bases satisfazem certas **restrições** na forma de frases que contêm, pelo que podemos utilizar mecanismos mais restritos e eficientes!

Uma destas restrições é a chamada **Horn Clause**.

Uma **Horn Clause** é uma **disjunção de literais** (ou seja, uma *clausula*) dos quais **no máximo apenas um é positivo**.

Alguns exemplos incluem: A , $(\neg A \vee B)$, $(\neg A \vee B \vee \neg C)$, $(\neg A \vee \neg B)$

Existem algoritmos de dedução baseados em clausulas de Horn cuja complexidade temporal é linear.

É também notável o facto das linguagens Prolog e Mercury (que vamos falar mais à frente) se baseiem em clausulas de Horn

Knowledge bases que só contenham **clausulas de definitivas** (clausulas de Horn que possuem exatamente um literal positivo) são interessantes por 3 razões:

1. Qualquer Definite Clause pode ser escrita como sendo uma implicação cuja premissa é a conjunção de literais positivos, e cuja conclusão é um único literal positivo

- a. P.ex $(\neg A \vee B)$ pode ser escrita como $(A \Rightarrow B)$
- 2. Inferencia com Horn Clauses pode ser feita com algoritmos de forward chaining ou backward chaining
- 3. Entailment com Horn clauses pode ser feito em linear time (relativamente à knowledge base) :D!

Lógica de Primeira Ordem

I. Lógica de Primeira Ordem – Sintaxe e Definição

Podemos pensar na ***lógica de primeira ordem*** como um incremento à ***lógica proposicional*** que a mistura com elementos de ***linguagens naturais***.

Como tal em lógica de primeira ordem temos, primeiramente:

- **Objetos ou Entidades**

- Uma entidade que faz parte do nosso modelo (pelo que vai ter de ser representada)
- Cada entidade pode possuir **propriedades**
- Estas entidades podem ter **relações** entre si
- O **domínio do modelo** é o conjunto de todos os objetos do modelo
- Ex:
 - Modelo: “O lindo Rey Edu andou em Aveiro”
 - **Objetos:** Edu, Aveiro
 - **Propriedades:** Edu – {lindo, Rey}
 - **Relações:** Andou(Aveiro,Edu)

- **Expressões Funcionais**

- Funções expressam relações obrigatórias de um objeto.
- Podemos considerar objetos como sendo funções de aridade (# de argumentos) 0
- Ex:
 - Braço(Edu) – Função que apontaria para o objeto que corresponde ao braço do edu
 - Potencia(4,3) – função que apontaria para o valor de 4^3

- **Relações ou Predicados**

- Ligação entre dois objetos
- Correspondem às frases elementares da lógica proposicional
- Ex:
 - $\text{Pai}(\text{Diogo}, \text{Edu})$ – Relação que exprime que o pai do Diogo é o Edu
 - Em logica proposicional teriamos a frase atômica P – O pai do Diogo é o Edu
 - $\text{Mae}(\text{Amina}, \text{Pai}(\text{Diogo}, \text{Edu}))$ – Relação que exprime que a mãe do pai do Diogo (o Edu) é a Amina
 - Em logica proposicional teriamos a frase atômica M – A mãe do Pai do Diogo é a Amina

Para além destes elementos devemos também defenir a ideia de **Termos**

- **Terms ou Termos**

- Expressão lógica que se refira a um objeto
- **Objetos, funções** e os **argumentos de um predicado são** Termos
- Ex:
 - Edu é um termo
 - $\text{Braço}(\text{Edu})$ – $\text{Braço}()$ e Edu são ambos termos
 - $\text{Pai}(\text{Diogo}, \text{Edu})$ – Diogo e Edu são termos
- Um **Complex Term** é formado por uma função e pelo seu conjunto de argumentos
- Ex:
 - $\text{Braço}(\text{Edu})$ é um Termo Complexo

Por fim temos também **variáveis** e **quantificadores**:

- **Variáveis**

- **Termos não especificados** usados nos argumentos dos predicados
- Ex:
 - $\text{Pai}(x)$ – x é uma variável usado para representar um termo não especificado

- **Quantificador Universal**

- Representa-se por $\forall$ e le-se “qualquer que seja”
- Ex:
 - $\forall x$ – “Qualquer que seja x ”
- Usa-se em conjunto com fórmulas, p.ex:
 - $\forall x A$ – “Qualquer que seja x , a fórmula A é verdadeira”

- **Quantificador Existencial**

- Representa-se por $\exists$ e le-se “existe pelo menos um”
- Ex:
 - $\exists x$ – “Existe pelo menos um x ”
- Usa-se em conjunto com fórmulas, p.ex:
 - $\exists x A$ – “Existe pelo menos um x tal que a fórmula A seja verdadeira”

Para terminar, vamos só dar alguns exemplos da transformação de frases em linguagem natural em lógica de 1ª ordem:

Todos os que estudam em Oxford são espertos

Objetos: Oxford, “Todos” – Que vamos representar pela variável x

Relações: Estuda : $\{ \langle x, \text{Oxford} \rangle \}$, Esperto : $\{ \langle x \rangle \}$

Frase: $\forall x \text{ Estuda}(x, \text{Oxford}) \Rightarrow \text{Esperto}(x)$

Note-se que se usássemos $\forall x \text{ Estuda}(x, \text{Oxford}) \wedge \text{Esperto}(x)$ estaríamos a dizer “Todos os que estudam em Oxford são espertos”

Alguem que estuda em Aveiro é esperto

Objetos: Aveiro, “Alguem” – Que vamos representar pela variável x

Relações: Estuda : $\{ \langle x, \text{Aveiro} \rangle \}$, Esperto : $\{ \langle x \rangle \}$

Frase: $\exists x \text{ Estuda}(x, \text{Aveiro}) \wedge \text{Esperto}(x)$

Note-se que se usássemos $\exists x \text{ Estuda}(x, \text{Aveiro}) \Rightarrow \text{Esperto}(x)$ estaríamos a dizer “Se alguém estuda em Aveiro, então é esperto” pelo que teríamos interpretações verdadeiras para estudantes que não fossem de aveiro

Rule of Thumb: O conector natural a usar com $\forall$ é o $\Rightarrow$, enquanto que para o $\exists$ é o $\wedge$

Existe alguém que gosta de toda a gente

Objetos: “Alguem” que vamos representar por x , “Toda a gente” que vamos representar por y

Relações: Gosta

Frase: $\exists x \forall y \text{ Gosta}(x, y)$

Toda a gente gosta de alguém

Objetos: “Toda a gente” que vamos representar por x , “Alguem” que vamos representar por y

Relações: Gosta

Frase: $\forall x \exists y \text{ Gosta}(x, y)$

Existe alguém de quem toda a gente gosta

Objetos: “Toda a gente” que vamos representar por x e “Alguem” que vamos representar por y,

Relações: Gosta

Frase: $\exists y \forall x \text{ Gosta}(x,y)$

Nota: Estes ultimos dois exemplos mostram que a ordem pela qual os quantificadores aparecem é importante para o significado da frase

Para terminar, temos então a gramática completa para lógica de primeira ordem:

$$\begin{aligned} \text{Sentence} &\rightarrow \text{AtomicSentence} \mid \text{ComplexSentence} \\ \text{AtomicSentence} &\rightarrow \text{Predicate} \mid \text{Predicate}(\text{Term}, \dots) \mid \text{Term} = \text{Term} \\ \text{ComplexSentence} &\rightarrow (\text{Sentence}) \mid [\text{Sentence}] \\ &\mid \neg \text{Sentence} \\ &\mid \text{Sentence} \wedge \text{Sentence} \\ &\mid \text{Sentence} \vee \text{Sentence} \\ &\mid \text{Sentence} \Rightarrow \text{Sentence} \\ &\mid \text{Sentence} \Leftrightarrow \text{Sentence} \\ &\mid \text{Quantifier Variable}, \dots \text{Sentence} \\ \\ \text{Term} &\rightarrow \text{Function}(\text{Term}, \dots) \\ &\mid \text{Constant} \\ &\mid \text{Variable} \\ \\ \text{Quantifier} &\rightarrow \forall \mid \exists \\ \text{Constant} &\rightarrow A \mid X_1 \mid \text{John} \mid \dots \\ \text{Variable} &\rightarrow a \mid x \mid s \mid \dots \\ \text{Predicate} &\rightarrow \text{True} \mid \text{False} \mid \text{After} \mid \text{Loves} \mid \text{Raining} \mid \dots \\ \text{Function} &\rightarrow \text{Mother} \mid \text{LeftLeg} \mid \dots \\ \\ \text{OPERATOR PRECEDENCE} &: \neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow \end{aligned}$$

Img 1.1 – Gramática da Lógica de 1ª Ordem.

II. Interpretação em Lógica de Primeira Ordem

Ao contrario da interpretação em Lógica Proposicional, em Lógica de 1ª Ordem, uma **interpretação** é o **estabelecimento** de uma **correspondencia entre as várias constantes** que ocorrem na fórmula e os **objetos** do mundo, **funções** e **relações** que essas constantes representam

(Note-se que por constantes estamos-nos a referir aos simbolos que utilizamos para representar os diferentes elementos de lógica de 1ª ordem)

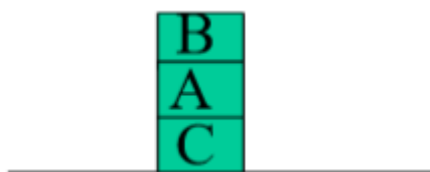
Por exemplo:

Objetos: A, B, C, Chão

Relações: EmCimaDe : {<B,A>, <A,C>, <C,Chão>} ; Livre : {}

Frase: $\text{EmCimaDe}(B,A) \wedge \text{EmCimaDe}(A,C) \wedge \text{EmCimaDe}(C,\text{Chao}) \wedge \text{Livre}(B)$

Podemos ter a interpretação:



Img 2.1 – Possível interpretação da nossa frase. Esta em específico, corresponde a um modelo

Esta interpretação corresponde a um **modelo** pois satisfaz a nossa fórmula

III. Regras de Substituição em Lógica de 1ª Ordem

Para além das leis de substituição que tínhamos já definidos para lógica proposicional:

Leis de DeMorgan:

- $\neg (A \wedge B) \equiv \neg A \vee \neg B$
- $\neg (A \vee B) \equiv \neg A \wedge \neg B$
- $A \wedge B \equiv \neg (\neg A \vee \neg B)$
- $A \vee B \equiv \neg (\neg A \wedge \neg B)$

Associação:

- $A \wedge B \wedge C \equiv A \wedge (B \wedge C)$
- $A \vee B \vee C \equiv A \vee (B \vee C)$

Distribuição:

- $A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$
- $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$

Temos também:

Leis de DeMorgan Generalizadas:

- $\neg (\forall x P(x)) \equiv \exists x \neg P(x)$
- $\neg (\exists x P(x)) \equiv \forall x \neg P(x)$
- $\forall x P(x) \equiv \neg \exists x \neg P(x)$
- $\exists x P(x) \equiv \neg \forall x \neg P(x)$

Definição da Bicondicional:

- $A \Leftrightarrow B \equiv (A \Rightarrow B) \vee (B \Rightarrow A)$

Definição da implicação:

- $A \Rightarrow B \equiv \neg A \vee B$

Comutação:

- $A \wedge B \equiv B \wedge A$
- $A \vee B \equiv B \vee A$

Transposição:

- $A \Rightarrow B \equiv \neg B \Rightarrow \neg A$

Dupla negação:

- $\neg \neg A \equiv A$

IV. Regras de Inferência em Lógica de 1ª Ordem

Para além das regras de inferencia que defenimos no capitulo de Logica Proposicional:

Modus Ponens: $\{ A, A \Rightarrow B \} \vdash B$

Modus Tolens: $\{ \neg B, A \Rightarrow B \} \vdash \neg A$

Silogismo hipotético: $\{ A \Rightarrow B, B \Rightarrow C \} \vdash A \Rightarrow C$

Conjunção: $\{ A, B \} \vdash A \wedge B$

Eliminação da conjunção: $\{ A \wedge B \} \vdash A$

Disjunção: $\{ A, B \} \vdash A \vee B$

Silogismo disjuntivo (ou resolução unitária):

$$\{ A \vee B, \neg B \} \vdash A$$

Resolução: $\{ A \vee B, \neg B \vee C \} \vdash A \vee C$

Dilema construtivo:

$$\{ (A \Rightarrow B) \wedge (C \Rightarrow D), A \vee C \} \vdash B \vee D$$

Dilema destrutivo:

$$\{ (A \Rightarrow B) \wedge (C \Rightarrow D), \neg B \vee \neg D \} \vdash \neg A \vee \neg C$$

Img 4.1 – Regras de Inferencia em Logica Proposicional

Temos também:

Instanciação universal:

$$\{ \forall x P(x) \} \vdash P(A)$$

Generalização existencial

$$\{ P(A) \} \vdash \exists x P(x)$$

Img 4.2 – Duas regras de inferencia adicionais exclusivas a Logica de 1ª Ordem

V. Forma Normal Conjuntiva – CNF em Lógica de 1ª Ordem

A conversão de uma formula para a Forma Normal Conjuntiva em Lógica de Primeira Ordem é ligeiramente diferente e envolve a eliminação dos quantificadores.

Temos então o algoritmo que está dividido em 3 fases:

1ª Fase – Preparação da fórmula

Esta fase envolve a aplicação das regras de transformação já definidas e consiste nos seguintes passos:

1. Renomear variáveis de forma a que cada quantificador tenha uma variável diferente
2. Remover as Implicações
3. Reduzir o ambito das negações (aplicar a negação)

2ª Fase – Remoção dos Quantificadores

Esta fase consiste na remoção dos quantificadores existenciais – através de **Skolemização** – e dos universais

1. Skolemização
 - a. Substituir todas as ocorrencias de cada variavel quantificada existentialmente por uma função cujos argumentos são variáveis dos quantificadores universais exteriores
2. Remover quantificadores universais

3ª Fase – Conversão em CNF

Nesta fase vamos aplicar os mesmos procedimentos que aplicavamos para a converter uma frase de lógica proposicional para CNF.

Para além disso temos também que renomear as variáveis de forma a que elas não apareçam em mais do que uma fórmula (tbh i dont really get why we have to do this, but ok)

Vamos ver um exemplo para melhor compreender este algoritmo:

Ex: Passe a formula $\forall x \forall y \neg(p(x,y) \Rightarrow \forall y q(y,y))$ para a CNF

$$\begin{aligned}
 & \forall x \forall y \neg(p(x,y) \Rightarrow \forall y q(y,y)) \\
 & \equiv \forall a \forall b \neg(p(a,b) \Rightarrow \forall c q(c,c)) \quad \text{Fase 1: Renomear Variáveis} \\
 & \equiv \forall a \forall b \neg(\neg p(a,b) \vee \forall c q(c,c)) \quad \text{Fase 1: Remover Implicação} \\
 & \equiv \forall a \forall b (\neg \neg p(a,b) \wedge \neg(\forall c q(c,c))) \quad \text{Fase 1: Lei de DeMorgan} \\
 & \equiv \forall a \forall b (p(a,b) \wedge \exists c \neg q(c,c)) \quad \text{Fase 1: Aplicação da negação} \\
 & \equiv \forall a \forall b (p(a,b) \wedge \neg q(f(a,b), f(a,b))) \quad \text{Fase 2: Skolemização – Trocamos} \\
 & \quad \text{todas as instancias da variavel c por uma função das variaveis a e b} \\
 & \equiv p(a,b) \wedge \neg q(f(a,b), f(a,b)) \quad \text{Fase 2: Remover Quantificadores Universais} \\
 & \equiv p(a,b) \wedge \neg q(f(a,b), f(a,b)) \quad \text{CNF - Vitória :) (Não foi precisa Fase 3)} \\
 & \text{Forma Clausal} = \{ p(a1,b1), \neg q(f(a2,b2), f(a2,b2)) \} \quad \text{(Fase 3:} \\
 & \quad \text{Renomear variaveis para que não apareçam em duas clausulas diferentes)}
 \end{aligned}$$

VI. Substituição e Unificação

A Lógica de 1ª Ordem possui duas operações exclusivas:
Substituição e Unificação

- **Substituição**

- Denota-mos a aplicação da substituição $s = \{ t_1/x_1, \dots, t_n/x_n \}$ por **SUBST(W,s)** ou **Ws**
- Significa que todas as ocorrências das variáveis $x_1, \dots, x_n$ em W vão ser substituídas pelos termos $t_1, \dots, t_n$

- **Unificação**

- Duas formulas A e B dizem-se **unificáveis** se existir uma substituição **s** tal que **As = Bs**
- Nesse caso dizemos que s é uma substituição unificadora
- À mais simples (menos extens) que permite a unificação chamamos de **substituição unificadora mais geral** (ou **minimal**)

VII. Refutação e Resolução na Lógica de 1ª Ordem

Esta lógica possui também a regra de inferencia **Resolução**, que pode ser escrita como: $\{A \vee B, \neg C \vee D\} \vdash \text{SUBST}(A \vee D, g)$, sendo B e C unificáveis com g a sua substituição unificadora mais geral.

Note-se que esta versão alterada da resolução normal - $\{A \vee B, \neg B \vee C\} \vdash A \vee C$ – vem do facto que, se B e C são unificáveis com g a ser a sua substituição unificadora mais geral, então efetivamente fazendo a substituição em $A \vee D$ usando g, vamos estar a tornar a B e C na mesma fórmula

A regra da resolução é novamente correta mas não completa

Porém tal como na lógica proposicional, podemos também aqui realizar a refutação por resolução – sendo este mecanismo completa... hurray :)

Linguagem KIF

I. Definição

A **KIF** – Knowledge Interchange Format – é uma **linguagem desenhada para representar o conhecimento trocado entre agentes**

A motivação que deu origem à criação do KIF é parecida à que deu origem a outros formatos de representação como o PostScript, ou seja, não foi desenhado primeiramente como uma linguagem para guardar e manipular documentos, mas sim como um interchange format para que sistemas e devices pudessem partilhar documentos.

O **objetivo** principal do **KIF** é portanto **facilitar a partilha de conhecimentos entre diferentes sistemas** (ou agentes), cada um podendo usar a sua própria linguagem, formalismos, plataformas, etc...

Para além disso, a **KIF** pode também ser **usada para representar os modelos internos de cada agente** (cus its a knowledge representation language afterall)

As características do KIF incluem:

- **Semântica puramente declarativa**
 - O Prolog também é uma linguagem declarativa, mas a semântica depende em parte do modelo de inferencia
- **Pode ser tão ou mais expressiva quanto a lógica de primeira ordem**
- **Permite a representação de meta-conhecimento**
 - Ou seja conhecimento sobre conhecimento

II. Características Gerais

No **KIF** o **mundo** é **conceptualizado** em **termos de objetos e relações entre objetos**.

Ou seja, o KIF usa semanticas declarativas com o proposito de tentar descrever factos sobre o mundo (ao inves de tentar descrever processos ou procedures).

Uma **relação** é um **conjunto arbitrário de listas de objetos**

Ex: Se a relação $<$ significar que $a < b \Rightarrow a$ menor que b , então a *relação $<$* é o conjunto de todos os pares de objetos (x,y) nos quais $x < y$

O **universo de discurso** é o **conjunto** de todos os **objetos** cuja **existência é conhecida, presumida ou suposta**

Os **objetos** podem ser:

- **Concretos** ou **Abstratos**
- **Primitivos** (não decomponíveis i.e não podem ser decompostos) ou **Compostos**

III. Componentes da Linguagem

Em KIF temos:

- **Caracteres**
- **Lexemas**
 - **Lexemas Especiais**
 - Que têm um papel pre definido na própria linguagem
 - **Palavras**
 - **Códigos de caracteres**

- **Blocos de código de caracteres**
- **Cadeias de Caracteres**
- **Expressões**
 - **Termos**
 - Objetos primitivos ou compostos
 - Podem ser:
 - Constantes
 - Variáveis Individuais
 - Expressões Funcionais
 - (*functor arg1 .. argn*)
 - (*functor arg1 .. argn seqvar*)
 - Lista
 - (*listof t1 ... tn*)
 - Termo Lógico
 - (*if c1 t1 ... cn tn default*)
 - Código de caracter, bloco de códigos de caracteres e cadeia de caracteres
 - Citação (Quotation)
 - (*quote lista*) ou *'lista*
 - **Frases**
 - Expressões com valor lógico
 - Podem ser:
 - Constantes
 - Equações
 - (*= termo1 termo2*)
 - Inequações
 - (*!= termo1 termo2*)
 - Frases Relacional
 - (*relação t1 .. tn*)
 - Frases Lógicas construídas com as conectivas lógicas ('not', 'and', 'or', '=>', '<=', '↔')
 - Frases Quantificadas
 - (*forall var1 ... varn frase*)
 - (*exists var1 ... varn frase*)

- **Definições**
 - Frases verdadeiras por definição
 - Podem ser:
 - **Definição de Objetos**
 - Igualdade – (defobject s := t)
 - Ex: (defobject nil := (listof))
 - Conjunção – (defobject s p1 .. pn)
 - Etc...
 - **Definição de Funções**
 - (deffunction f(v1 .. vn) := t)
 - Ex:
 - (defunction head (?l := (if (= (listof ?x @items) ?1) ?x)
 - **Definição de Relações (ou predicados)**
 - (defrelation r(v1 .. vn) := p)
 - Etc...
 - Ex:
 - (defrelation null (?1) := (= ?1 (listof)))
 - (defrelation list (=x) := (exists (@1) (= ?x (listof (@1)))))

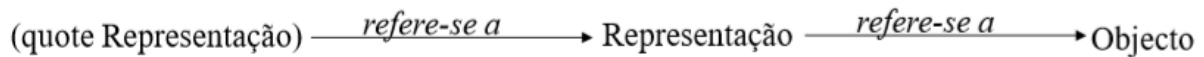
IV. Meta-Conhecimento

Pode **formalizar-se conhecimento** sobre **conhecimento**

O mecanismo da citação (quotation) permite tratar expressões como objetos

Por exemplo, a ocorrência da palavra *joão* numa expressão designará uma pessoa, entretanto a expressão (quote *joão*) ou '*joão*' designa a própria palavra *joão* e não o objeto ou pessoa a que ele se refere.

Graficamente isto pode ser representado da seguinte forma:



Img 4.1 – Representação de meta conhecimento

V. Dimensões de Conformação

O KIF é uma linguagem altamente expressiva, porém tende a sobrecarregar os sistemas de geração e inferência

Para melhorar isso, foram definidas várias **dimensões de conformação**.

Um **perfil de conformação** é uma **seleção** de **níveis** de **conformação** para cada uma das **dimensões** referidas

Foram definidos os seguintes perfis de conformação:

- Lógica
 - *Atômica, conjuntiva, positiva, lógica, baseada em regras* (de horn ou não, recursivas ou não)
- Complexidade dos termos
 - *Termos simples* (constantes e variáveis), *termos complexos*
- Ordem
 - *Proposicional, primeira ordem* (contem variáveis, mas os funtores e as relações são constantes), *ordem superior* (os funtores e relações podem ser variáveis)
- Quantificação
 - Conforme se usa ou não
- Meta conhecimento
 - Conforme se usa ou não

Ontologia Geral

I. Engenharia do Conhecimento

Quando falamos em lógica referimos a existencia de algo a que chamamos Knowledge Base (KB ou BC – Base de conhecimento). Para relembrar, uma **KB é um conjunto de representações de factos e regras de funcionamento do mundo**. Alternativamente, podemos generalizar factos e regras com a designação **frases**

Engenharia do conhecimento é portanto o processo de construção de knowledge bases

Engenharia do conhecimento, portanto, engloba:

- **Estudar o dominio da aplicação, frequentemente através de entrevistas com peritos**
 - Processo de aquisição de conhecimento
- **Determinar os objetos, conceitos e relações que será necessario representar**
- **Escolher um vocabulário para entidades, funções e relações**
 - Ou seja, arranjar forma de representar estes elementos
 - A isto chamamos **Ontologia**
- **Codificar conhecimento genérico sobre o domínio**
 - Este passo vai nos permitir definir um conjunto de **axiomas** (formulas que são verdadeiras e que não dependem de outras)

- **Codificar descrições para problemas concretos, interrogar o sistema e obter respostas**

Para domínios demasiado complexos, não é boa ideia tentar representar e codificar à mão todo o conhecimento necessário. Nesses casos utiliza-se **aprendizagem automática**

II. Identificação de objetos, conceitos e relações

Quando procuramos modelar um domínio de conhecimento, podemos facilmente reparar que algumas das palavras que usamos para o descrever em linguagem natural se rendem facilmente à fácil identificação de conceitos, objetos e relações:

- **Substantivos Comuns**
 - **Conceitos** (tambem chamados de classes ou tipos)
- **Substantivos Próprios**
 - **Objetos** (tambem chamados de instancias)
- **Verbo “ser”**
 - **Relação de instanciação** (entre objeto e tipo) ou
 - **Relação de generalização** (entre subtipo e tipo)
- **Verbos “ter” e “conter”**
 - Podem indicar uma relação de composição
- **Outros verbos**
 - Podem sugerir outras relações relevantes

Por exemplo, dado o domínio:

“O **DS** é uma peessoa (que é um tipo de animal). Para além disso, o DS é Português e *tem* um **computador** e *gosta* de jogos”

Temos:

- Objetos: {DS, Computador}
- Tipos: {animal, Português, jogos}
- Subtipos: {pessoa}
- Relações:
 - De Instanciação: {ser<DS,Pessoa>, ser<DS,Português>}
 - De Generalização: {subtipo<pessoa,animal>}
 - Composição: {tem<DS,Computador>}
 - {gosta<DS,jogos>}

Note-se que Deixei a **bold** os proprios proprios (DS, computador), a underline os substantivos comuns (pessoa, animal, Portugues, jogos) e a *itálico* os verbos (é, tem, gosta)

Quando fazemos este tipo de análise do domínio devemos de manter em mente a relevancia dos objetos, conceitos e relações que identificamos para o problema em questão. Muitas vezes o domínio pode estar cheio de merda que é desnecessária para o que queremos.

Por exemplo, se o problema relacionado com o domínio anterior não tivesse nada a ver com nacionalidades, dizer que o DS é Português é desnecessário!

Devemos então:

- Não devemos considerar substantivos que identifiquem objetos, conceitos ou relações irrelevantes para o problema
- Quando vários substantivos aparecem a referir-se ao mesmo conceito, devemos escolher o mais representativo ou adequado

Para simplificar podemos tambem criar **conceitos mais abstratos**:

Um conceito mais abstrato pode ser criado atribuindo-lhe o que é comum a outros dois ou mais conceitos previamente identificados.

III. Ontological Engineering

Uma **ontologia** é um **vocabulário sobre um domínio conjugado com relações hierárquicas** como membro e subtipo (pode eventualmente haver outras)

O objetivo de uma ontologia é captar a essência da organização do conhecimento num domínio

Ontological Engineering é então a forma como vamos representar conceitos abstratos do domínio.

A Ontologia Geral resultante de Ontological Engineering é aplicável a uma grande variedade de domínios de aplicação e, como tal, deve envolver as seguintes noções:

- **Categorias**
 - A.k.a **Tipos** ou **Classes**
 - Basicamente são como as classes das linguagens OOP
 - Abstrações que ligam objetos que partilham propriedades em comum
 - Podemos ainda ter **Subcategorias** (ou SubTipos) que acrescentam mais granularidade à nossa classe original
 - P.ex Mamífero e Reptil podem ser duas subcategorias da categoria Animal
 - Subcategorias ligam-se a Categorias através de relações de **herança**
 - Para além disto podemos ainda realizar, uma operação de **Reify** sobre uma classe. Ou seja, transformar essa classe num Objeto
 - Em Lógica de 1ª Ordem, p.ex, podemos representar categorias como predicados ou objetos
 - Ex Podemos ter a categoria Basketballs e usar o predicado Basketball(b) ou ter o Objeto

Basketballs e usar o predicado
Member(b,Basketballs)

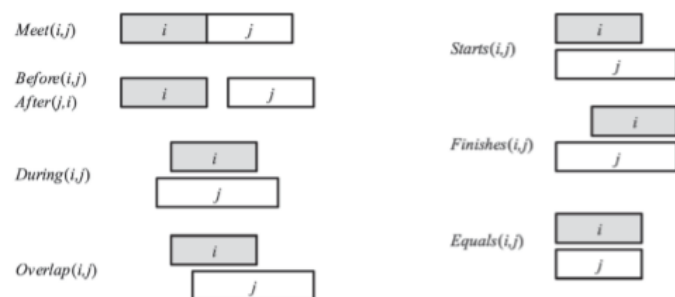
- Podemos ainda comparar categorias pelos membros:
 - Dizemos que duas categorias são **disjuntas** se não tiverem membros em comuns
 - Ex. Disjoint({Mamiferos, Repteis})
 - Dizemos que duas categorias constituem uma **decomposição exaustiva** se um membro não pertencer a uma, obrigatoriamente pertence à outra
 - Ex. ExhaustiveDecomposition({Americans, Canadians}, NorthAmericans)
 - Note-se que isto não é uma Partition porque pode haver pessoas com dupla nacionalidade
 - Note-se ainda que uma **disjoint exhaustive decomposition** é conhecida como uma **partição**
 - Ex. Partition({Male,Female}, Animals)
-
- **Objetos Compostos**
 - Objetos identificaveis por **pertencerem** a uma **categoria** que se **caracterize** por um **conjunto** de **relações estruturais entre** várias **partes**
 - P.ex Um objeto que pertence à categoria Bipede é caracterizado por ter duas pernas ligadas a um corpo
-
- **Medidas**
 - Valores aos quais damos propriedades (como unidades)
 - P.ex comprimentos, pesos, quantias de dinheiro, etc...

- Podemos ter **medidas quantitativas** (como peso) ou **medidas que não são representáveis** com números (como dificuldade ou beleza)
- Note-se que podemos comparar medidas não numéricas através de ordenações (com $>$ e $<$)
 - P.ex para representar que a dificuldade do exercício a é mais difícil que o b dizemos
 - $\text{Difficulty}(a) > \text{Difficulty}(b)$
 - Repara que seria ingenuo atribuímos valores numéricos à dificuldade
- **Objetos Físicos**
 - Objetos físicos podem ser artefactos (i.e alguma coisa concreta) ou substâncias (sólido, líquido, gás)
 -
 - Podemos ter dois tipos de objetos físicos:
 - **Things**
 - **Stuff**
 - As **Things** possuem a característica da **individualidade**. Basically para haver distinção entre estes objetos eles têm que ter alguma coisa que os distinga dos outros
 - **Stuff** porém, desafiam esta característica
 - A distinção entre estas duas classificações é mais facilmente explicável pela ideia de nomes contáveis e não contáveis
 - Ex. Coelho e Açúcar
 - Podemos dizer: 1 Coelho, 2 Coelhos, ...
 - É uma **Thing**
 - Dizer: 1 Açúcar, 2 Açúcars ... não faz sentido
 - É uma **Stuff**
 - Temos também que distinguir entre propriedades **intrínsecas** – característica partilhada por todos os objetos de uma classe – e **extrínsecas** – características individuais de cada objeto

- Quando dividimos uma **Thing** a meio, esta vai perder propriedades intrínsecas
 - P.ex se cortarmos um coelho a meio, vai perder a propriedade de estar vivo...sad
 - Mas quando dividimos **Stuff** a meio, estas não vão perder as propriedades intrínsecas (podem só modificar as extrínsecas)
 - Se cortarmos açúcar a meio, continuamos com açúcar (talvez com menos peso though)
- Por fim convém também mencionar que para podermos representar **stuff** precisamos de ter como objetos na nossa ontologia “lumps” ou **pedaços** dessa stuff.
- P.ex podemos dizer que certo pedaço de açúcar é o nosso objeto, mas temos também que definir a classe Açúcar para podermos dizer que esse pedaço de açúcar (que se vai comportar como um objeto normal) faz parte e tem certas categorias em comum com outros pedaços de açúcar (como melting point ou o facto de ser dissolvível)
-
- **Tempo, Espaço Temporal e Mudanças**
 - Para falarmos de eventos temos de primeiro falar do conceito de tempo.
 - E para falarmos de tempo temos que fazer de **espaço temporal**.
 - Temos dois tipos de **time intervals**:
 - **Moments**
 - Intervalo de tempo com duração 0
 - **Extended Intervals**
 - Intervalo de tempo com determinada duração
 - Note-e que a escala de tempo que usamos é arbitrária

- Dois intervalos encontram-se se o end time do primeiro for igual ao start time do segundo.
- Alias, temos varios tipos de relações entre intervalos:

$Meet(i, j)$	$\Leftrightarrow$	$End(i) = Begin(j)$
$Before(i, j)$	$\Leftrightarrow$	$End(i) < Begin(j)$
$After(j, i)$	$\Leftrightarrow$	$Before(i, j)$
$During(i, j)$	$\Leftrightarrow$	$Begin(j) < Begin(i) < End(i) < End(j)$
$Overlap(i, j)$	$\Leftrightarrow$	$Begin(i) < Begin(j) < End(i) < End(j)$
$Begins(i, j)$	$\Leftrightarrow$	$Begin(i) = Begin(j)$
$Finishes(i, j)$	$\Leftrightarrow$	$End(i) = End(j)$
$Equals(i, j)$	$\Leftrightarrow$	$Begin(i) = Begin(j) \wedge End(i) = End(j)$



Img 3.1 –
Relações entre
intervalos

- Quando dizemos **mudanças**, referimo-nos às mudanças que ocorrem/ocorreram num dado intervalo de tempo

● Eventos e Processos

- Um **event** é algo que sucede, ou seja, uma ação e a sua reação
- Um event pode ser descrito como instancias de **event categories**
 - P.ex o Evento E – O DS está a conduzir de Espinho até Aveiro pode ser descrito como
 - $E \in \text{Conduzir}(\text{DS}, \text{Espinho}, \text{Aveiro})$
- Para dizer que um evento ocorre num dado intervalo de tempo podemos utilizar a seguinte nomenclatura:

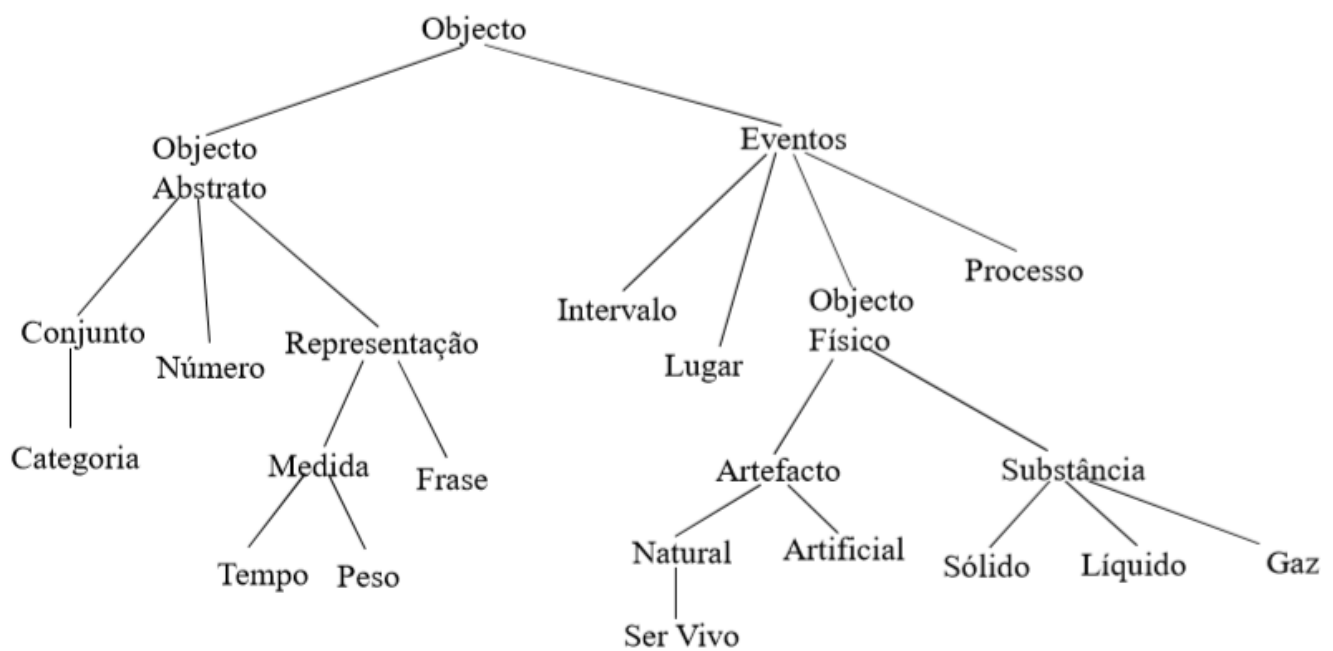
Img 3.2 – Interação entre evento e tempo. Note-se que **fluent** ou **variável de estado** é um estado do mundo sujeito a mudança

$T(f, t)$	Fluent f is true at time t
$Happens(e, i)$	Event e happens over the time interval i
$Initiates(e, f, t)$	Event e causes fluent f to start to hold at time t
$Terminates(e, f, t)$	Event e causes fluent f to cease to hold at time t
$Clipped(f, i)$	Fluent f ceases to be true at some point during time interval i
$Restored(f, i)$	Fluent f becomes true sometime during time interval i

- Um processo é nada mais nada menos que um **evento contínuo**

- **Objetos Abstratos e Crenças**

- ...



Img 3.3 – Uma possível Ontologia Geral

Redes Semánticas

I. Definição

São simples **representações gráficas do conhecimento**. A sua popularidade provém do facto que, por serem tão simples, tornam-se bastante legíveis

Redes que tenham semânticas bem definidas podem ser pensadas como sendo um tipo de representação lógica, pelo que podem ser tão expressivas quanto a lógica de 1ª ordem!

II. Representação

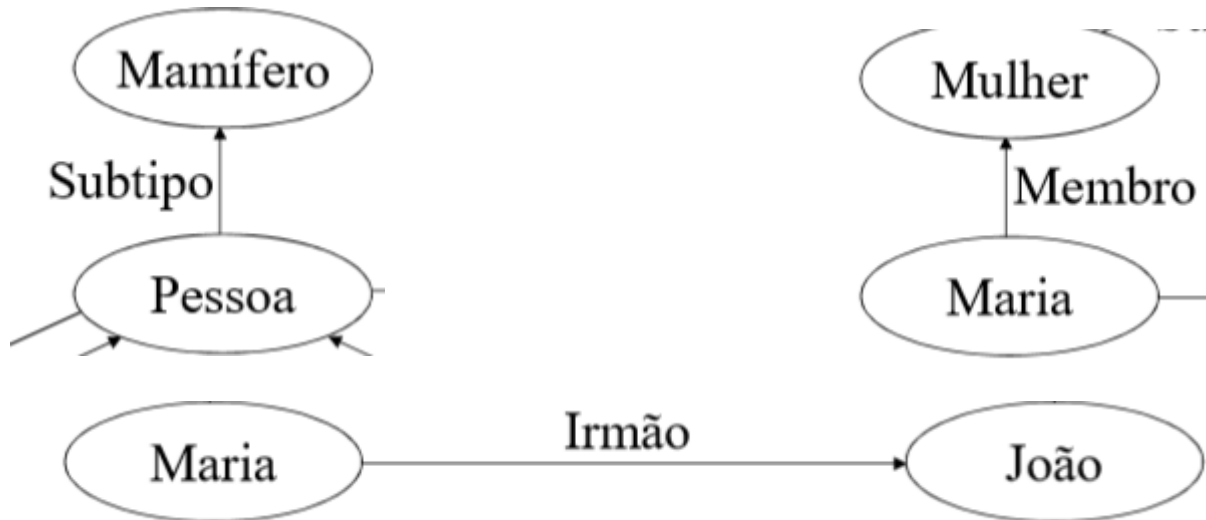
Existem vários tipos de possíveis representações de redes semânticas, mas as que vamos usar caracterizam-se por:

- **Objetos e Categorias** representados por **ovais**



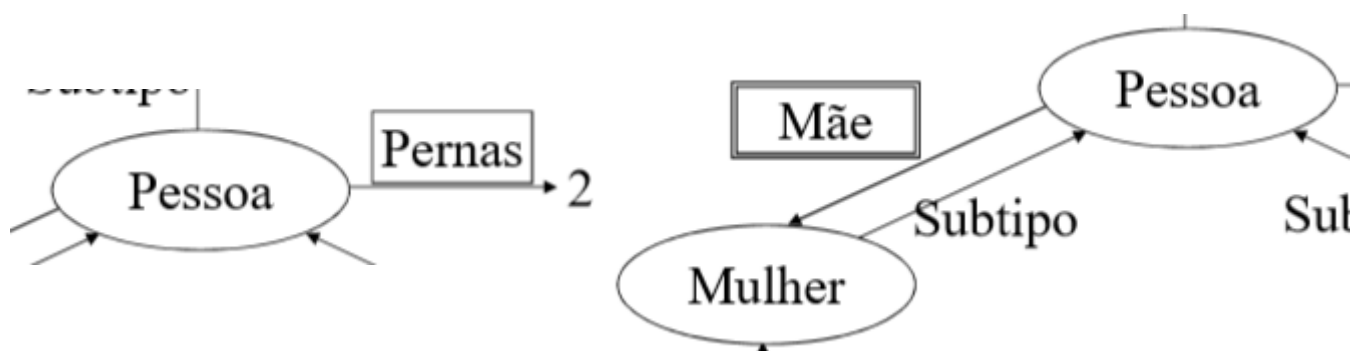
Img 2.1 – Representação de uma categoria, subcategoria e objeto

- **Relações** representados por **arestas** nomeadas entre ovals
 - Estas relações podem ser entre Objetos e Categorias
 - Ou entre Subcategorias e Categorias (**herança**)



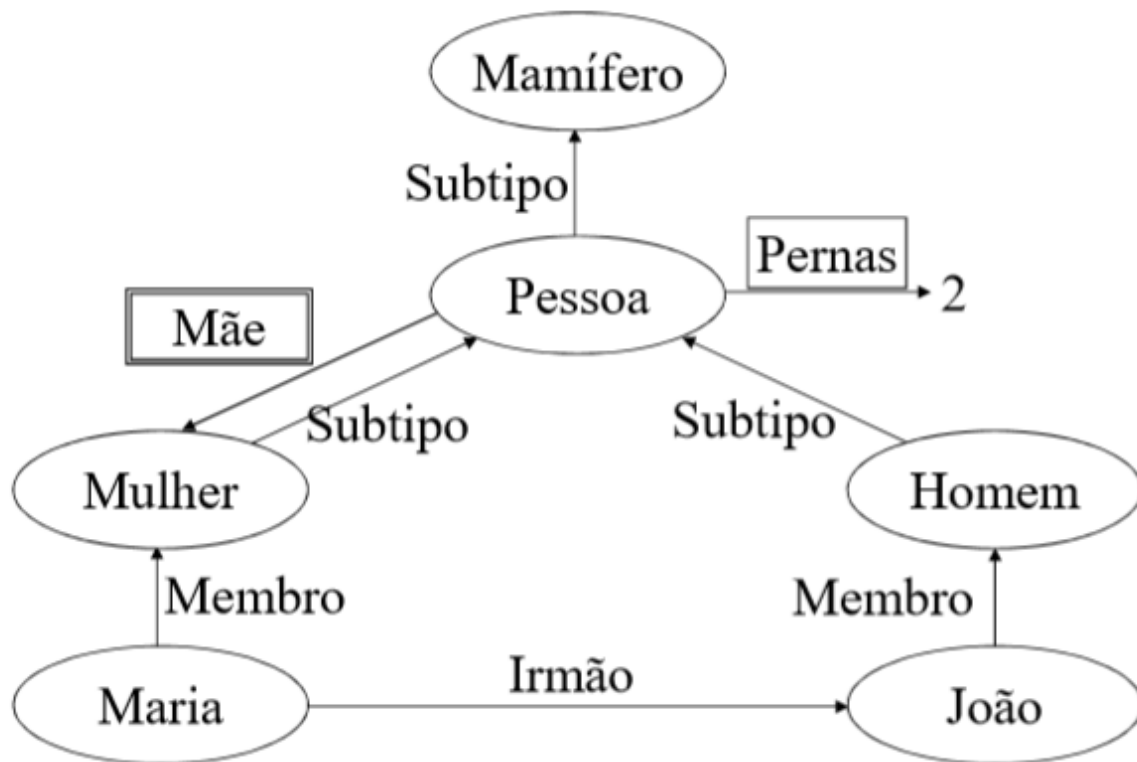
Img 2.2 – Representação de uma relação de herança, membro e entre dois objetos

- Podemos ainda representar ligações de propriedades das categorias com **double-boxed arestas**
 - Utilizado no caso em que p.ex queremos dizer que “todas as pessoas tem uma mãe”, então ligamos a categoria Pessoas à categoria “Mulher”



Img 2.3 – Representação de duas relações de propriedades da classe Pessoa (“Cada pessoa tem Mãe que é um objeto do tipo Mulher e duas pernas”)

Por fim vamos apresentar a rede semântica completa:



Img 2.4 – Rede semântica que representa o conhecimento: Pessoas são um subtipo de mamífero, mulher e homem são dois subtipos de pessoa, Maria é um objeto do tipo mulher, João é um objeto do tipo homem, o objeto João e o objeto Maria são irmãos, cada pessoa tem uma mãe que é do tipo mulher e cada pessoa tem duas pernas

III. Herança

As relações de **SUBTYPE** e **INSTANCE** permitem a herança de propriedades.

Quando um tipo é um subtipo de outros, o subtipo herda **todas as propriedades** dos tipos mais abstratos dos quais descende (podem ser múltiplos)

Quando um objeto é uma instância de um tipo, **herda todas as propriedades** à qual pertence

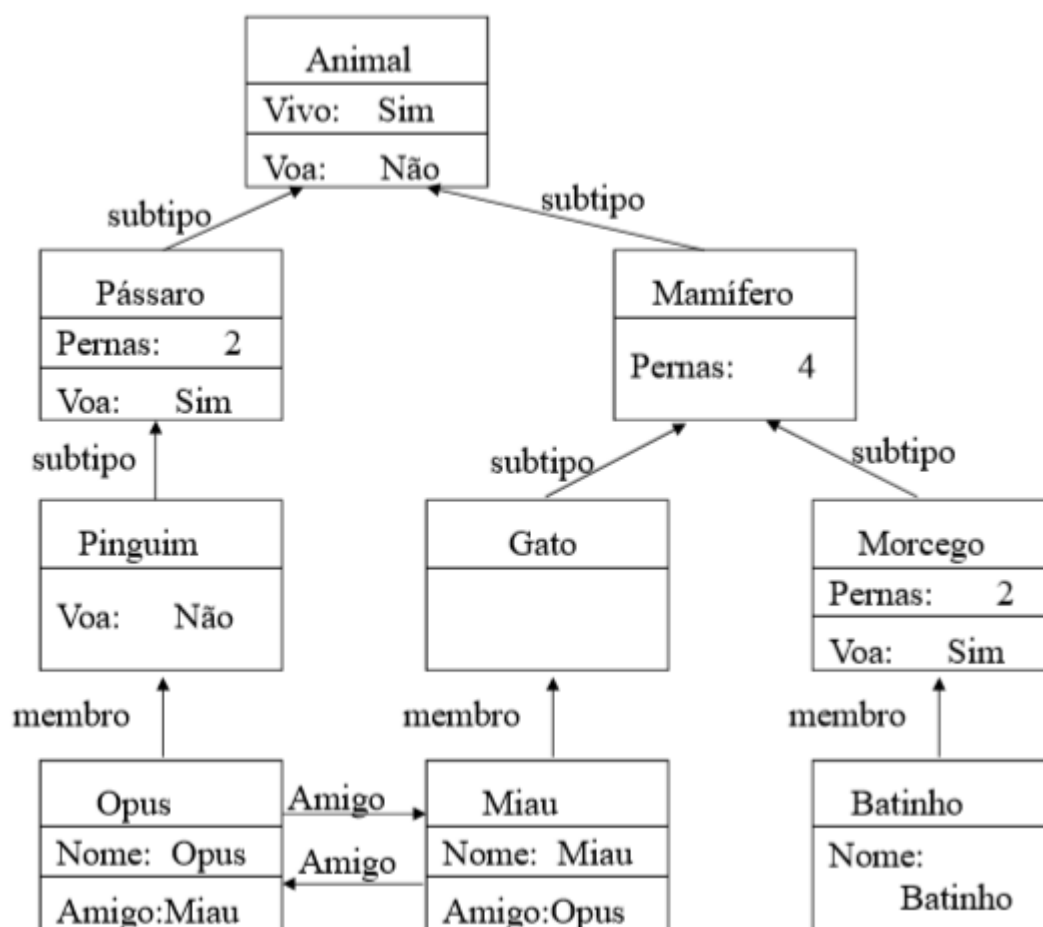
A inferência pode ser vista como o seguimento das ligações entre entidades com vista à herança de propriedades

Mas então, e se um subtipo herdar de dois tipos uma propriedade que entre em conflito? (p.ex herdar a propriedade: vivo e não vivo)

Para isso temos de estabelecer **valores por omissão** que serão os valores que cada propriedade toma caso os subtipos não realizem **cancelamento de herança**, que corresponder a dar override a esses valores por omissão

Este estabelecimento pode corresponder a uma implementação de **raciocínio não monotónico** visto que o set de crenças não cresce monotonicamente quando adicionamos novas frases à KB

Note-se que ser monotónico corresponde a uma propriedade da lógica que diz que todas as entailed sentences se mantenham entailed depois de novas frases serem adicionadas à knowledge base. Efetivamente quando fazemos override estamos a fazer exatamente o oposto



Img 2.4 – Rede semantica que representa as propriedades de cada objeto/categoria. Podemos ver, p.ex que o subtipo passaro vai dar override do valor Voa que é implementado por default no seu supertipo – animal

IV. Description Logics

A lógica de 1a ordem foi desenhada para permitir que facilmente digamos coisas sobre objetos. Porém, por razões computacionais, as Redes Semânticas tendem a usar lógicas que sejam menos expressivas do que a lógica de primeira ordem. Um exemplo destas é a **Description Logic** – Notação desenhada para tornar facil a descrição de definições e propriedades das categorias

As principais tarefas de inferencia para este tipo de lógicas são:

- **Subsumption**
 - Corresponde a verificar se uma categoria é um subset de outra através da comparação das suas definições
- **Classification**
 - Verificar se um objeto pertence a uma categoria

Note-se que nestas lógicas não existem:

- **Negação**
- **Disjunção**
- **Quantificação**

Em contra partida, nomeadamanete nos sistemas de frames, temos **Métodos e Demónios**:

- **Métodos**
 - Tem uma semantica similar à da programação orientada a objetos
- **Demónios**
 - São procedimentos cuja execução é disparada automaticamente quando certas operações de leitura ou escrita são efetuadas

Um exemplo de um sistema de frames que implementa Description Logic é o **GOLOG** desenvolvido pelo docente da cadeira – Seabra Lopes...Fucking flex dude, tinhas msm que incluir esta merda nos slides?

Muito brevemente, e só para aqueles que não têm mais nada que fazer da vida, o GOLOG é um gestor de objetos Prolog (uma linguagem de programação lógica que implementa logica de primeira ordem)

Algumas das primitivas do GOLOG incluem:

```
new_frame(Frame)
new_slot(Slot)
new_value(Frame,Slot,Value)
new_relation(Rel,Trans,Restrictions,Inv)
  • Trans ::= transitive | intransitive
  • Restrictions ::= all | none | inclusion(Slots) | exclusion(Slots)
call_method(Frame,Method,ParamList)
new_demon(Frame,Slot,Proc,Access,When,Effect).
  • Access ::= if_read | if_write | if_delete | if_execute
  • When ::= before | after
  • Effect ::= alter_value | side_effect
```

Img 4.1 – Exemplos de primitivas Golog

V. UML

Vamos fazer aqui uma tangente e falar da linguagem gráfica **UML** – Unified Modelling Language. Esta *linguagem permite **criar diagramas de classes que definem as relações existentes entre as diferentes classes de objetos** num dado domínio*, pelo que é pertinente salientar os paralelos entre o UML e as Semantic Networks

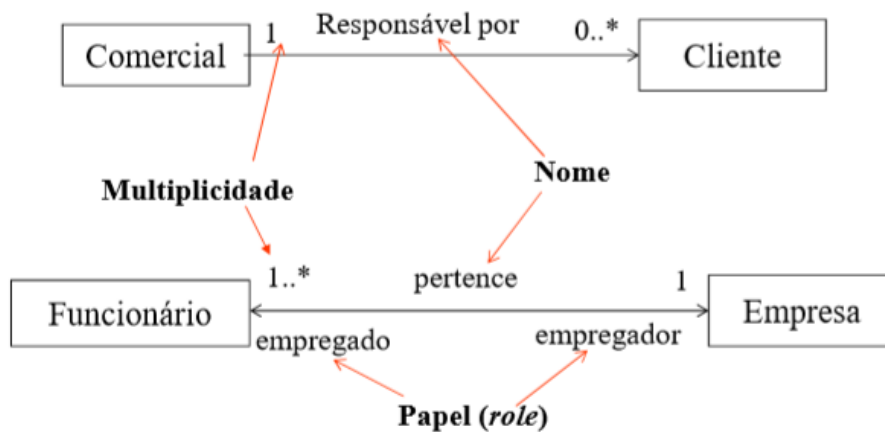
Em UML temos os seguintes elementos:

- **Classe**
 - Descrição de um conjunto de objetos que partilham os mesmos atributos, operações, relações e semantica
 - Estes objetos podem ser
 - Objetos físicos
 - Conceitos que não tenham uma existencia palpavel (substancias)
- **Atributo**
 - Propriedade de uma classe

- **Operação (ou método)**
 - Implementação de um serviço que pode ser executado por qualquer objeto da classe
- **Instância**
 - Objeto que pertence a uma classe, ou seja, constitui uma concretização da classe que está a instanciar
 - As instancias de uma classe diferenciam-se pelos valores dos atributos
 - As instancias herdam todos os atributos e metodos da sua classe

Para além destes elementos, UML permite-nos também representar **relações**:

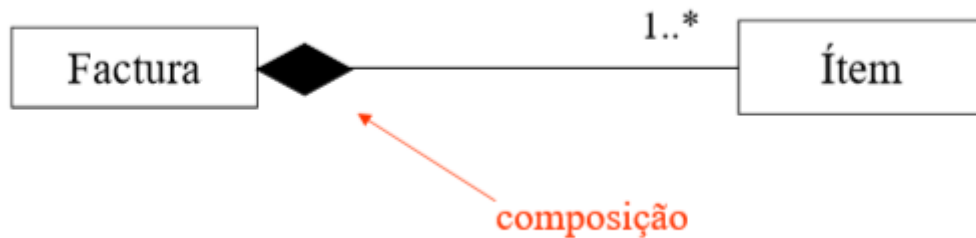
- **Associação**
 - Classe A “usa” ou “interage com” a classe B
 - Ex: O DS escreve um resumo



Img 5.1 – Exemplo de relações de associação

- **Composição**

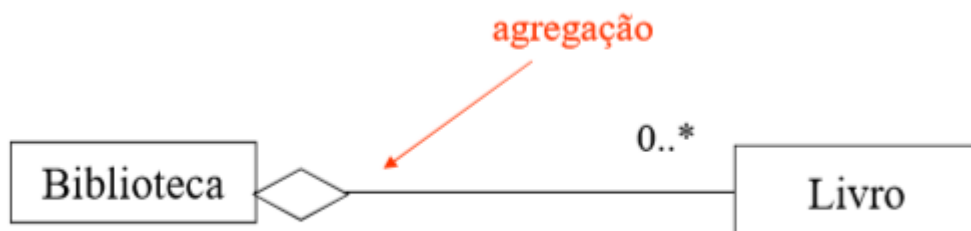
- Relação que liga um todo a uma parte
- Ex: Um processador é composto por varios cores (os cores compoem o processador)



Img 5.2 – Exemplo de relação de Composição

- **Agregação**

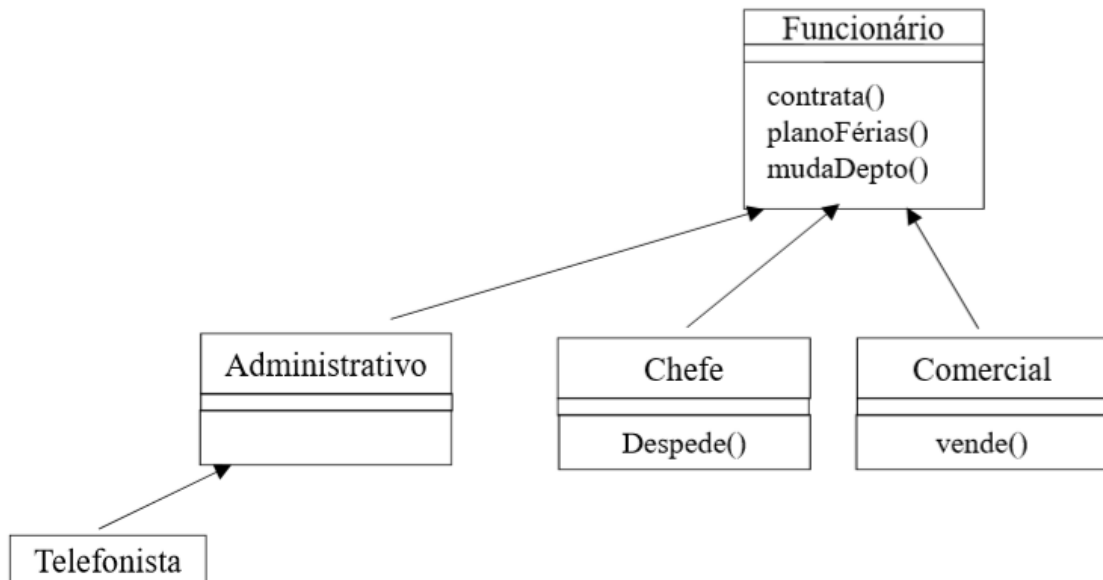
- Representa a relação estrutural entre instancias de duas classes, na qual a classe agregada existe independentemente da outra
- Ex: Um biblioteca tem varios livros (mesmo sem livros, a biblioteca continua a existir)



Img 5.3 – Exemplo de relação de agregação

- **Generalização**

- Classe A generaliza classe B, e B especializa a A
- Relação de superclasse/subclasse
- Ex: Uma humano é um animal



Img 5.4 – Exemplo de relação de relações de generalização

Por fim, convem salientar as diferenças entre Redes Semânticas e UML:

<u>Redes semânticas</u>	<u>UML</u>
subtipo(SubTipo,Tipo)	Generalização em diagramas de classes
membro(Obj,Tipo)	Diagramas de objectos
Relação Objecto/Objecto	Associação, agregação e composição em diagramas de objectos
Relação Objecto/Tipo	não tem
Relação Tipo/Tipo	Associação, agregação e composição em diagramas de classes

Img 5.5 – Diferenças entre Redes Semânticas e UML

VI. Indução e Dedução

Agora que chegamos às redes semânticas é importante fazer a distinção entre **Dedução** e **Indução**:

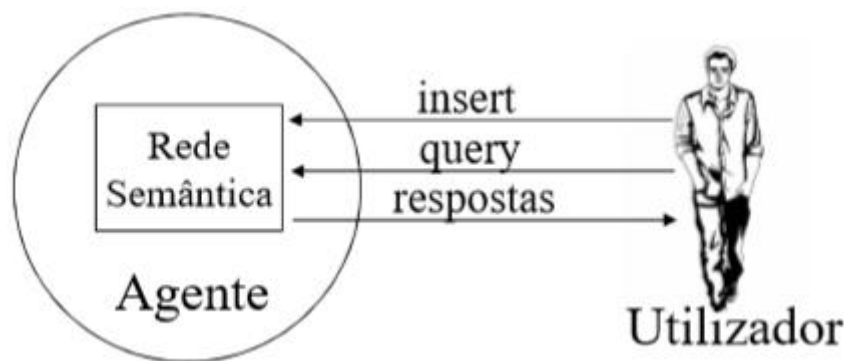
- **Dedução**
 - Permite inferir casos particulares a partir de regras gerais
 - Preserva a verdade
 - As regras de inferencia que temos vindo a apresentar são regras dedutivas
- **Indução**
 - É exatamente o oposto da dedução
 - Permite inferir regras gerais a partir de casos particulares
 - Principal base da aprendizagem
 - Nas redes semanticas, a indução pode ser vista como uma herança de baixo para cima (i.e vamos às instancias inferir regras sobre as classes, classes superclasses, and so on)
 - Ex:
 - Sabendo que:
 - O Pro Gamer DS gosta de CS:GO
 - O Pro Gamer Açoreano gosta de CS:GO
 - Inferimos que:
 - Os Pro Gamers (normalmente) gostam de CS:GO
 - Isto não prova claro que todos os Pro Gamers gostem de CS:GO – apenas que por analise de tendencias, podemos assumir que a norma é gostarem - daqui para a frente então vamos assumir que todos os Pro Gamers gostam de CS:GO exceto para os Pro Gamers que explicitamente provem ou digam que não gostam de CS:GO

VII. Redes Semânticas em Python

Para fechar este capítulo vamos demonstrar como é que se podem criar redes semânticas em python

Isto pode ser feito através da definição de um conjunto de **declarações**

Cada **declaração** vai associar uma relação semântica ao indivíduo que a declarou – Declaration(User,Relation). Estas relações podem depois ser guardadas em estruturas de dados adequadas



Img 7.1 – Diagrama que mostra como é que um utilizador interage com uma rede semântica

As relações (Declaração) vão poder ser de 3 tipos:

- **Member(Obj, Type)**
 - Objeto é membro de um tipo
- **Subtype(SubType,SuperType)**
 - Tipo é um subtipo de outro
- **Association(Entity1, AssocName, Entity2)**
 - Entidade (objeto ou tipo) esta associada a outra

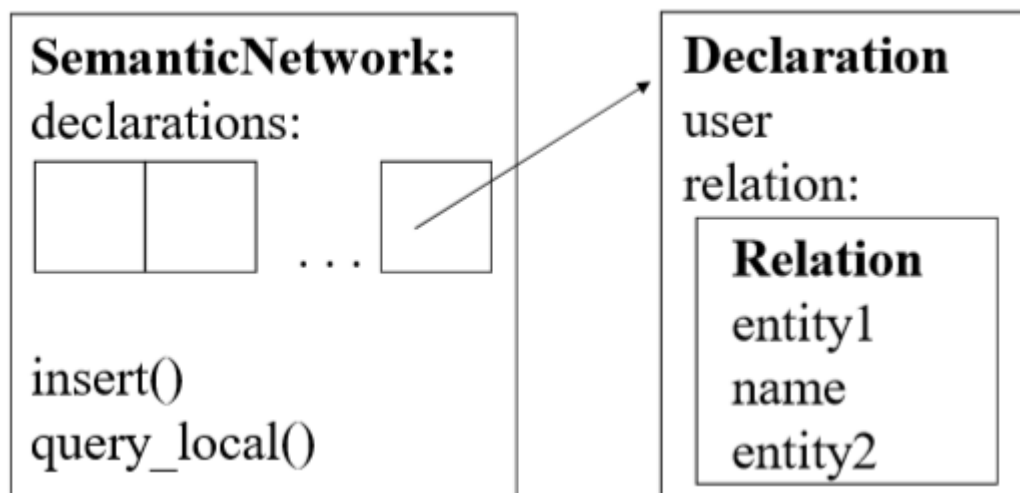
Vamos também ter as seguintes operações principais:

- **insert**
 - Introduzir uma nova declaração
- **Query_local**

- Questionar a rede semantica sobre as declarações existentes

Através da introdução incremental de declarações por diferentes interlocutores vamos emular, de forma simplificada, um processo de aprendizagem, no qual o conhecimento vai ser adquirido através da interação com outros agentes

Para veres um exemplo disto na prática sugiro que vejas a resolução dos guiões práticos



Img 7.2 – Diagrama que mostra como é que a rede semántica vai estar representada em python (com a classe SemanticNetwork e várias classes Declaration)

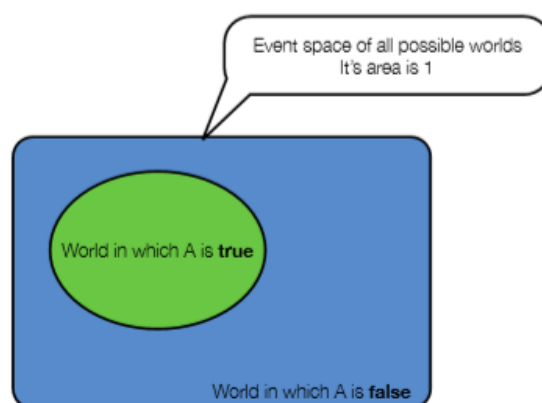
Redes de Bayes

I. Redes de Crença Bayesianas

Agentes inteligentes podem precisar de lidar, não só com Bases de conhecimento que são factuais e certas, mas também com **incerteza**

Isto pode provir do facto de ambientes serem ser **parcialmente observáveis, não deterministas** ou uma combinação dos dois.

Uma possível forma de fazer com que os agentes consigam lidar com esta incerteza é através da introdução de um **Belief State** – Uma representação do set de todos os possible world states em que o agente pode estar, e gerar um plano de contingencia capaz de lidar com qualquer eventualidade.



Img 1.1 – Mundo no qual a proposição A é verdadeira e mundo no qual é falsa

É fácil de verificar que as redes semânticas que vimos até agora são insuficientes para representar incertezas. É aí que entram as **redes de crenças bayesianas** ou simplesmente **Redes de Bayes**

Estas redes permitem representar **conhecimento impreciso** em termos de um **conjunto de variáveis aleatorias e as respetivas dependencias** dispostos num **grafo dirigido acíclico** nas quais as dependencias são expressas através de **probabilidades condicionadas**

II. Probabilidades

Antes de continuarmos convêm fazer um pequeno “apanhado” sobre os axiomas e teoremas das probabilidades:

Para uma qualquer proposição a , a sua probabilidade é um valor entre 0 e 1:

$$0 \leq P(a) \leq 1$$

Proposições necessariamente verdadeiras têm probabilidade 1

$$P(\text{true}) = 1$$

Proposições necessariamente falsas têm probabilidade 0

$$P(\text{false}) = 0$$

A probabilidade da disjunção é a soma das probabilidades subtraída da probabilidade da intercepção:

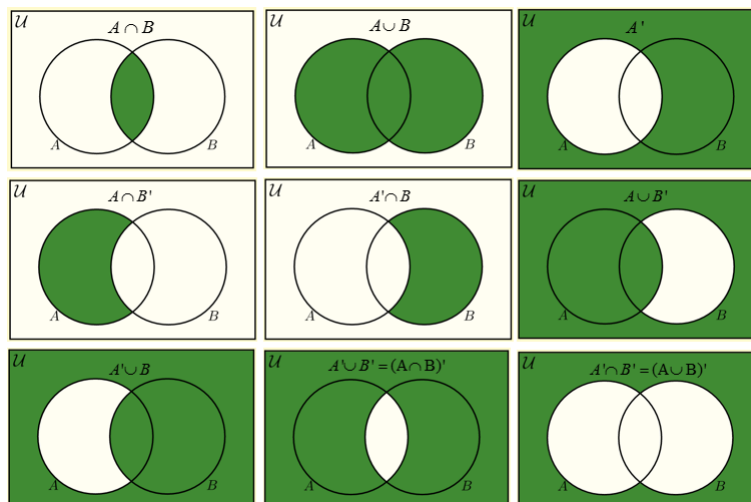
$$P(a \vee b) = P(a) + P(b) - P(a \wedge b)$$

Img 2.1 – Axiomas das Probabilidades

Destes axiomas conseguimos ainda provar que:

$$P(\text{NOT } A) = P(\sim A) = 1 - P(A)$$

$$P(A) = P(A \wedge B) + P(A \wedge \sim B)$$



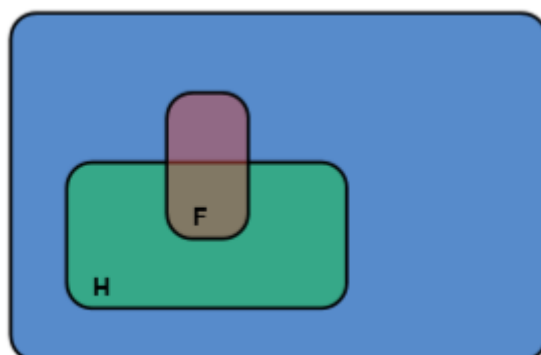
Img 2.2 – Diagramas de Venn que demonstram interações das probabilidades

Para os que se lembram do 12º ano, vão se recordar do conceito de **probabilidade condicionada**.

Probabilidade condicionada, representada por **$P(A|B)$** identifica a **probabilidade da proposição A ser verdadeira sabendo que a proposição B é verdadeira**. Ou seja representa “Probabilidade de A sabendo que B”

Matematicamente, a probabilidade condicionada é equivalente a:

$$P(A|B) = \frac{P(A \wedge B)}{P(B)}$$



Img 2.3 – Ex de Probabilidade Condicional. $H \cap F$ corresponde à área da parte onde o retângulo vermelho e o verde se intercetam a dividir pela área do retângulo vermelho.

Da fórmula da condicional parte também o importante coroloario a que chamamos Chain Rule ou Regra do Produto:

$$P(A \wedge B) = P(A|B) * P(B)$$

$$P(A \wedge B) = P(B|A) * P(A)$$

Para terminar convém também lembrar o conceito de **Independência**.

Dizemos que A e B são proposições **independentes** se a **probabilidade** do evento **A não afetar** de nenhuma forma a **probabilidade** do evento **B** .

Para A e B serem independentes, a seguinte regra verifica-se:

$$P(A \wedge B) = P(A) * P(B)$$

Se estiveres a prestar atenção consegues facilmente deduzir que, se A e B forem independentes, então:

$$P(A|B) = \frac{P(A \wedge B)}{P(B)} = \frac{P(A) * P(B)}{P(B)} = P(A)$$

III. Regra de Bayes

A regra de Bayes é uma derivação da regra do produto e dita que:

$$P(A|B) = \frac{P(A \wedge B)}{P(B)} = \frac{P(B|A) * P(A)}{P(B)}$$

Esta regra é importantíssima para IA e lógica porque conseguimos deduzir qual é a probabilidade de uma “causa” acontecer sabendo que aconteceu o “efeito” através do conhecimento da probabilidade do “efeito” sabendo a “causa”, a probabilidade da “causa” e a probabilidade do “efeito”:

$$P(Causa|Efeito) = \frac{P(Efeito|Causa) * P(Causa)}{P(Efeito)}$$

Para terminar vamos só deixar aqui só algumas derivações/formas gerais da Regra de Bayes:

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B|A) * P(A) + P(B|\sim A) * P(\sim A)}$$

$$P(A|(B \wedge X)) = \frac{P(B|A \wedge X) * P(A \wedge X)}{P(B \wedge X)}$$

IV. Full Joint Distribution – Probabilidade Conjunta

Dadas duas ou mais variáveis aleatórias, $X, Y, \dots$, definidas num espaço probabilístico, a **Probabilidade Conjunta** para $X, Y, \dots$, é uma distribuição probabilista que dá a **probabilidade** que cada $X, Y, \dots$ caia num certo valor.

Basicamente é uma mass function para a probabilidade de cada combinação dos valores das nossas variáveis aleatórias - $p(a, b) = P(X = a, Y = b)$

Isto pode ser representado na forma de uma tabela.

Digamos que temos duas urnas e que, para cada urna, vamos retirar uma bola. Em cada urna a probabilidade da bola ser azul é 1/3 e de ser vermelha é 2/3. A experiencia consiste em retirar uma bola de cada urna. Desenhe a tabela de probabilidade conjunta

Temos então duas variáveis aleatórias – A (retirar a primeira bola) e B (retirar a segunda bola). Como as variaveis são independentes, a tabela vai ficar:

	A=Red	A=Blue
B=Red	$(2/3)(2/3)=4/9$	$(1/3)(2/3)=2/9$
B=Blue	$(2/3)(1/3)=2/9$	$(1/3)(1/3)=1/9$

Img 4.1 – Tabela de probabilidade conjunta

Note-se que a soma de todos os valores na tabela de probabilidades conjunta tem que ser igual a 1 !

Depois de termos joint distribution conseguimos calcular o valor de qualquer variável aleatória e fazer mesmo aquilo que se chama de inferencia probabilística.

Porém, estas tabelas são humanamente impossíveis de criar quando temos demasiadas variáveis aleatórias porque...well iam ficar enormes

V. Probabilidade Individual e Inferencia Probabilística

Probabilidade Individual é o nome que damos à *probabilidade de um uma variável aleatória tomar um valor específico*

Da tabela de Probabilidades Conjunta conseguimos **calcular a probabilidade de qualquer uma das variáveis independentes** através da soma da linha ou coluna, pertinente a essa variável

Por exemplo, da tabela seguinte conseguimos extrair que:

$$P(\text{gender} = \text{Female}) = 0.25 + 0.03 + 0.04 + 0.01 = 0.33$$

$$P(\text{wealth} = \text{poor}) = 0.25 + 0.04 + 0.33 + 0.13 = 0.75$$

$$P(\text{hours/worked} = >40) = 0.29$$

gender	hours/ worked	wealth	Prob
Female	<40	poor	0.25
		rich	0.03
	>40	poor	0.04
		rich	0.01
Male	<40	poor	0.33
		rich	0.10
	>40	poor	0.13
		rich	0.11

Img 5.1 – Tabela de Joint Distribution

Para além disso conseguimos também realizar **inferencia probabilística**:

$$P(A|B)) = \frac{P(A \wedge B)}{P(B)} = \frac{\sum_{\text{rows matching } A \text{ e } B} P(\text{row})}{\sum_{\text{rows matching } B} P(\text{row})}$$

VI. Redes de Bayes - Representação

Como já mencionamos, **Redes De Bayes** são **grafos dirigidos e acíclicos** que permitem **representar conhecimento impreciso** em termos de um **conjunto de variáveis aleatórias** e as **respetivas dependencias** (que são expressas através de probabilidades condicionadas)

Temos então:

- **Nodes**
 - Correspondem a uma variável aleatória (que pode ser discreta ou continua)
- **Arestas direcionadas entre pares de nodes**
 - **Se houver uma seta do node X para o node Y dizemos que X é pai de Y**
- Cada node X_i tem uma conditional probability distribution **$P(X_i | \text{Parents}(X_i))$** que quantifica o quanto os seus pais afetam o node

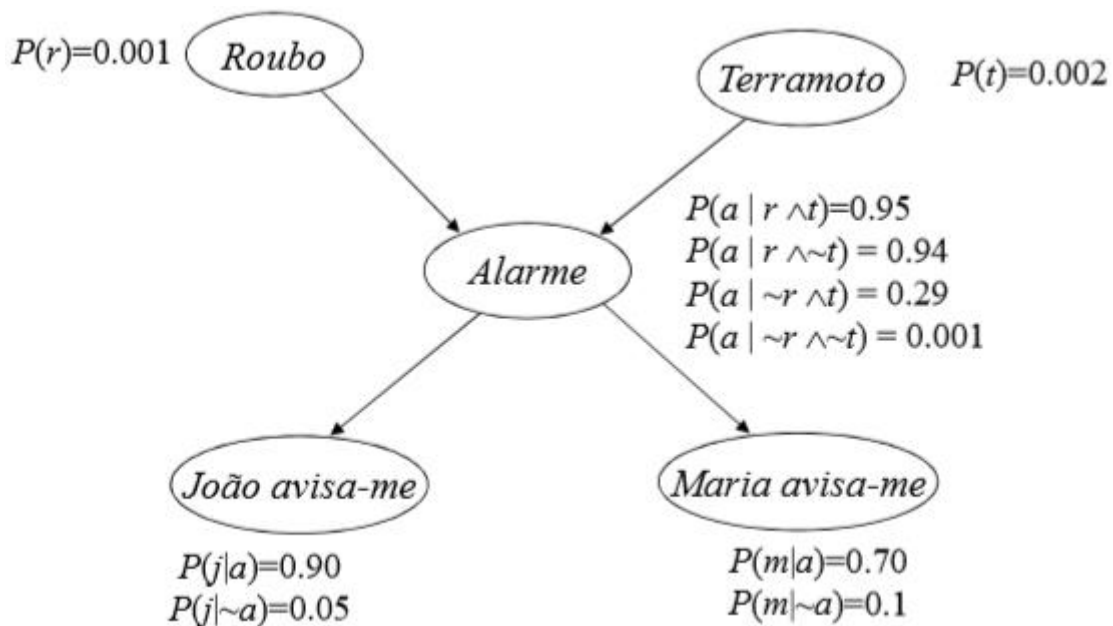
Para exemplificar temos o seguinte enunciado:

Temos um novo sistema de alarme contra roubo instalado em casa. É bastante fiável no que toca à deteção de roubos), mas também é capaz de ativar no caso de ocorrer um terramoto. Para além disso temos também dois vizinhos – o John que liga-nos quase sempre que ouve o alarme, mas que por vezes confunde o som do alarme com o som de um telemóvel a tocar, e a Mary que ouve musica bastante alto, portanto às vezes não ouve o alarme a tocar (mas se ouvir liga na mesma).

Por análise sabemos que vamos ter 5 variáveis aleatórias – B (Burglary), E (Earthquake), A (Alarm), J (John calls me) e M (Mary calls me).

As variáveis (para motivos de simplificação) só podem ser t – true ou f – false.

Assumindo que as probabilidades são dadas, vamos então chegar à rede de bayes:



Img 6.1 – Exemplo Rede de Bayes

VII. Redes de Bayes – Probabilidade Conjunta e Individual

Voltando ao exemplo anterior, imaginemos que queremos saber qual é a probabilidade do alarme tocar, ou de o alarme tocar e o João avisar...How?

Para isso temos que utilizar o que já aprendemos sobre probabilidade conjunta e individual e aplicar estes conceitos às redes bayesianas.

Em redes bayesianas:

- **Probabilidade Conjunta**

- Identifica a probabilidade de ocorrer uma dada combinação de valores de todas as variáveis da rede
- É dada pela formula:

$$P(x_1 \wedge \dots \wedge x_n) = \prod_{i=1}^n P(x_i \mid \text{pais}(x_i))$$

- Conseguimos portanto calcular, p.ex:
 - Qual a probabilidade de o alarme tocar, o João e a Maria ambos avisarem, num cenário em que não há roubo nem terremoto?
 - R:

$$P(J \wedge M \wedge A \wedge \sim E \wedge \sim R)$$

$$= P(J \mid A) * P(M \mid A) * P(A \mid \sim E \wedge \sim R) \\ * P(\sim E) * P(\sim R)$$

$$= 0.90 * 0.70 * 0.001 * 0.999 * 0.998$$

$$= 0.000628$$

- **Probabilidade Individual**

- Para isto, basta somar as probabilidades conjuntas de todas as situações em que a variável que queremos calcular tem um valor específico
- O calculo das probabilidades conjuntas pode restringir-se à variavel considerada e às outras variáveis das quais esta depende (ascendentes na rede bayesiana)
- Temos então a fórmula:

$$P(x_i = v_i) = \sum_{\substack{a_j \in \{v, f\} \\ j=1, \dots, k}} P(x_i \wedge a_1 \wedge \dots \wedge a_k)$$

Seja:

- $C = \{x_1, \dots, x_n\}$ – conjunto de variáveis da rede
- $x_i \in C$ – uma qualquer variável da rede
- $v_i \in \{v, f\}$ – valor de x_i cuja probabilidade se pretende calcular
- $\{a_1, \dots, a_k\} \subset C$ – conjunto das variáveis da rede que são ascendentes de x_i

- Conseguimos portanto calcular, p.ex:
 - Qual a probabilidade do alarme tocar?
 - R:

$$\begin{aligned} P(A) &= P(A \wedge R \wedge E) + P(A \wedge R \wedge \sim E) + P(A \wedge \sim R \wedge E) + P(A \wedge \sim R \wedge \sim E) \\ &= P(A | R \wedge E) * P(R) * P(E) + P(A | R \wedge \sim E) * P(R) * P(\sim E) + P(A | \sim R \wedge E) * \\ &\quad P(\sim R) * P(E) + P(A | \sim R \wedge \sim E) * P(\sim R) * P(\sim E) \\ &= 0.95 * 0.001 * 0.002 + 0.94 * 0.001 * (1-0.002) + 0.29 * (1-0.001) * \\ &\quad 0.002 + 0.001 * (1-0.001) * (1-0.002) \\ &= \mathbf{0.0015765536470347} \end{aligned}$$

(Nota – Verificar se isto está bem)

VIII. Redes de Bayes em Python

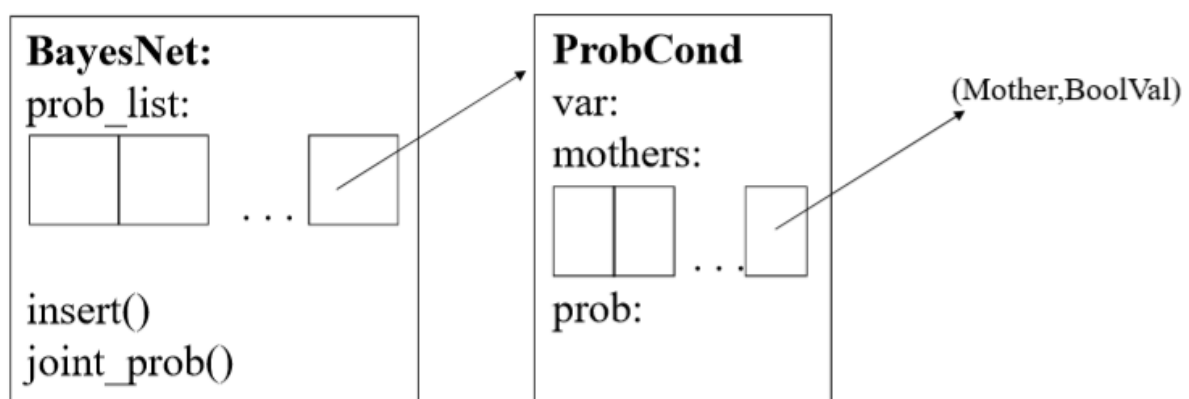
Para representarmos Redes de Bayes em python vamos criar uma lista de probabilidades condicionadas e coloca-las na classe **BayesNet()**

A probabilidade condicionada de uma dada variavel ser verdadeira (dados os valores das variaveis pais) vai ser representado pela classe **ProbCond(var, parent_vals, prob)**.

P.ex ProbCond("A", [("R",True), ("T", True)], 0.95) representa $P(A \mid R^{\wedge}T)$

Vamos também ter que ter as seguintes operações principais:

- **Insert**
 - Introduzir uma nova probabilidade condicionada na rede
- **Joint_Prob**
 - Obter a probabilidade conjunta para uma dada conjunção de valores de todas as variáveis da rede



Img 8.1 – Diagrama das estruturas que criamos em Python para representar Bayes Networks (Com a classe BayesNet e várias ProbCond). Note-se que em vez de parent usamos mother...thank you stor

Introdução à Pesquisa Clássica

I. Introdução

Um dos tipos de agentes que vimos no 2º capítulo eram os agentes baseados em goals / utility. Para reiterar, goals ajudam-nos a organizar o comportamento de um agente limitando os objetivos que este deve tentar alcançar e, de forma transitiva, as ações que este deve tomar para lá chegar.

Goal Formuation, baseado na situação atual e no **performance measure** do agente deve, portanto, ser o primeiro passo para problem solving.

Convém também explicar-mos que “**Problem Formulation** se refere ao processo de decidir que ações e estados considerar, dado um certo goal”.

Em geral, podemos considerar que “um agente com a possibilidade de escolher de entre várias ações num certo instante deve **começar por analisar ações a realizar no futuro** que eventualmente levem a estados desejáveis de forma a escolher o que fazer”.

Ao processo de procurar por uma sequência de ações que nos leve ao nosso goal – a **Solution** - chamamos **Search**.

Se depois de termos a nossa Solution o agente realizar as suas ações sem considerar novos percepts, dizemos que o sistema é **Open-Loop**.

Um **problema** pode ser definido segundo 5 componentes:

- **Estado Inicial**
 - Estado inicial no qual o agente é colocado
- **Ações disponíveis ao agente**
 - Descrições das ações que o agente pode realizar

- **Transition Model**
 - Descrição daquilo que cada ação realiza
- **Goal Test**
 - Teste usado para determinar se um dado estado é o nosso goal state
- **Path Cost**
 - Custo da sequencia de ações descritas na solução

II. Search Tree

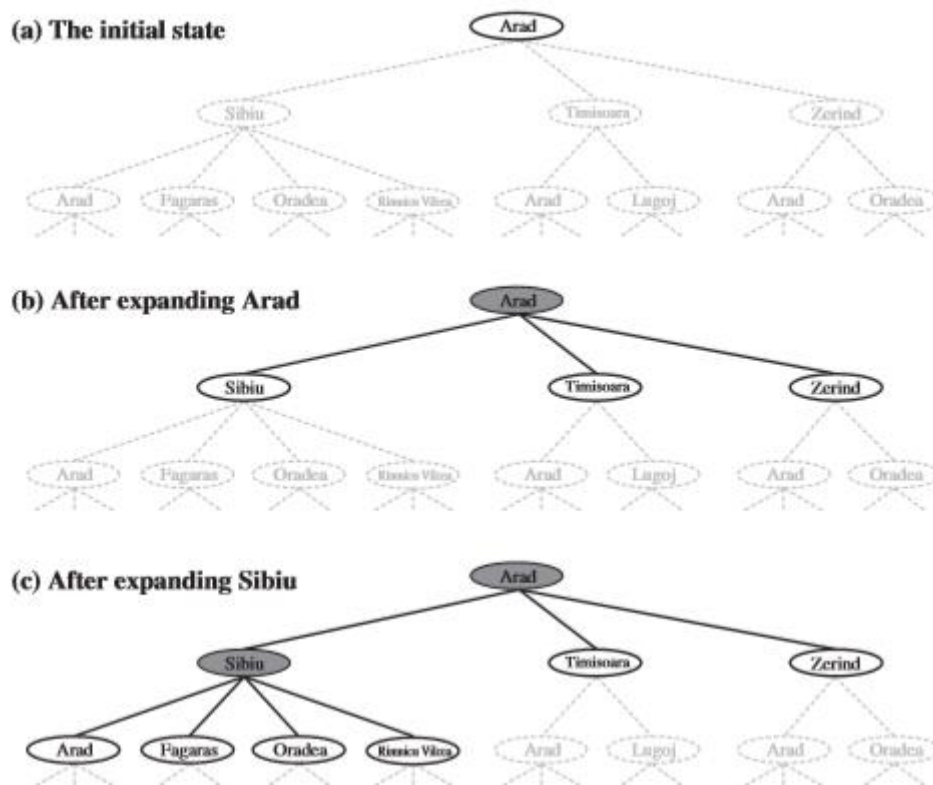
Podemos descrever cada possível sequencia de ações, começando no estado inicial – **Root Node** – através de uma tree structure – **Search Tree**.

Nestas trees, cada um dos **branches** corresponde a uma determinada ação, e os **nodes** correspondem aos diversos possíveis estados do problema.

Quando estamos num estado e aplicamos cada uma das possíveis ações partindo deste dizemos que estamos a **expandir o estado atual** – o que vai fazer com que **geremos novos estados**. Note se que a diferença entre os vários search algorithms que vamos apresentar de seguida é apenas a ordem pela qual decidimos expandir a nossa arvore – **Search Strategy**

Cada node tem um **parent node** – i.e o node que o gerou. No caso do node não ter filhos dizemos que se trata de um **leaf node**. Ao conjunto de todos os nossos leaf nodes chamamos **frontier** ou **open list**.

Podemos ter casos em que dois estados se liguem um ao outro, dando então origem a uma arvore possivelmente infinita. A estes paths chamamos **loopy path**. Podemos simplesmente ignorar estados em camadas mais baixas da arvore caso estes tenham ja sido abertos antes através da intuição que, como os **path costs são aditivos**, e um step nunca tem um custo negativo, um estado num loopy path nunca nos dará um path melhor do que o que teríamos antes.



Img 2.1 – Exemplo de uma search Tree e de duas expansões

III. Generic Tree and Graph Search Algorithms

Tendo já a noção do que é uma Search Tree podemos então definir o seguinte algoritmo genérico:

```

function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
  
```

Img 3.1 – Algoritmo genérico de Tree Searching. O passo “choose a leaf node and remove it from the frontier” deve ser realizado com a **search strategy** escolhida a priori

Mais especificamente, este algoritmo pode ser implementado através de queues, ficando com o seguinte aspeto:

```

pesquisa_em_arvore(Problema,AdicionarFila) retorna a Solução, ou ‘falhou’
Fila ← [ fazer_nó(estado inicial do Problema) ]
Ciclo
    se Fila está vazia, retornar ‘falhou’
    Nó ← remover_cabeça(Fila)
    se estado(Nó) satisfaz o objectivo
        então retornar a solução(Nó)
        senão Fila ← AdicionarFila(Fila, expansão(Nó))

```

Img 3.2 – Algoritmo genérico de Tree Searching implementado usando uma fila

Este algoritmo tem um grande problema. Não trata **caminhos redundantes** (tipo genérico de **loopy paths** que acontece quando temos várias formas de chegar de um node a outro), pelo que com muita facilidade pode entrar em **loops infinitos** :(

Por norma, o espaço de estados de um problema está representado segundo um grafo – Transições a partir de diferentes estados podem convergir no mesmo estado. É nestes casos em que vamos ter **caminhos redundantes**!

O que devemos portanto fazer é memorizar os estados já visitados de forma a evitar visita-los de novo. Note-se que devemos memorizar **APENAS** o melhor caminho até cada estado!

Para resolver isto basta adicionarmos ao nosso algoritmo uma **lista de nós já explorados** (ou set, ou queue, ou whatever dude), à qual chamamos o **explored set** (ou **fila de nós FECHADOS**).

A lista que já usavamos na tree search para guardar os nós na fronteira que ainda não foram expandidos vamos chamar **fila de nós Abertos**

A este novo algoritmo vamos chamar **Graph-Search Algorithm**, tendo ele a seguinte forma genérica:

```

function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
    only if not in the frontier or explored set

```

Img 3.2 – Algoritmo genérico de Graph Searching. Nota o facto de que este algoritmo já não entrará em loops infinitos visto que os nodes só são adicionados à fronteira caso ainda não tenham sido previamente explorados e não estejam na fronteira e continua a precisar de ter uma search strategy definida para escolher como seleccionar o próximo leaf node a expandir

Este algoritmo utiliza na mesma uma search tree, porem enquanto que a pesquisa em arvore ignora a possibilidade de o espaço de estados ser um grafo (com caminhos redundantes), esta pesquisa tem em conta que o espaço de estados é normalmente um grafo, grantindo que a **arvore de pesquisa nao tera mais do que um caminho para cada estado.**

IV. Tree and Graph Searching (em Python)

Vamos agora analisar a infraestrutura necessária para programáticamente (especificamente em Python) implementar um tree ou um graph search algorithm. No mínimo, são portanto necessárias as seguintes classes:

SearchDomain()

- Classe abstrata que formata a estrutura de um domínio da aplicação

SearchProblem(domain, initial, goal)

- Classe usada para especificar o problema(s) concreto(s) que queremos resolver

- Deve receber um domínio, initial state e goal state

SearchNode(state, parent)

- Classe usada para representar cada um dos nós da nossa search tree resultante.

- Em geral deve possuir o estado que representa e o parent que lhe deu origem

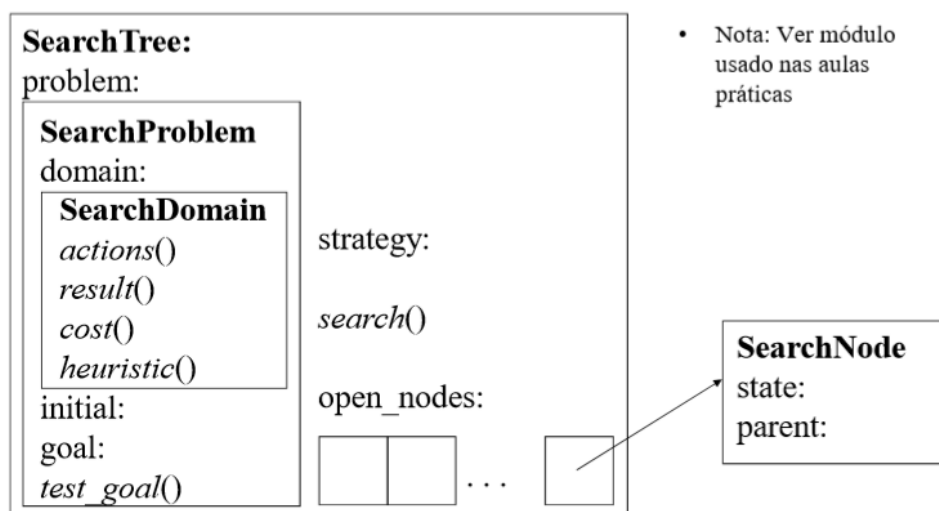
- Podemos ainda expandir esta classe para guardar mais informação como a ação que foi aplicada para chegar ao node ou o path cost total desde o root node até ele.

- Mais a frente vamos ver algoritmos que requerem que o node guarde certos valores heurísticos e custo.

SearchTree(problem)

- Classe das search trees que vai resultar de um dado problema.

- Deve, obviamente, conter os métodos necessários para gerar a árvore para dado problema.



Img 4.1 – Exemplo da estruturação das classes usadas para um Tree Search Algorithm

Sugere-se que, para melhor compreender esta estruturação e os métodos necessários por cada classe, vejas as resoluções dos guiões práticos.

VI. Avaliação do performance de Search Strategies

De forma a justificar a existência de várias search strategies precisamos de alguma forma de avaliar o performance de cada uma. De facto temos quatro tipos de avaliação:

- **Completeness**
 - Consideramos uma estratégia **completa** se for capaz de encontrar uma **solução** sempre que esta existir
- **Optimality**
 - Consideramos uma estratégia **optima** se for capaz de encontrar a **melhor solução (expectável) possível**
- **Time Complexity**
 - A **complexidade temporal** está relacionada com a **quantidade de tempo** que a estratégia **demora a encontrar a solução**
- **Space Complexity**
 - A **complexidade espacial** está relacionada com o **espaço de memória** que a estratégia precisa de **ocupar para encontrar a solução**

Note-se também que para facilitar a avaliação da **complexidade**, exprimimos-la em termos de:

- **Branching Factor, b**
 - Número máximo de sucessores que cada node pode possuir

- **Depth, d**
 - Numero de steps ao longo da arvore necessários até atingir o goal node mais superficial
- **Maximum Path Length, m**
 - Comprimento máximo que cada path pode possuir

Por outro lado, para avaliar o quão **eficiente** um search algorithm é consideramos o **search cost** – valor que depende da complexidade temporal, podendo tambem considerar a quantidade de memória usada – ou o **total cost** – combinação do **search cost** + **path cost da solução encontrada**.

No caso da pesquisa em árvores temos ainda mais outros branching factors a considerar – **Fatores de Ramificação**

Seja:

- N – número de nós da árvore de pesquisa no momento em que se encontra a solução
- X – Número de nós expandidos (não terminais)
- d – comprimento do caminho na árvore correspondente à solução

Ramificação média – número médio de filhos por nó expandido:

$$RM = \frac{N - 1}{X}$$

Nota: a ramificação média é um indicador da dificuldade do problema.

Factor de ramificação efectivo – número de filhos por nó, B, numa árvore com ramificação constante e com profundidade constante d. Portanto:

$1 + B + B^2 + \dots + B^d = N$ ou seja: $\frac{B^{d+1} - 1}{B - 1} = N$ (resolve-se por métodos numéricos).

- O factor de ramificação efectiva é um indicador da eficiência da técnica de pesquisa utilizada.

Img 6.1 – Explicação dos fatores de ramificação – Ramificação Média e Fator de Ramificação Efetivo

Algoritmos de Pesquisa Classica

I. Uninformed Search Strategies

Vamos começar por analisar aquilo que se chama **uninformed search strategies** (a.k.a **blind search**).

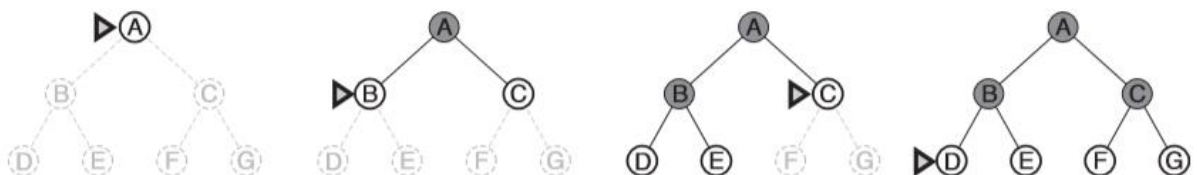
Este tipo de estratégias “*não possuem mais nenhuma informação sobre os estados para além da fornecida pela definição do problema. Como tal, tudo o que elas conseguem fazer é gerar sucessores e distinguir goal states de non-goal states, distinguindo-se apenas pela ordem em que os nodes são expandidos*”.

Este tipo de estratégias caracterizam-se pela simplicidade de implementação...e pelo quão merda em geral tendem a ser em termos de performance.

I.I Breadth-First Search

Breadth-First é uma estratégia simples na qual o root node é expandido primeiro, depois todos os sucessores do root node, depois os sucessores destes, and so on.

Todos os nodes num dado nivel são portanto expandidos antes que se prossiga à expansão dos nodes no nivel seguinte.



Img 1.1.1 – Exemplo da expansão dos nodes numa Breadth-First Search

Em termos de implementação isto pode ser simplesmente realizado através do uso de uma **FIFO Queue** (First in, First Out) para guardar os nodes na frontier. Os novos nodes abertos são portanto colocados no final da fila, enquanto que os nodes no nível atual (que devem estar no início da fila) são escolhidos para expansão.

Note-se que o **goal test deve ser aplicado a cada node quando é generated e não quando é selecionado para expansion**. Isto é feito para que não tenhamos de expandir todos os nodes numa dada layer antes de podermos detetar se o node que acabamos de gerar é o nosso goal state.

Temos então o algoritmo:

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier) /* chooses the shallowest node in frontier */
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
        frontier ← INSERT(child, frontier)
```

Img 1.1.2 – Algoritmo de uma Breadth-First Search. Note-se que o primeiro if é utilizado para verificar que o root node não é também o goal node. Por outro lado é de frisar que este algoritmo é exatamente igual ao Generic Graph Search Algorithm com Estado interno (explored), com a única diferença sendo que estamos agora a especificar o nosso passo de “choose a leaf node and remove it from the frontier”: `node <- Pop(frontier)`

Ou, alternativamente, baseando-nos no Generic Tree Search algorithm implementado utilizando uma queue:

```

pesquisa_em_arvore(Problema,AdicionarFila) retorna a Solução, ou 'falhou'
  Fila ← [ fazer_nó(estado inicial do Problema) ]
  Ciclo
    se Fila está vazia, retornar 'falhou'
    Nó ← remover_cabeça(Fila)
    se estado(Nó) satisfaz o objectivo
      então retornar a solução(Nó)
    senão Fila ← AdicionarFila(Fila, expansão(Nó))

```

```

pesquisa_em_largura(Problema) retorna a Solução, ou 'falhou'
retornar pesquisa_em_arvore(Problema,juntar_no_fim)

```

Img 1.1.3 – Algoritmo de uma Breadth-First Search baseado no Generic Tree Search Algorithm

Em termos de performance temos:

- **Completeness - SIM**

- É fácil de ver que se uma solução se encontrar numa profundidade não infinita, a breadth first search vai acabar pela encontrar (mais especificamente, vai encontrar a “**shallowest goal node at some finite depth**” (i.e o goal node que estiver no nível mais baixo)

- **Optimality – NOT GUARANTEED**

- Breadth-First vai encontrar a solução que estiver a uma profundidade mais baixa. Isto não garante que esta seja a mais optima.
- De facto, esta search strategy só é optimal se o path cost for uma função non-decreasing da profundidade do node, tendo cada action o mesmo custo.

- **Time Complexity - BAD**

- Imaginando uma arvore uniforme em que cada estado tem b successors. Cada node vai portanto gerar b novos nodes.
- Para chegarmos a uma solução vamos portanto ter de gerar b^d nodes, onde d é a depth da solução.
- Temos portanto uma complexidade **$O(b^d)$**

- **Space Complexity - BAD**

- Visto que vamos ter que guardar cada node expandido no **explored set**, este terá uma complexidade espacial de $O(b^{d-1})$
- Para além disso temos também que guardar os nodes na **frontier** – $O(b^d)$
- A complexidade espacial é portanto $O(b^d)$

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Img 1.1.3 – Exemplo da má space e time complexity do Breadth First

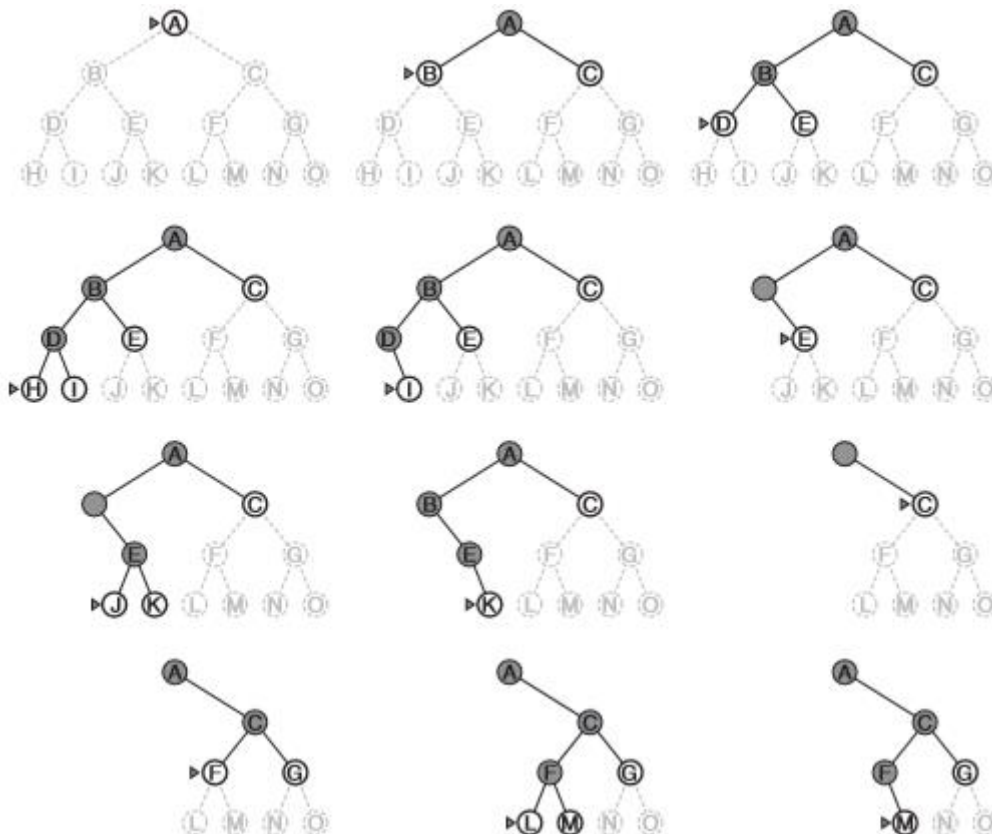
NOTA: Existe uma variante do Breadth-First chamada **Uniform-Cost Search**. A unica diferença é que esta estratégia opta por “*expandir o node com o menor path cost – $g(n)$* ”.

Isto vai fazer com que o algoritmo seja optimo mesmo nos casos em que o step-cost não é uniforme.

I.II Depth-First Search

Comparativamente à Breadth-First strategy, a **Depth-First** opta por expandir sempre o node mais profundo na atual fronteira.

Expandimos sempre o node mais profundo até chegarmos ou à solução, ou a um leaf node que não possua mais sucessores, ou à uma profundidade máxima (Depth-Limited Search Variant)



Img 1.2.1 – Exemplo da expansão dos nodes numa Depth-First Search. Os nodes na depth 3 não tem sucessores, e o goal node é o M

Em termos de implementação isto pode ser simplesmente realizado através do uso de uma **LIFO Queue** (Last in, First Out) para guardar os nodes na frontier. Os novos nodes abertos são portanto colocados no final da fila sendo portanto escolhidos primeiro para expansão.

Temos então o algoritmo:

```
function   Depth-First-Search(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← LIFO queue with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier) /* chooses the Deepest node in frontier */
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
        frontier ← INSERT(child, frontier)
```

Img 1.2.2 – Algoritmo de uma Depth-First Search.

Ou, alternativamente, baseando-nos no Generic Tree Search algorithm implementado utilizando uma queue:

```
pesquisa_em_arvore(Problema,AdicionarFila) retorna a Solução, ou ‘falhou’
  Fila ← [ fazer_nó(estado inicial do Problema) ]
  Ciclo
    se Fila está vazia, retornar ‘falhou’
    Nó ← remover_cabeça(Fila)
    se estado(Nó) satisfaz o objectivo
      então retornar a solução(Nó)
      senão Fila ← AdicionarFila(Fila, expansão(Nó))
```

```
pesquisa_em_profundidade(Problema) retorna a Solução, ou ‘falhou’
retornar pesquisa_em_arvore(Problema,juntar_à_cabeça)
```

Img 1.2.3 – Algoritmo baseado na Generic Tree Search implementation com queue

Note-se que podemos também implementar o Depth-First Search Algorithm de forma recursiva. O exemplo seguinte implementa o Depth-Limited Search Variant (que limita a profundidade máxima de pesquisa) de forma recursiva baseando-se no Graph-Search Algorithm

```

function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
    cutoff_occurred?  $\leftarrow$  false
    for each action in problem.ACTIONS(node.STATE) do
      child  $\leftarrow$  CHILD-NODE(problem, node, action)
      result  $\leftarrow$  RECURSIVE-DLS(child, problem, limit - 1)
      if result = cutoff then cutoff_occurred?  $\leftarrow$  true
      else if result  $\neq$  failure then return result
    if cutoff_occurred? then return cutoff else return failure

```

Img 1.2.4 – Algoritmo Recursivo de uma Depth-Limited Search

Em termos de performance temos:

- **Completeness – Graph-Search: SIM ; Tree-Search: NÃO**
 - Isto deve-se ao facto da Graph-Search version evitar estados repetidos e redundant paths, tornando-se **complete in a finite state spaces** (porque vai acabar por expandir todos os nodes)
 - A tree search version não é complete porque não tem maneira de evitar a proliferação de redundant paths.
 - Note-se que apenas a **Depth-Limited Variant** vai ser capaz de evitar loops infinitos quando o state space é infinito.
- **Optimality – NOT GUARANTEED**
- **Time Complexity - BAD**
 - No pior dos casos vamos ter que gerar $O(b^m)$ nodes, onde m é a maximum depth de cada node.

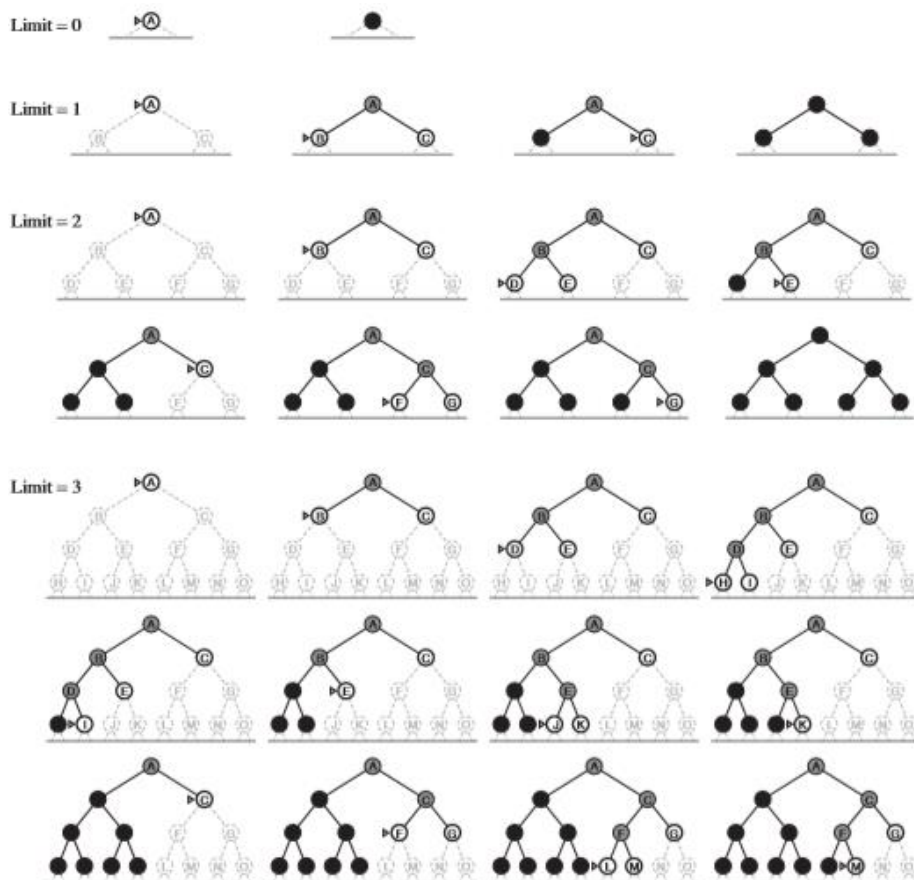
- **Space Complexity - DECENT**

- Ao contrario da breadth-first, vamos apenas ter que guardar um unico path da root até a um leaf node (e todos os unexpanded sibling nodes para cada node no path).
- Quando um node for fully expanded pode ser removido da memória assim que todos os seus descendentes forem completamente explorados.
- Temos então a complexidade **$O(bm)$**

I.III Iterative Deepening Depth-First Search (Depth-First Variant)

Esta variação do Depth-First Search baseia-se na ideia de encontrar o **best depth limit**.

Isto é conseguido aumentando de forma gradual o nosso depth limit, até conseguirmos encontrar um goal.



Img 1.3.1 – Evolução do depth limit no Iterative Deepening Algorithm

Desta formas vamos conseguir combinar os beneficios de depth-first e breadth-first:

- **Completeness – SIM**
 - Este algoritmo é completo quando o branching factor for finito (tal como o breadth-first)
- **Optimality – NOT GUARANTEED**
 - Temos um algoritmo optimo quando o path cost for uma nondecreasing function da depth dos nodes.
- **Time Complexity - BAD**
 - No pior dos casos vamos ter que gerar $O(b^m)$ nodes, onde m é a maximum depth de cada node.
- **Space Complexity – Decent**
 - Conseguimos manter uma modesta space complexity - **$O(bd)$**

Pegando no nosso Graph-Search algorithm para Depth-Limited, temos que a unica diferença em termos de implementação entre o algoritmo normal e esta variante é:

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
  for depth = 0 to  $\infty$  do
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)
    if result  $\neq$  cutoff then return result

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
    cutoff_occurred?  $\leftarrow$  false
    for each action in problem.ACTIONS(node.STATE) do
      child  $\leftarrow$  CHILD-NODE(problem, node, action)
      result  $\leftarrow$  RECURSIVE-DLS(child, problem, limit - 1)
      if result = cutoff then cutoff_occurred?  $\leftarrow$  true
      else if result  $\neq$  failure then return result
    if cutoff_occurred? then return cutoff else return failure
```

Img 1.3.2 – Para chegarmos ao Iterative Deepening Depth-First Search mudamos apenas a forma como chamamos a nossa Depth-Limited Search Function

I.IV Síntese das variações ao Depth-First Algorithm

Pesquisa em profundidade sem repetição de estados – para evitar ciclos infinitos, convém garantir que estados já visitados no caminho que liga o nó actual à raiz da árvore de pesquisa não são novamente gerados.

Pesquisa em profundidade com limite – não são considerados para expansão os nós da árvore de pesquisa cuja profundidade excede um dado limite.

Pesquisa em profundidade com limite crescente – consiste no seguinte procedimento:

- 1) Tenta-se resolver o problema por pesquisa em profundidade com um dado limite N
- 2) Se foi encontrada uma solução, retornar.
- 3) Incrementar N .
- 4) Voltar ao passo 1.

Img 1.4.1 – Os 3 tipos de variantes do Depth-First de que falamos

NOTA: Existe ainda outra variação chamada **Backtracking Depth-First Search** que usa menos memória que a Depth Search Normal. Neste algoritmo apenas um sucessor é gerado de cada vez, em vez de serem gerados todos os sucessores. Cada partially expanded node lembra-se que sucessor é que vai gerar a seguir, desta forma, apenas é usada $O(m)$ memória, em vez de $O(bm)$. A ideia que leva a esta redução é a de gerar um sucessor modificando o atual state description diretamente, ao invés de a copiar. Para que isto funcione, temos de ser capazes de fazer undo de cada modificação quando fizermos backtrack para gerar outro sucessor

II. Comparação entre as Uninformed Search Strategies

Antes de falarmos das Informed Search Strategies (que só por si são melhores que qualquer uninformed search strategy referida anteriormente), vamos comparar o performance destas:

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$
Optimal?	Yes ^c	Yes	No	No	Yes ^c

Img 2.1 – Comparação, segundo os 4 criterios de performance, dos algoritmos falados até agora

III. Informed (Heuristic) Search Strategies

Comparativamente com as estratégias que discutimos até agora, as **Informed Search Strategies** usam problem-specific knowledge para além da definição do problema em si.

Com isto, são capazes de encontrar soluções de forma mais eficiente do que as uninformed search strategies.

A forma mais comum de adicionar conhecimento extra sobre o problema é através de **Heuristic Functions** – **$h(n)$** onde n é um dado node

III.I Best-First Search

Este algoritmo é uma instancia do General Tree Search (ou Graph Search) algorithm, sendo que a diferença é que a seleção do leaf node a expandir é feita de acordo com uma **evaluation function** – **$f(n)$** .

A evaluation function é construída com um cost estimate, tal que o node com a evaluation mais pequena seja expandido primeiro.

Vai ser a escolha e especificação da evaluation function que vai determinar qual a search strategy

Por norma, a maioria dos best-first algorithms incluem como componente da evaluation function a **heuristic function** que definimos anteriormente, onde:

$h(n)$ = estimativa do custo do path mais barato do estado no node n , i.e o estado atual, até ao goal state

pesquisa_informada(Problema,FuncAval) **retorna** a Solução, ou ‘falhou’
Estratégia $\leftarrow$ estratégia de gestão de fila de acordo com FuncAval
pesquisa_em_arvore(Problema,Estratégia)

Img 3.1.1 – Algoritmo de uma Best-First Search Strategy.

III.II Greedy Best-First Search

Este algoritmo tenta expandir sempre o node que estiver mais perto do goal, com base na ideia de que este provavelmente será o que nos levará a uma solução mais rapidamente.

Evaluation Function: $f(n) = h(n)$

Infelizmente, o algoritmo acaba por não ser lá grande coisa :

- **Completeness – NÃO**
 - Mesmo num finite state space, este algoritmo não pode ser considerado complete.
 - Isto acontece graças a **dead ends**

- **Optimality – NÃO**

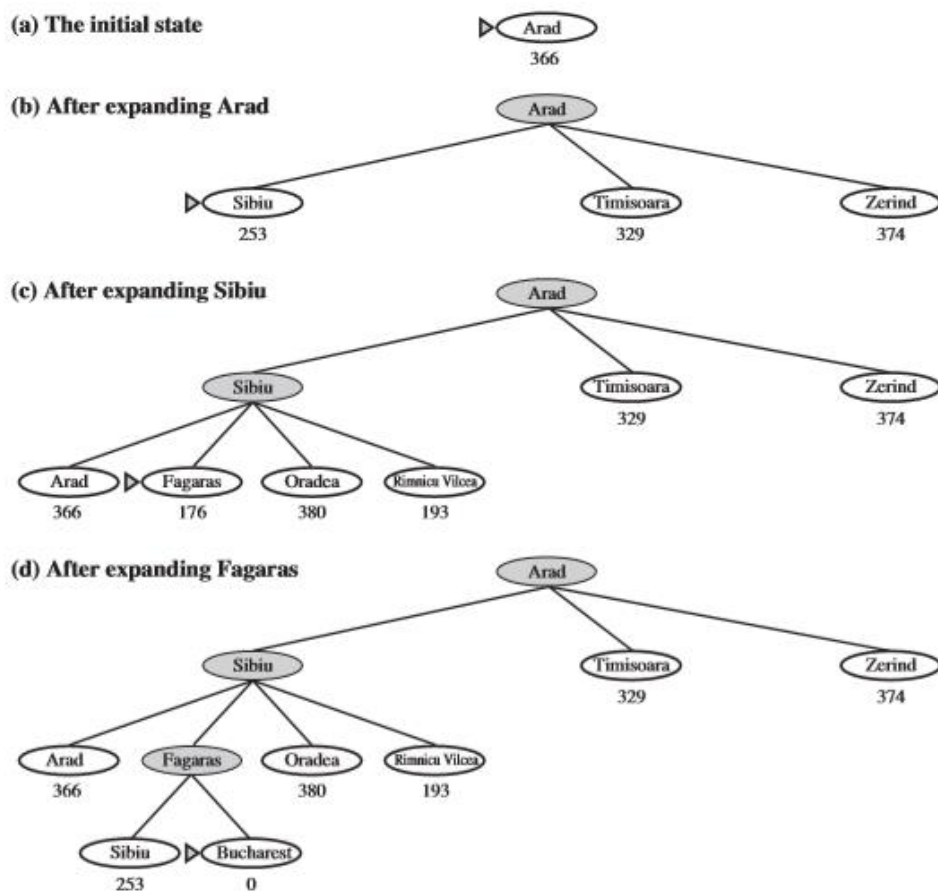
- Caso encontre uma solução, vamos ter a solução cujo **search cost** seja mínimo. Porém não nos é garantido que teremos o path com o **path cost** mais curto!

- **Time Complexity - BAD**

- No pior dos casos vamos ter que gerar $O(b^m)$ nodes, onde m é a maximum depth de cada node.

- **Space Complexity – BAD**

- No pior dos casos vamos ter que gerar $O(b^m)$ nodes, onde m é a maximum depth de cada node.



Img 3.2.1 – Exemplo da evolução da search tree usando Greedy

III.III A* Search

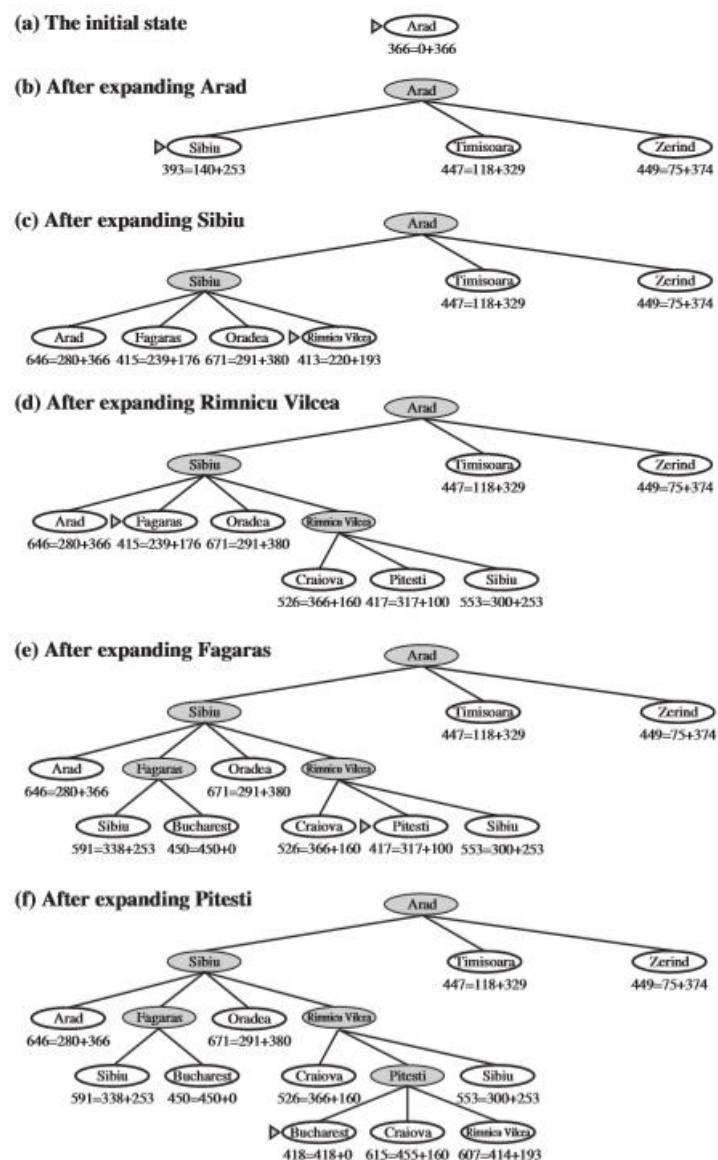
Algoritmo de Best-First Search mais famoso.

Este algoritmo vai avaliar cada node combinando o **custo de chegar até esse node**, $g(n)$ – Cost Function - com a **estimativa do custo desse node chegar até ao goal**, $h(n)$ – Heuristic Function

Evaluation Function: $f(n) = g(n) + h(n)$

A evaluation function vai então corresponder ao “*estimated cost of the cheapest solution through n*”

Esta search strategy é **completa** !



Img 3.3.1 –
Exemplo da
evolução da
search tree
usando A*

Note-se que o algoritmo de **Uniform Cost Search**, que referimos anteriormente, pode ser visto como uma variante do A^* na qual o **$h(n)$ é sempre igual a 0**. Este é também conhecido como algoritmo de Dijkstra e é garantido que encontre sempre uma solução ótima primeiro, caso ela exista

Noções sobre Heurísticas:

- Uma função heurística, $h(n)$ diz-se **admissível** se nunca sobrestimar o custo real de chegar a uma solução a partir do node n

- O A^* encontrará sempre uma solução **ótima** no caso da Tree-Search Version caso se prove que $h(n)$ é admissível!

- Por outro lado, $h(n)$ diz-se **consistente** se para cada node n , cada sucessor n' gerado a partir deste por qualquer ação a , o custo estimado de chegar ao goal através de n não for maior que o step cost de chegar até n' mais o custo esperado de chegar do nó n' até à solução

- O A^* encontrará sempre uma solução **ótima** no caso da Graph-Search Version caso o $h(n)$ seja consistente!

- Uma *“heurística é tanto melhor quanto mais se aproximar do custo real”*. A qualidade desta pode ser medida através do **fator de ramificação efetiva** – Quanto melhor a heurística, mais baixo será esse fator.

- Claro que em alguns domínios há funções de estimação de custos que são naturalmente **heurísticas admissíveis** (p.ex, Distância em linha reta no domínio dos caminhos entre cidades).

- Porém, existem outros em que não há nenhuma heurística admissível que seja óbvia (p.ex Planeamento no mundo dos blocos)

- Um “**problema simplificado** é um problema com menos restrições do que o **problema original**”. É possível gerar automaticamente formulações simplificadas de problemas a partir da formulação original (ideia de partir um problema complexo em vários mais simples).

- A resolução do problema simplificado será feita usando pouca ou mesmo nenhuma pesquisa.

- Desta forma podemos “inventar” heurísticas, escolhendo a melhor, ou, **combinando-as numa nova heurística**.

- **É importante frisar que o custo de uma solução ótima de um problema simplificado constitui uma heurística admissível para o problema original!**

- Se tivermos várias heurísticas admissíveis podemos depois combiná-las numa nova heurística (escolhendo o max de entre elas).

- Esta nova heurística será **admissível** e dado que é uma melhor aproximação ao custo real, vai ser uma heurística melhor do que todas as outras!

$$H(n) = \max(\{h(n)_1, \dots, h_n(n)\})$$

Img 3.3.2 – Combinação de heurísticas

Para terminar, vamos só demonstrar as vantagens e desvantagens da A*

Principais vantagens

- Completa
- Óptima

Principais desvantagens

- Na maior parte das aplicações, o consumo de memória e tempo de computação têm um comportamento exponencial em função do tamanho da solução
- Em problemas mais complexos, poderá ser preciso utilizar algoritmos mais eficientes, ainda que sacrificando a optimalidade
- Ou então, usar heurísticas com uma melhor aproximação média ao custo real, ainda que não sendo estritamente admissíveis, e não garantindo portando a optimalidade da pesquisa

Img 3.3.3 – Vantagens e Desvantagens da A*

III.IV IDA* Search

Podemos reduzir os custos de memória que o A* requer através da adaptação da ideia de **iterative deepening** ao contexto de pesquisa heurística. Disto resulta o **Iterative-Deepening A*** algorithm.

A maior diferença entre este e o iterative deepening normal é o facto do cutoff usado ser o **valor da evaluation function, f-cost** ($g(n) + h(n)$) em vez da profundidade.

Em cada iteração, o cutoff value é o **f-cost** mais pequeno de cada node que exceda o cutoff da iteração anterior.

Basicamente estamos a aplicar a limitação de profundidade de forma indireta através da aplicação de um limite à função de avaliação:

$$\text{avaliação } f(n) = g(n) + h(n) \leq f_{\max}$$

- Ou seja: Qualquer nó n com $f(n) > f_{\max}$ não será expandido

Img 3.4.1 – Limitação que aplicamos a cada iteração

O algoritmo procede, portanto, incrementando o A^* que já discutimos da seguinte forma:

1. $f_{max} = f(\text{raiz})$
2. Executar pesquisa em profundidade com limite f_{max}
3. Se encontrou solução, retornar solução encontrada
4. $f_{max} \leftarrow$ menor $f(n)$ que tenha sido superior a f_{max} na última execução do A^*
5. Voltar ao passo 2

Img 3.4.2 – Funcionamento do IDA*

O problema com este algoritmo é o facto de não lidar bem com real-valued costs, sendo preferível quando temos steps com custo unitário.

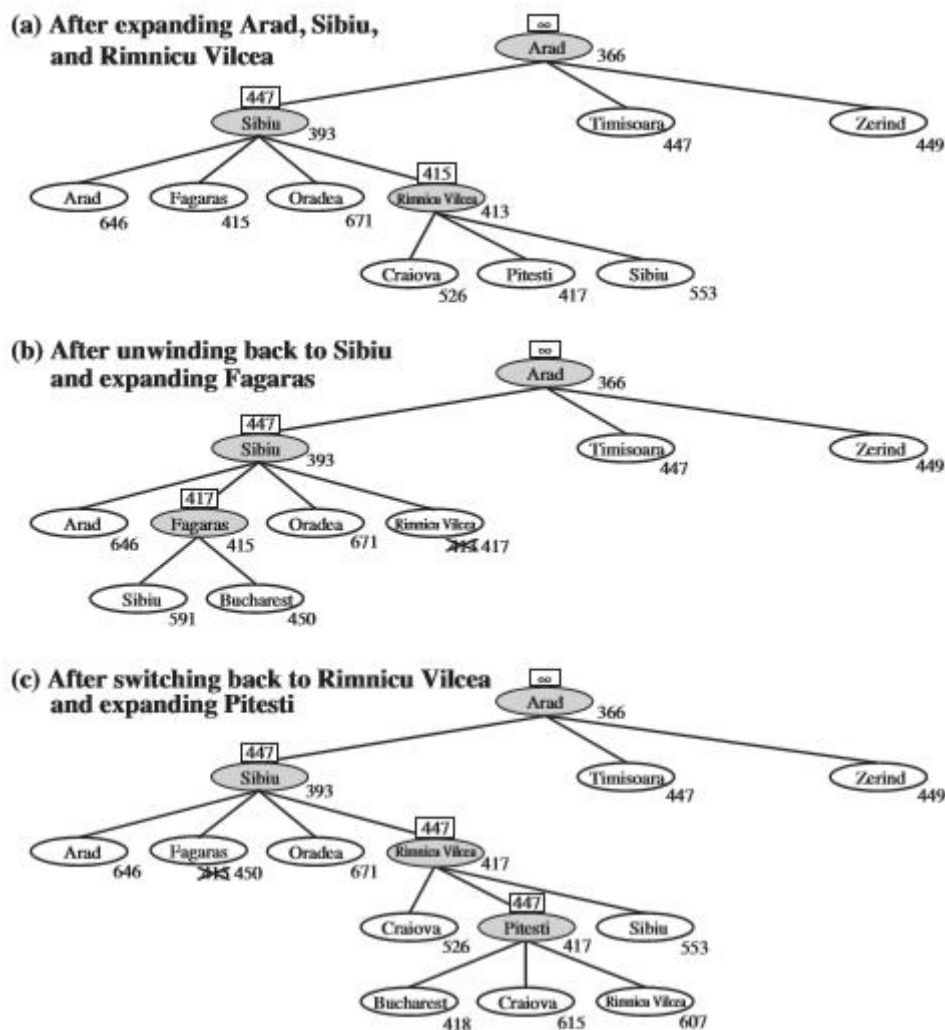
III.V RBFS Search

Outro memory-bounded algorithm é o chamado **Recursive Best-First Search**.

Este algoritmo tenta copiar o funcionamento de um best-first search normal, mas ocupando apenas espaço linear

Para cada node, o algoritmo não guarda o valor da sua **evaluation function, $f(n)$** , mas sim o menor valor **$f(x)$** , sendo x um node descendente desse node que se encontre na fronteira.

Sempre que um node é expandido temos que ter a preocupação de atualizar os custos armazenados nos nodes ascendentes



Img 3.5.1 – Evolução da search tree num RBFS

“Este algoritmo vai trabalhar de forma similar ao Backtracking Depth-First Search no sentido que, quando um leaf node m , com menor custo $f(m)$ não for filho do ultimo node extendido, n , então o algoritmo vai fazer backtrack até ao ascendente comum de m e n ”

III.V SMA* Search

SMA* - **Simplified Memory Bounded A*** - é um incremento ao A* que...well..limita a memória que este vai utilizar.

Basicamente, vamos utilizar a ideia de **memória disponível** (ao contrario do IDA* e do RBFS que estão desenhados para poupar memória independentemente se ela existir de sobra ou não).

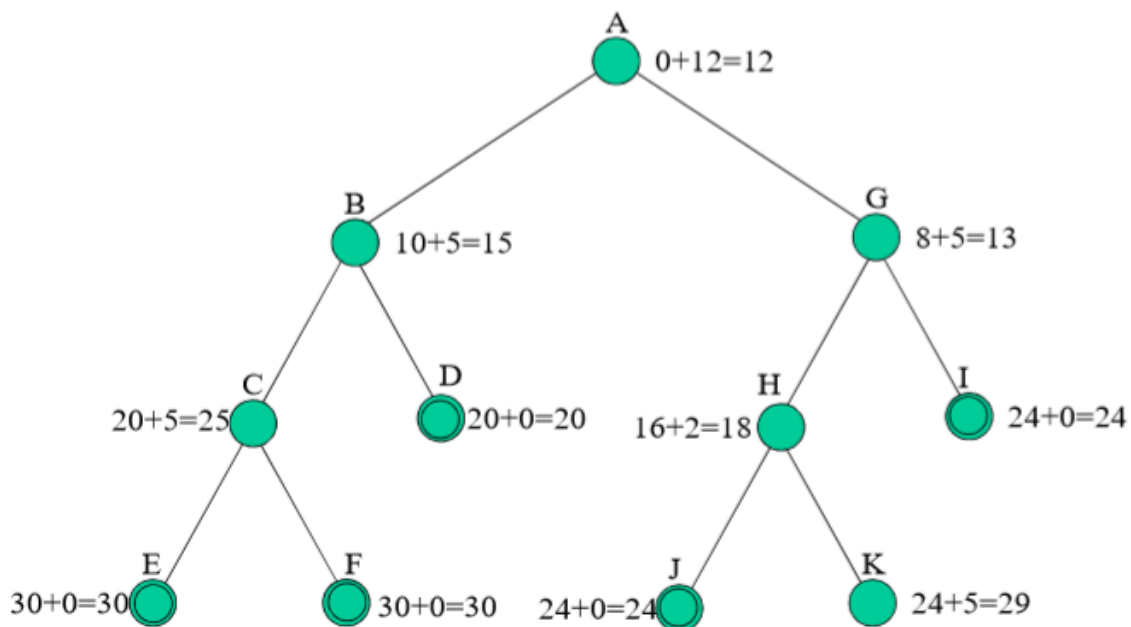
Quando a memória disponível chega ao seu limite, vamos esquecer – i.e, remover – o node com o **maior custo de evaluation function, $f(n)$** , e atualizar em cada um dos nodes ascendentes o custo do melhor node esquecido.

Só voltamos a gerar o node n quando o custo do melhor node esquecido registrado no antecessor de n for inferior aos custos dos restantes nodes.

Em cada iteração é gerado **apenas um node sucessor**. Existindo já um ou mais filhos de um node, apenas se gera ainda outra se o custo do node pai for menor do que qualquer dos custos dos filhos. Quando se gerarem todos os nodes filhos de um node o custo do pai é atualizado, tal como no RBFS

Em síntese: Vamos expandir o melhor leaf node e apagar o worst leaf node. Mas e o que acontece quando todos os nodes tiverem o mesmo f -value? Para evitar selecionar o mesmo node para ser apagado e expandido, o SMA* expando o **newest best leaf node** e apaga o **oldest worst leaf**.

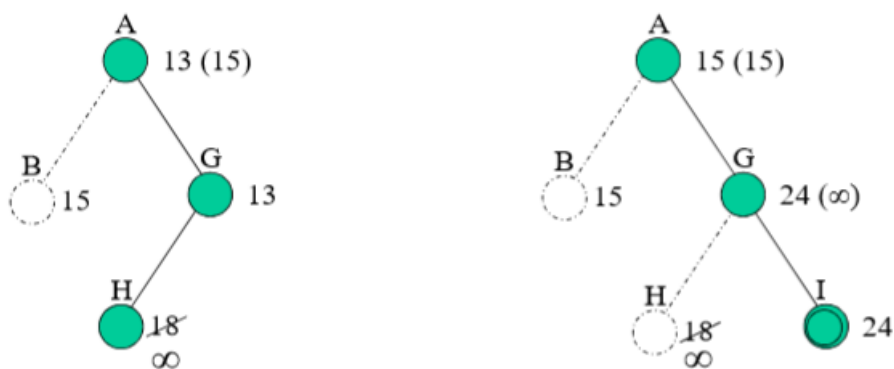
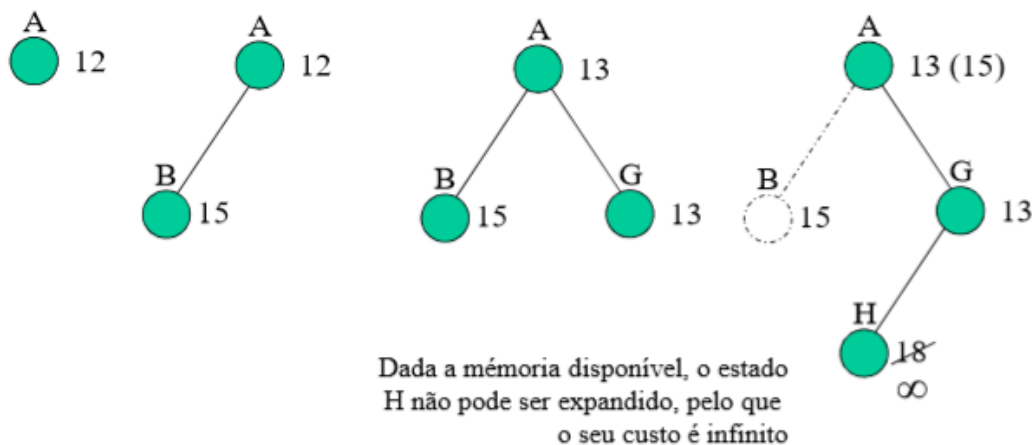
Vejemos o seguinte exemplo:



Img 3.6.1 – Conjunto de Estados do nosso problema de exemplo e valores de cada evaluation function. Os node goals são o E, F, J, D ou I

Neste exemplo: memória = 3 nós

– Melhores custos de nós esquecidos anotados entre parêntesis



- Chegámos a uma solução (estado I)
- Se quisermos continuar: Das restantes folhas já exploradas, a que tinha o estado B era a melhor, por isso a pesquisa retrocede e continua expandindo esse folha

Img 3.6.2 – Resolução do exercício com SMA* de memória disponível = 3

Técnicas de Pesquisa Local

I. Local Search: Para além da pesquisa clássica

Os problemas que temos vindo a analisar são observáveis, determinísticos, e conhecidos, onde a solução é uma sequência de ações.

Neste capítulo vamos analisar técnicas que realizam **pesquisa local no state space, avaliando e modificando um ou mais estados**, ao contrário das técnicas de pesquisa clássica que sistematicamente explorarem caminhos partindo de um dado estado inicial, mantendo, em memória, um ou mais caminhos, e quais alternativas é que já foram exploradas (e quais é que não foram).

Caracterizadamente operam utilizando apenas um **current node** (em vez de múltiplos paths) movendo-se, por norma, apenas para vizinhos desse node. Os **paths** seguidos pela pesquisa **não são guardados**

A ideia geral é que vamos pegar numa dada configuração inicial, e refina-la sucessivamente até obtermos uma configuração que achemos satisfatória. É por isso que estes algoritmos também se podem chamar de **pesquisas por melhorias sucessivas**

Como é óbvio, Local Search algorithms não são sistematicos, porém, possuem a vantagem de **utilizarem pouca memória** (por norma um valor constante) e de **poderem encontrar soluções razoáveis em state spaces grandes** (ou infinitos) (para os quais os algoritmos sistematicos tendem a falhar/entrar em loop)

Estes algoritmos são indicados para problemas nos quais o path cost é irrelevante e tudo o que nos interessa é chegarmos a um certo **solution state**, não sendo relevante como é que lá chegamos. Porém, são também úteis no que toca a **resolver problemas de optimização**, nos

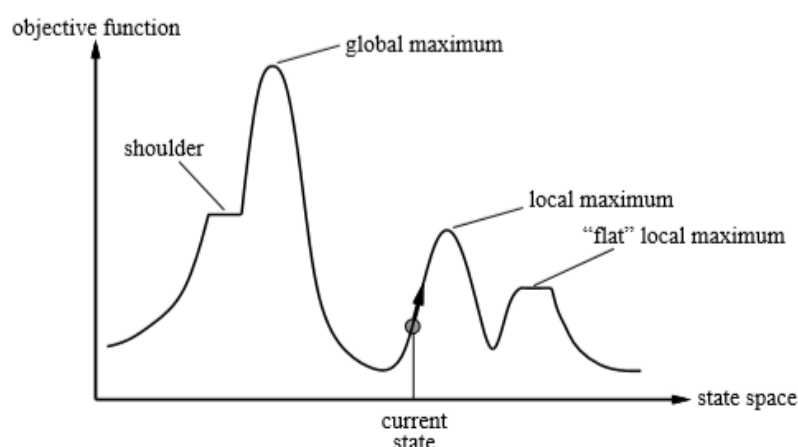
*quais o objetivo é encontrar um best state de acordo com uma dada **objective function***

Antes de prosseguirmos para os algoritmos, convém definirmos a existencia do chamado **State-Space Landscape**.

Isto não passa de uma representação gráfica do quão ótimo dado estado é relativamente a uma objective function e de que forma é que o state space evolui.

Um state space landscape é constituído por:

- **Location** – Definida pelo state
- **Elevation** – Definida pela objective function ou por um custo
- **Global Minimum** – Ponto com a menor elevation (corresponde ao goal state quando elevation for calculada em função de um custo e queremos o menor custo)
- **Global Maximum** – Ponto com a maior elevation (corresponde ao goal state quando elevation for calculada em função da objective function)
- **Local Maximum/Minimum** – Ponto mais alto/baixo comparativamente aos vizinhos
- **Plateaux** – Area onde os valores de elevação entre os vizinhos é igual. Pode ser um Flat Local Maximum ou um Shoulder
- **Ridge** – Sequencia de Local Maxima



Img 1.1 – Exemplo de um One Dimensional State Space Landscape. A **elevação** corresponde à objective function. O objetivo de um local search algorithm é o de encontrar um **global maximum**

II. Pesquisa por Reparação Heurística

É a técnica de pesquisa local mais básica. Basically, isto faz o que temos vindo a descrever – pegamos na solução inicial e vamos aplicando correções de acordo com uma heurística local.

Falamos mais deste algoritmo no final do próximo capítulo quando nos referimos a problemas de satisfação de restrições, onde a heurística utilizada é uma que escolha para correção o node que mais contribua para reduzir os conflitos entre restrições

III. Hill-Climbing Search – Pesquisa por Montanhismo

O *Hill Climbing Search Algorithm* não passa de um simples **loop** que **continuamente se move na direção que nos leve a um estado com mais valor do que o atual** – i.e que esteja uphill – **terminando quando atingirmos um “peak”** – i.e um estado com maior valor do que todos os seus vizinhos

Há quem chame a este algoritmo de **greedy local search** porque pega sempre no melhor neighbour state sem pensar no que vai fazer ou para onde vai a seguir.

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

Img 3.1 – O algoritmo Hill Climbing

Este algoritmo não usa backtracking!

Por norma, Hill Climbing progressa de forma muito rápida para uma solução, mas fica preso (não conseguindo chegar à solução correta) nas seguintes situações:

- **Local Maxima**
 - Quando o algoritmo chega a um local maxima não vai saber para onde se mexer visto que o estado em que está agora é melhor do que qualquer um dos seus vizinhos
- **Ridges**
- **Plateaux**
 - No caso dos flat local maximum, como não existe uphill não vai haver forma de nos mexermos
 - No caso dos shoulders podemos forçar a que o algoritmo se mexa sempre no caso de valores vizinhos iguais, e dessa forma vamos acabar por conseguir sair do plateaux. A isto chama-se um **sideways move**. Devemos ter cuidado quando implementamos esta regra visto que no caso de flat local maxima podemos entrar num loop infinito de sideway moves. Por norma deve-se impor um limite de quantos sideway moves consecutivos são permitidos

Existem algumas variantes do montanhismo:

- **Stochastic Hill Climbing**
 - Escolhe-se para onde vamos a seguir aleatoriamente (de entre os uphill moves)
 - A probabilidade de seleção pode variar com a steepness de um uphill move
 - Por norma vamos convergir mais lentamente do que a forma base do algoritmo, mas em alguns state landscapes vamos ser capazes de encontrar soluções melhores.
- **First Choice Hill Climbing**
 - Itera sobre a ideia do Stochastic Hill Climbing através de geração de sucessores de forma aleatória até que

um seja gerado um que seja melhor do que o estado atual.

- É uma boa estratégia quando cada estado tem muitos sucessores

Antes de falarmos do próximo algoritmo, é interessante referir que todos os Hill Climbing algorithms que vimos até aqui são **Incomplete** no sentido que não conseguem encontrar um goal (quando este existe) se ficarem presos numa das situações referidas anteriormente

- **Random-Restart Hill Climbing**

- Adota a ideia de “If at first you don’t succeed, try again”
- Vamos realizar várias Hill Climbing Searches, começando em randomly generated initial states, e parando quando um goal for encontrado
- Tem quase 100% probabilidade de ser Complete (se mais nada, então pelo facto que eventualmente vamos gerar o goal state como initial state)
- Se houverem poucos local maxima e plateaux, vamos ser capazes de encontrar uma solução muito rapidamente

IV. Simulated Annealing Search – Recozimento Simulado

Um Hill Climbing Search Algorithm nunca faz downhill moves que nos levem a um lower value (ou higher cost) nunca nos dará a garantia de ser complete porque pode ficar preso. Por outro lado, um random walk é complete vai ser extremamente ineficiente.

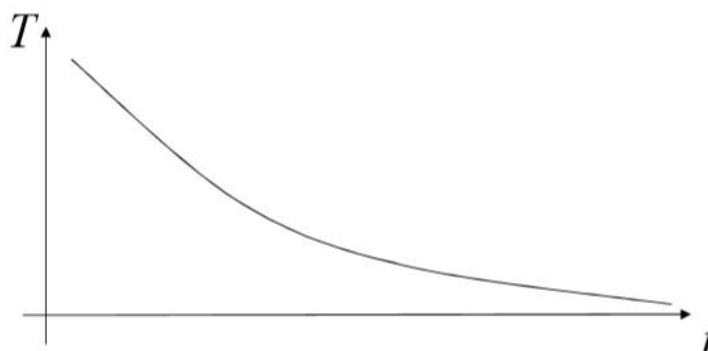
Simulated Annealing Search é um algoritmo que combina hill climbing com random walk. Tal como no hill climbing vamos procurar sempre ir para um estado melhor que o anterior, porém, vão também ser aceites movimentos que nos levem a um estado pior.

A ideia é a de escolhermos um sucessor aleatoriamente. Se o move melhorar a nossa situação, este será aceite, se não, vamos diminuir aceitar o move segundo uma probabilidade menor que 1.

Essa probabilidade, vai diminuir exponencialmente com a “badness” do move (i.e o quão pior o move escolhido é comparativamente ao atual).

A probabilidade também vai descendo à medida que uma variável de controlo, chamada **Temperatura** (aludindo para o significado metalurgico de annealing – temperar ou endurecer metais e vidro aquecendo-os e gradualmente arrefecendo-os) , vai diminuindo. A Temperatura deve também ir diminuindo à medida que o algoritmo progrressa

Bad moves são mais provavelmente permitidos quando a temperature está alta.



- $t \rightarrow \infty$
- $T \rightarrow 0$
- $Ganho/T \rightarrow -\infty$ (dado que o Ganho é negativo)
- Probabilidade: $\exp(Ganho/T) \rightarrow 0$
- Ou seja: À medida que o tempo passa, a pesquisa arrisca cada vez menos quanto a aceitar alterações com ganho negativo

Img 4.1 – Gráfico que mostra o decrescimo da probabilidade de escolha de um mau valor dependendo da temperatura

A pesquisa termina quando a Temperatura chegar a zero (devendo esta ir diminuindo)

É notável o facto que **para um decrescimo suficientemente lento da temperatura, o algoritmo consegue encontrar um global optimum com quase 100% de certeza**

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to “temperature”
  local variables: T, a “temperature” controlling the probability of downward steps

  current  $\leftarrow$  MAKE-NODE(problem.INITIAL-STATE)
  for t  $\leftarrow$  1 to  $\infty$  do
    T  $\leftarrow$  schedule(t)
    if T = 0 then return current
    next  $\leftarrow$  a randomly selected successor of current
     $\Delta E \leftarrow$  next.VALUE – current.VALUE
    if  $\Delta E > 0$  then current  $\leftarrow$  next
    else current  $\leftarrow$  next only with probability  $e^{\Delta E/T}$ 
```

Img 4.2 – O algoritmo de Simulated Annealing

V. Local Beam Search – Pesquisa local alargada

Por vezes, quando a sanidade atinge e paramos de tar todos mocados, podemos achar que manter apenas um node em memória é demasiado extremo. Well the person who invented this next algorithm thought so too:

O Local Beam Search Algorithm keeps track de *k* states em vez de apenas um. Começa com *k* randomly generated states e a cada step, todos os sucessores de todos os *k* states são gerados. Se qualquer um destes for o goal, o algoritmo para, se não, seleciona os *k* melhores sucessores da lista gerada e repete o algoritmo

A diferença entre este algoritmo e um random-restart search algorithm é o facto deste passar informação útil entre as *k* parallel search threads que ocorrem. Muito rapidamente searches de merda são abandonadas sendo os resources transferidos para os outcomes das searches que retornaram os melhores progressos.

Este algoritmo pode sofrer de pouca diversidade entre os k states (que podem rapidamente começar a ficarem concentrados numa pequena região do state space, tornando a pesquisa nada mais que uma versão mais cara do hill climbing)

Existe uma variant chamada **Stochastic Beam Search** que procura remediar isto de forma analoga à stocahistic hill climbing. *Em vez de escolhermos os k melhores estados da pool de sucessores candidatos, esta variante vai pegar em k sucessores, escolhidos aleatoriamente, sendo que a **probabilidade** de **escolher** cada sucessor é feita em função do seu valor* (pelo que sucessores melhores terão uma maior probabilidade de serem escolhidos).

VI. Genetic Algorithms – Algoritmos Genéticos

Um **algoritmo genético** *uma variação do stochastic beam search no qual os **successor states** são gerados através da combinação de dois parent states* (ao inves da modificação de um unico state).

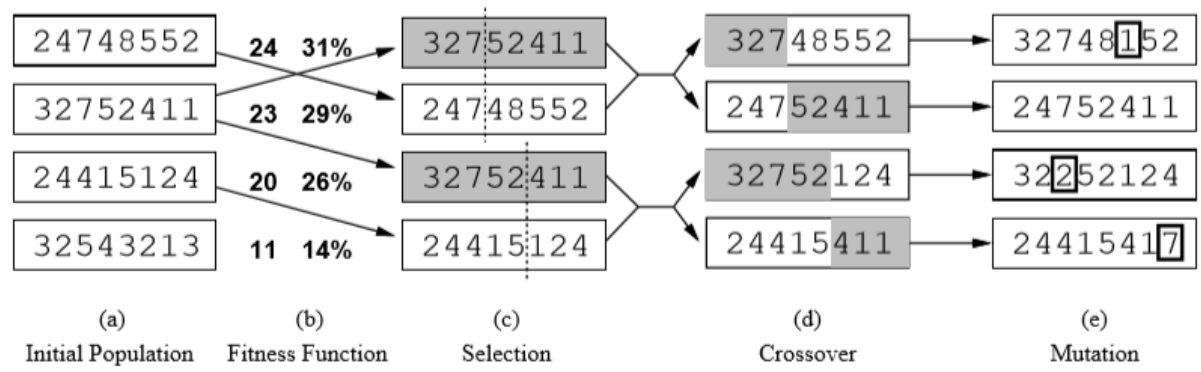
Tal como o beam search, começamos com um set de k randomly generated states, ao qual vamos chamar **População**

Cada state, ou **individual**, é representado por uma string sobre um alfabeto finito (por norma uma string de 0s e 1s), ou por um conjunto de digitos num determinado range.

Vamos utilizar uma **fitness function** que vai avaliar cada state. A fitness function deve returner valores mais altos para states melhores e, por norma, vamos seleccionar states para reproduzirem com uma propabilidade que vai depender do valor da fitness function.

Um sucessor vai ser creado através da combinação dos strings dos pais num determinado crossover point.

Por norma pode ocorrer uma random **mutation** na forma de um valor que se vai alterar de forma aleatoria e independente dos pais.



Img 6.1 – Exemplo do processo de reprodução num algoritmo genético

Este algoritmo combina a tendência uphill com exploração aleatória mais troca de informação entre parallel search threads – tal como o stochastic beam search algorithm.

A maior vantagem é o crossover entre states, vindo esta do facto que combinar largos blocos de letras na string que evoluíram de forma independente para serem capazes de realizar funções uteis, elevando o nível de granularidade à qual a search opera.

function GENETIC-ALGORITHM(*population*, FITNESS-FN) **returns** an individual

inputs: *population*, a set of individuals

FITNESS-FN, a function that measures the fitness of an individual

repeat

new_population $\leftarrow$ empty set

for $i = 1$ **to** SIZE(*population*) **do**

$x \leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

$y \leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

child $\leftarrow$ REPRODUCE(x, y)

if (small random probability) **then** *child* $\leftarrow$ MUTATE(*child*)

add *child* to *new_population*

population $\leftarrow$ *new_population*

until some individual is fit enough, or enough time has elapsed

return the best individual in *population*, according to FITNESS-FN

function REPRODUCE(x, y) **returns** an individual

inputs: x, y , parent individuals

$n \leftarrow$ LENGTH(x); $c \leftarrow$ random number from 1 to n

return APPEND(SUBSTRING($x, 1, c$), SUBSTRING($y, c + 1, n$))

Img 6.2 – Implementação de um genetic algorithm

Técnicas de Pesquisa com Restrições

I. Problemas de Constraint Satisfaction ou Problemas de Atribuição

Nos *problemas de atribuição* pretende-se **atribuir valores a um conjunto de variáveis respeitando um conjunto de restrições**

Caracterizam-se, portanto, por serem definidos por um **conjunto de variáveis**, $X_1, X_2, \dots, X_n$, e um **set de restrições**, $C_1, C_2, \dots, C_m$.

Cada uma das **variáveis** possui um **domínio**, D_i que engloba todos os possíveis **valores** para dada **variável**. Este domínio não deve ser vazio!

Cada uma das **constraints envolve um subset de variáveis e especifica as combinações de variáveis permitidas** para esse subset

Um **State do problema** é então definido pelo **assignment** de values para algumas, ou todas (sendo nesse caso **complete**), as variáveis. Caso o assignment não viole nenhuma constraint, dizemos que é **consistent** ou **legal**.

Temos uma **solução** quando possuímos um assignment completo e consistente! Note-se que alguns CSPs podem ainda requerer uma solução que maximize uma objective function

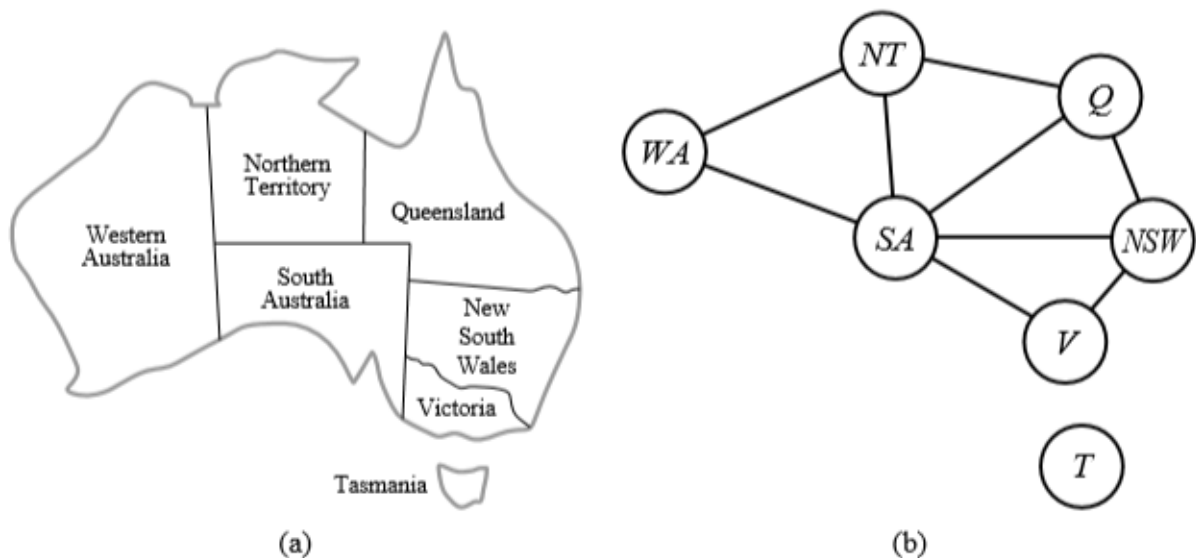
No decorrer das próximas secções vamos utilizar o seguinte problema como exemplo:

“Dada a tarefa de pintar cada estado e território da Australia – WA, NT, Q, NSW, V, SA e T - com as cores – red, green, blue – qual seria uma possível configuração de estado-cor, tal que, estados que sejam adjacentes não sejam pintados com a mesma cor”

Variáveis: { WA, NT, Q, NSW, V, SA, T }

Domínio: {red, green, blue} (note-se que os domínios de cada variável são todos iguais)

Restrições: { $\text{Cor}(\text{WA}) \neq \text{Cor}(\text{NT})$, $\text{Cor}(\text{WA}) \neq \text{Cor}(\text{SA})$, $\text{Cor}(\text{NT}) \neq \text{Cor}(\text{Q})$, $\text{Cor}(\text{NT}) \neq \text{Cor}(\text{SA})$, $\text{Cor}(\text{Q}) \neq \text{Cor}(\text{NSW})$, $\text{Cor}(\text{Q}) \neq \text{Cor}(\text{SA})$, $\text{Cor}(\text{NSW}) \neq \text{Cor}(\text{V})$, $\text{Cor}(\text{NSW}) \neq \text{Cor}(\text{SA})$, $\text{Cor}(\text{V}) \neq \text{Cor}(\text{SA})$ }



Img 1.1 – Mapa dos estados da Australia e Grafo de Restrições do problema. Note-se que nestes grafos, cada node corresponde a variáveis do problema e os arcos correspondem a constraints

Obviamente, podemos formular este (ou qualquer CSP) com a mesma incremental formulation que damos aos search problems normais:

Initial State: Empty Assignment { } (na qual todas as variáveis estão unassigned)

Successor Function: Um valor pode ser assigned a qualquer unassigned variable, dado que este não entre em conflito com previously assigned variables (Esta function engloba tanto as **ações disponíveis ao agente** – atribuir valores a variáveis, como o **transition model** – cada ação cria um novo assignment)

Goal Test: O atual assignment é complete

Path Cost: Um custo constante para cada step

Convém dizer que sabemos que a **solution** será uma **complete assignment** (visto que o agente nunca fará assignments que violem restrições), que aparecerá na depth **n**, sendo n o número de variáveis.

Aliás, a tree nunca poderá ter uma depth maior que n pois a solução do problema requer que seja atribuído apenas um e apenas um valor a cada uma das variáveis.

Estas razões que acabamos de descrever explicam o porque dos **depth-first search algorithms** serem tão populares para a resolução de CSPs

Antes de continuarmos, temos só mais algumas definições a fazer sobre as constraints e sobre os domínios das variáveis

As **constraints** podem ser de tres tipos:

- **Restrições Unárias**
 - Envolvem apenas **uma variável**
 - Podem ser satisfeitas através de pre-processamento do domínio de valores da variável, bastando limitar-mos a priori o domínio da variável para englobar apenas os valores que satisfazem a restrição
 - Ex:
 - Variável X de domínio {DS, Pedro, Rey}
 - Restrição: X não pode ser Pedro
 - Basta restringimos o domínio de X a {DS, Rey}

- **Restrições Binárias**

- Envolvem **duas variáveis**
- É diretamente representada por uma aresta num grafo de restrições
- Ex:
 - Variável X de domínio {DS, Pedro, Rey} e Y de domínio {Programador, Jogador, Gigolo}
 - Restrição: Se X for Pedro, Y só pode ser Gigolo

- **Restrições de Ordem Superior**

- Envolvem **tres ou mais variáveis**
- Através de **variáveis auxiliares**, uma restrição de ordem superior pode ser **transformada** num conjunto de **restrições binárias** e/ou **unárias**

- Ex:

- Num puzzle Cryptaritmético insiste-se que cada letra represente um dígito diferente.
- No seguinte exemplo de um puzzle cryptaritmético temos então as variáveis T,W,O,F,U,R e a **restrição** que estas representem todas digitos diferentes (Alldif(F,T,U,W,R,O))
- Isto é portanto uma restrição de ordem 6 (envolve 6 variáveis)
- Temos ainda as **restrições**, onde X1 e X2 são variáveis internas que representam o transporte das unidades para as dezenas e o transporte das dezenas para as centenas (podendo então ser 0 ou 1):

$$\begin{array}{r} T \ W \ O \\ + \ T \ W \ O \\ \hline F \ O \ U \ R \end{array}$$

- $2*O = R + 10*X1$
- $2*W + X1 = U + 10*X2$
- $2*T + X2 = O + 10*F$
- Podemos facilmente representar esta restrição utilizando uma coleção de **variáveis auxiliares**:
 - $2*O = R + 10*X1$, por exemplo, pode ser transformada no conjunto de restrições:
 - $O = \text{primeiro}(\text{Aux})$

- $R = \text{segundo}(\text{Aux})$
- $X1 = \text{terceiro}(\text{Aux})$
- $2 * \text{primeiro}(\text{Aux}) = \text{segundo}(\text{Aux}) + 10 * \text{terceiro}(\text{Aux})$
- As 3 primeiras restrições são binárias, enquanto que a ultima é unária, envolvendo estas a **nova variável auxiliar, Aux** que possuirá a seguinte concatenação de dígitos: $\{0,9\} \times \{0,9\} \times \{0,1\}$.
- Note-se que $\text{primeiro}()$, $\text{segundo}()$ e $\text{terceiro}()$ indicam que vamos buscar o primeiro, segundo ou terceiro dígito da variável Aux

No que toca aos **domínios**, podem ser **finitos**, ou **infinitos**. No caso destes últimos já não vamos conseguir descrever constraints através da enumeração de todas as combinações de valores possíveis – vamos precisar de uma constraint language

II. Backtracking Search para CSP, a.k.a Método Construtivo

Já dissemos na secção anterior que podemos facilmente resolver um CSP utilizando as técnicas de tree searching que falamos no capítulo da pesquisa clássica. Umas, obviamente, serão mais merda que outras.

Por exemplo, Breadth First-Search é péssimo nestes casos porque o branching factor é possivelmente enorme! Basicamente, não é boa ideia tentarmos gerar todas as possibilidades para todos os nodes de cada nível antes de passarmos ao próximo.

Os CSP tem uma propriedade que ainda não falamos – **Commutativity**. Isto implica que a ordem pela qual aplicamos as nossas ações não vai ter nenhum efeito (importante) no outcome. Nos CSP só queremos que cada variável fique com um estado tal que não haja

rompimento de nenhuma restrição. Não nos interessa porque ordem é que os estados foram atribuídos visto que vamos chegar ao mesmo assignment.

Uma propriedade dos CSP Search Algorithms é o facto de *gerarem sucessores considerando possíveis assignments para uma única variável em cada node da search tree*. Ou seja, em cada passo da pesquisa atribui-se um valor a uma variável.

Note-se o estado de cada node, nos CSP, corresponde não a uma possível configuração do mundo, mas sim ao atual assignment (i.e atribuição de valores a certas variáveis)

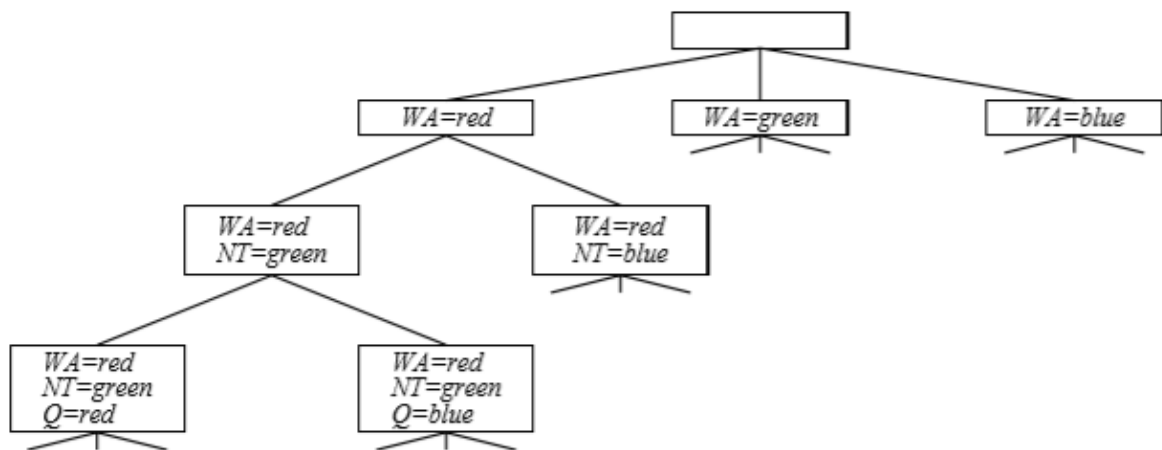
Temos então a **Backtracking Search** também conhecido por **Método Construtivo** que se baseia num **Depth-First Search** que vai escolher uma variável de cada vez e fazer backtrack quando a variável já não possuir valores legais que possam ser atribuídos

Temos então o algoritmo recursivo:

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment according to CONSTRAINTS[csp] then
      add {var = value} to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove {var = value} from assignment
  return failure
```

Img 2.1 – Backtracking Search Algorithm. Note-se as similaridades com o recursivo Depth Search Algorithms discutidos anteriormente. As funções SELECT-UNASSIGNED-VARIABLE e ORDER-DOMAIN-VALUES foram deixadas abstratas podendo ser alteradas dependendo das heurísticas de escolha que vamos usar



Img 2.2 – Excerto da possível Search Tree gerada pelo algoritmo descrito sobre o problema que introduzimos na secção anterior

Este algoritmo é obviamente um **Uninformed Search Algorithm**, pelo que não se espera que seja muito eficiente para problemas grandes.

III. Pesquisa com Propagação de Restrições

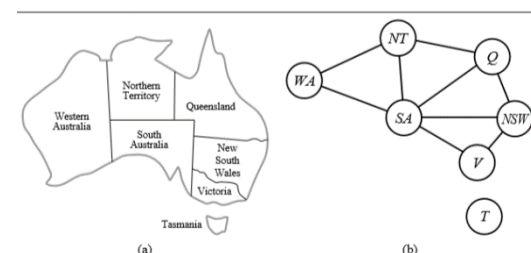
No algoritmo anterior, só consideramos as restrições sobre uma variável quando essa variável for escolhida pelo SELECT-UNASSIGNED-VARIABLE.

Podemos **diminuir o search space considerando as restrições mais cedo** na pesquisa (ou mesmo antes desta começar) !

Uma forma que temos de fazer isto é através de **Constraint Propagation** – propagação das implicações de uma restrição de uma variável para as outras.

Por exemplo, considerando o nosso problema da primeira secção:

- Quando WA for red e Q for green, tanto NT como SA serão forçadas a ser blue. Isto porém não vai ser possível pois NT e SA são adjacentes



- Neste caso teríamos de propagar de WA e Q até NT e SA e depois até à restrição entre NT e SA, de forma a que a violação seja detetada

Um possível algoritmo para Constraint propagation baseia-se na ideia de **arc consistency**. Neste caso, arco refere-se ao arco direcionado no constraint graph, (por exemplo o arco entre NT e SA). Dados os atuais domínios de SA e NT, o arco é consistente se, para cada valor de SA, existir QUALQUER valor y, de NT, que seja consistente com x.

Temos então o algoritmo:

1. Inicialização: o nó inicial da árvore de pesquisa é composto por todas as variáveis e todos os valores possíveis para cada uma delas
2. Se pelo menos uma variável tem um conjunto de valores vazio, falha e retrocede; se não puder retroceder, a pesquisa falha
3. Se todas as variáveis têm exactamente um valor possível, tem-se uma solução; retornar com sucesso
4. Expansão: Escolher arbitrariamente uma variável V_k e, de entre os valores possíveis, um dado valor X_{kl} – descartar os restantes valores possíveis dessa variável
5. Propagação de restrições: para cada arco (i,j) no grafo de restrições, remover os valores na variável V_i por forma a que o arco fique consistente
6. Caso tenha sido preciso remover valores na origem de algum arco, voltar a repetir o passo 5.
7. Voltar ao passo 2.

Img 3.1 – Pseudocódigo do algoritmo de pesquisa com propagação de restrições baseado em arc consistency

Especificamente, são os passos **5** e **6** deste algoritmo que executam a propagação de restrições. Esta parte deve ser suportada por uma queue de arestas do grafo de restrições.

Inicialmente, essa queue deve conter as arestas que apontam para a variável cujo valor foi fixado

Podemos ver isto em ação analisando a implementação do algoritmo AC-3 que realiza os passos 5 e 6:

function AC-3(*csp*) **returns** the CSP, possibly with reduced domains

inputs: *csp*, a binary CSP with variables $\{X_1, X_2, \dots, X_n\}$

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\text{queue})$

if REMOVE-INCONSISTENT-VALUES(X_i, X_j) **then**

for each X_k **in** NEIGHBORS[X_i] **do**

 add (X_k, X_i) to *queue*

function REMOVE-INCONSISTENT-VALUES(X_i, X_j) **returns** true iff we remove a value

removed $\leftarrow$ false

for each x **in** DOMAIN[X_i] **do**

if no value y in DOMAIN[X_j] allows (x, y) to satisfy the constraint between X_i and X_j

then delete x from DOMAIN[X_i]; *removed* $\leftarrow$ true

return *removed*

Img 3.2 – Algoritmo de Arc Consistency AC-3

Note-se que arc-consistency Checking pode ser realizado como um passo de pré-processamento (antes de começarmos o search process) ou como um step de propagação (após cada assignment) durante a pesquisa

IV. Local Search para CSPs – Pesquisa por Melhorias Sucessivas

Os local search algorithms que vimos no capítulo anterior também são bastante eficientes a resolver certos CSPs utilizando uma **complete-state formulation**: O estado inicial atribui um valor a cada variável, e a successor function normalmente funciona mudando o valor de uma variável de cada vez. (basically were gonna work backwards)

Ao escolher um novo valor para uma variável, a heurística mais óbvia é a de escolher um valor que resulte no **minimo numero de conflitos com outras variáveis** – chamamos a isto a **Min-Conflicts Heuristics**

```
function MIN-CONFLICTS(csp, max_steps) returns a solution or failure
  inputs: csp, a constraint satisfaction problem
           max_steps, the number of steps allowed before giving up

  current  $\leftarrow$  an initial complete assignment for csp
  for i = 1 to max_steps do
    if current is a solution for csp then return current
    var  $\leftarrow$  a randomly chosen, conflicted variable from VARIABLES[csp]
    value  $\leftarrow$  the value v for var that minimizes CONFLICTS(var, v, current, csp)
    set var = value in current
  return failure
```

Img 4.1 – Algoritmo que resolve um CSP implementando a heurística Min-Conflicts. O estado inicial pode ser escolhido de forma aleatória ou por um greedy assignment process que escolhe o minimal-conflict value para cada variável de cada vez. A função CONFLICTS conta o número de constraints violadas por um valor específico, dado o resto do assignment atual.

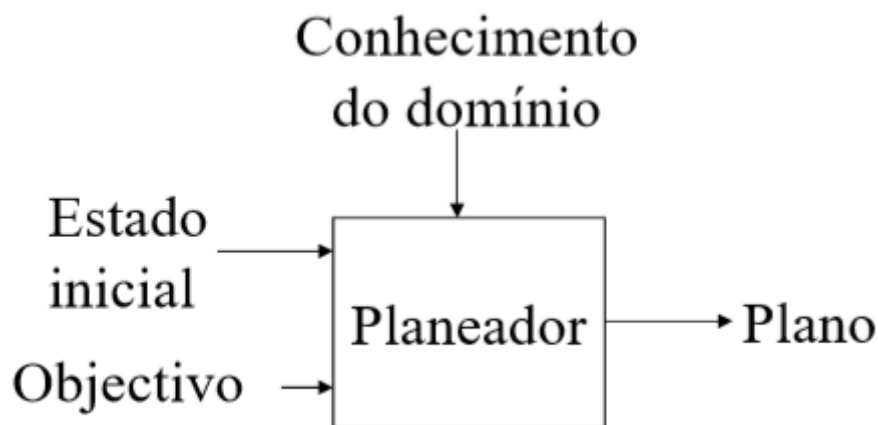
Este algoritmo é surpreendentemente eficiente, especialmente quando o estado inicial for decente. Por outro lado, é um tipo de local search, pelo que pode ser usado num “online setting quando o problema muda” (particularmente importante em Scheduling problems !)

Planeamento

I. Definição de Planning

Planning define-se como a **criação de um plano de ação para atingir um objetivo**

Os problem solving agents que vimos nos anteriores capitulos conseguem encontrar sequencias de ações que resultam num goal state, mas para isso precisam que as domain-specific heuristics sejam bem definidas (porque lidam com representações atómicas de estados).



Img 1.1 – Um Planeador na pesquisa clássica pega no estado inicial, no objetivo e no conhecimento do domínio para criar um plano que leve o agente até ao objetivo pretendido

Por outro lado, agentes que se baseiam na representação de conhecimento com lógica conseguem encontrar planos mesmo sem domain-specific heuristics porque usam domain-independent heuristics baseadas na estrutura lógica do problema. Porém, dependem de propositional inference, pelo que podem ficar sobrecarregados de ações e estados.

O que nós queremos é que um **agente consiga aproveitar-se da estrutura do problema para construir planos de ação complexos** (que é basicamente o que os agentes baseados em lógica fazem mas mal).

O consenso geral é que o uso de uma **factored representation** – uma representação na qual o estado do mundo seja apresentado como uma coleção de variáveis - consegue resolver o problema dos agentes lógicos

II. STRIPS – O primeiro Planeador

A STRIPS – Stanford Research Institute Problem Solver - foi um **planeador automático** desenvolvido por Fikes e Nillson em 1971 que mais tarde surgiu também uma **planning language** (com o mesmo nome) que serve como inputs desse mesmo planeador.

Esta linguagem serviu como base para a maioria das outras linguagens que expressam automated planning problem instances – também conhecidas como **action languages**

Em STRIPS a **funcionalidade** (ou reação) de um dado tipo de **operação** é **defenida** através de uma **estrutura** chamada **operador** que inclui a seguinte informação:

- **Pre-Condições**
 - Conjunto de formulas atómicas que representam as **condições de aplicabilidade** deste tipo de operação
 - Ou seja, condições que têm que se verificar para que se possa realizar a ação
- **Efeitos Negativos (DELETE list)**
 - Um conjunto de fórmulas atómicas que representam **propriedades** do mundo que **deixam de ser verdade** ao executar-se a operação

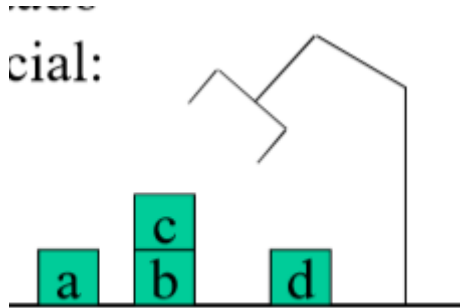
- **Efeitos Positivos** (ADD list)
 - Conjunto de fórmulas atômicas que representam **propriedades** do mundo que **passam a ser verdade** ao executar-se a operação

Ex: Dado o problema

*Temos **4 blocos**, a, b, c e d tal que **inicialmente** o bloco a, d e b estão **em cima do chão** e o bloco c está em **cima do b** e um braço robótico. **Queremos** chegar a um estado no qual o bloco c e b estejam **em cima do chão** e o a e d **em cima** do c e b, correspondentemente. As ações possíveis são levantar um bloco do chão, pousar um bloco no chão, empelhar um bloco em cima de outro (faz com que o bloco fique pusado em cima de outro), desempelhar um bloco de cima de outro (faz com que o bloco fique levantado). Note-se que o braço só pode pegar num bloco de cada vez*

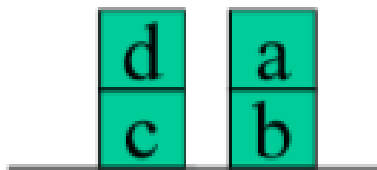
Podemos então identificar:

- **Estado inicial:**



Img 2.1 – Estado inicial do nosso problema

- **Objetivo:**



Img 2.2– Goal state do nosso problema

- **Ações:**
 - Levantar(x)
 - Pousar(x)
 - Desempelhar(x,y)
 - Empelhar(x,y)
- **Objetos:**
 - A (bloco)
 - B (bloco)
 - C (bloco)
 - D (bloco)
 - Robot (Braço robótico)
- **Estados dos objetos:**
 - Robot_livre
 - No_chão(X)
 - Livre(X)
 - Em_cima(X,Y)

O STRIPS estaria portanto encarregue de representar cada ação usando **pre condições**, **efeitos negativos** e **efeitos positivos** e criar um plano. Note-se que a definição das pre-condições, efeitos negativos e efeitos positivos é realizada através da análise do problema:

- **Levantar(X)**
 - Pre-condições: [no_chão(X), robot_livre]
 - Efeitos Negativos: [no_chão(X), robot_livre]
 - Efeitos Positivos: [no_robot(X)]
- **Pousar(X)**
 - Pre-condições: [no_robot(X)]
 - Efeitos Negativos: [no_robot(X)]
 - Efeitos Positivos: [no_chão(X), robot_livre]

- **Desempilhar(X,Y)**
 - Pre-condições: [em_cima(X,Y), robot_livre]
 - Efeitos Negativos: [em_cima(X,Y), robot_livre]
 - Efeitos Positivos: [no_robot(X), livre(Y)]

- **Empilhar(X,Y)**
 - Pre-condições: [no_robot(X), livre(Y)]
 - Efeitos Negativos: [no_robot(X), livre(Y)]
 - Efeitos Positivos: [em_cima(X,Y), robot_livre]

O STRIPS inseriria estas **especificações das ações** como input ao planeador automático e chegaria a um plano como:

[desempilhar(c,b)
 Pousar(c),
 Levantar(d),
 Empilhar(d,c),
 Levantar(a),
 Empilhar(a,b)]

Agora que sabemos, o que é que o STRIPS faz e como é que a definição da sua linguagem de inputs funciona convém também vermos como é que ele o faz. Temos então o algoritmo do planeador do STRIPS:

```

STRIPS(EI,Objectivos)  % EI é argumento de entrada/saída
Plano ← [ ]
repetir {
    C ← uma condição em Objectivos não satisfeita em EI
    OP ← um operador que pode ter C como efeito positivo
    A ← acção, dada por uma completa instanciação de OP
    PC ← pré-condições de A
    SubPlano ← STRIPS(EI,PC)
    Plano ← concatenar(Plano,concatenar(SubPlano,[A]))
    EI ← novo estado, resultante da aplicação de A em EI
    se Objectivos satisfeitos em EI,
        retornar Plano
}

```

Img 2.3– Algoritmo do planeador STRIPS

Abreviaturas de condições:

- Bloco em cima de bloco: $ec(A,B)$
- Bloco no chão: $c(B)$
- Bloco no robô: $r(X)$
- Robô livre: rl
- Bloco livre: $l(X)$

Abreviaturas de operadores:

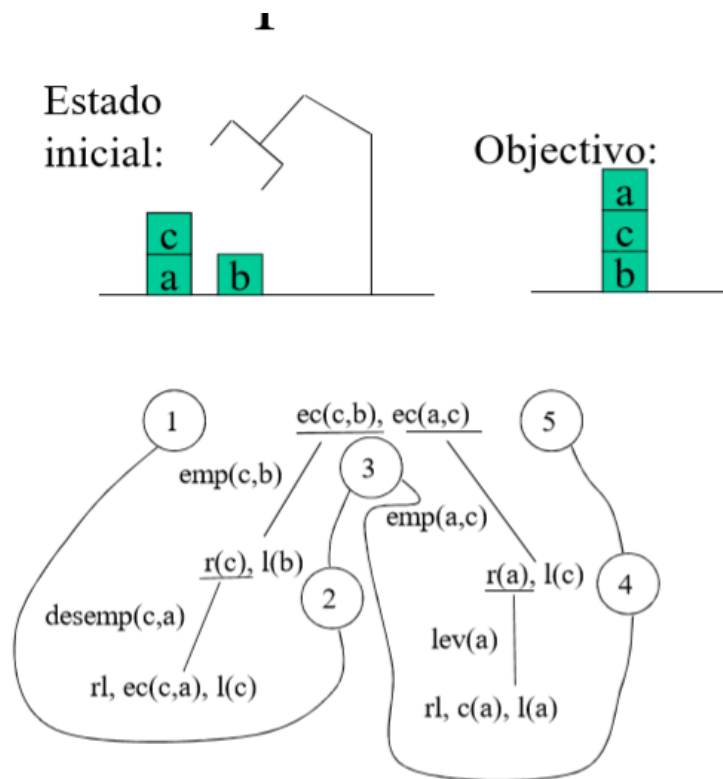
- Empilhar: $emp(A,B)$
- Desempilhar: $desemp(A,B)$
- Levantar: $lev(X)$
- Poisar: $p(X)$

Plano:

- $desemp(c,a)$, $emp(c,b)$, $lev(a)$, $emp(a,c)$

Sucessão de estados:

- 1: $ec(c,a)$, $c(a)$, $l(c)$, $c(b)$, $l(b)$, rl
- 2: $r(c)$, $l(a)$, $c(a)$, $c(b)$, $l(b)$
- 3: $ec(c,b)$, $l(c)$, $l(a)$, $c(b)$, $c(a)$, rl
- 4: $r(a)$, $ec(c,b)$, $c(b)$, $l(c)$
- 5: $ec(a,c)$, $ec(c,b)$, $c(b)$, rl



Img 2.4– Overview do processo de resolução do problema dos blocos pelo planeador STRIPS

III. A Anomalia de Sussman

O STRIPS tem um problema – Dependendo da ordem pela qual o STRIPS trata os objetivos, os seguintes planos no problema que apresentamos na ultima seccção podem ser originados:

[desempilhar(c,a), poisar(c), levantar(a), empilhar(a,b),
desempilhar(a,b), poisar(a), levantar(b), empilhar(b,c), levantar(a),
empilhar(a,b)]

[levantar(b), empilhar(b,c), desempilhar(b,c), poisar(b),
desempilhar(c,a), poisar(c), levantar(a), empilhar(a,b), desempilhar(a,b),
poisar(a), levantar(b), empilhar(b,c), levantar(a), empilhar(a,b)]

Nenhum destes é um plano otimo...alialis, o STRIPS não vai conseguir gerar um plano otimo para este problema

Este problema (dos blocos) demonstra a anomalia de Sussman é um problema em AI que ilustra a fraqueza dos noninterleaved planning algorithms.

IV. Planeamento no espaço de soluções

Nas aproximações ao planeamento anteriormente apresentadas **cada nó da pesquisa corresponde a um estado do mundo**

A isto chamamos **planeamento no espaço de estados**

Planeamento no espaço de soluções é uma tecnica alternativa consiste em **partir de um plano vazio e adicionar sucessivamente operações e restrições de sequenciamento**

Neste caso, cada **nó de pesquisa corresponde a uma solução parcial para o problema**

Para tal temos as operações de transformação da solução:

- Adicionar um operador
- Re-ordenar operadores
- Instanciar um operador

V. Planeamento Hierárquico - ABSTRIPS

O planeamento hierárquico é realizado naquilo que se chama uma **hierarquia de níveis de abstração**

O ABSTRIPS foi o primeiro sistema que automatizou a construção de abstraction hierarchies para planeamento.

Um **valor de criticalidade** é **atribuído a cada uma das condições que podem aparecer na descrição do estado do mundo levando-nos portanto a uma organização hierárquica das ações**

O algoritmo é:

1. $CM \leftarrow$ valor inicial para o nível de criticalidade mínima.
2. Gerar um plano que satisfaça todas as condições com nível de criticalidade $\geq CM$.
3. $CM \leftarrow CM-1$
4. Usando o plano anterior como guia, gerar um plano que satisfaça todas as condições com criticalidade $\geq CM$.
5. **se** todas as condições estão satisfeitas, **retornar** a solução.
6. **voltar** ao passo 3.

Img 5.1– Algoritmo de planeamento hierárquico

Voltando ao nosso problema da secção anterior podemos verificar a diferença entre os dois planeadores:

Abreviaturas de condições:

- Bloco em cima de bloco: $ec(A,B)$
- Bloco no chão: $c(B)$
- Bloco no robô: $r(X)$
- Robô livre: rl
- Bloco livre: $l(X)$

Abreviaturas de operadores:

- Empilhar: $emp(A,B)$
- Desempilhar: $desemp(A,B)$
- Levantar: $lev(X)$
- Poisar: $p(X)$

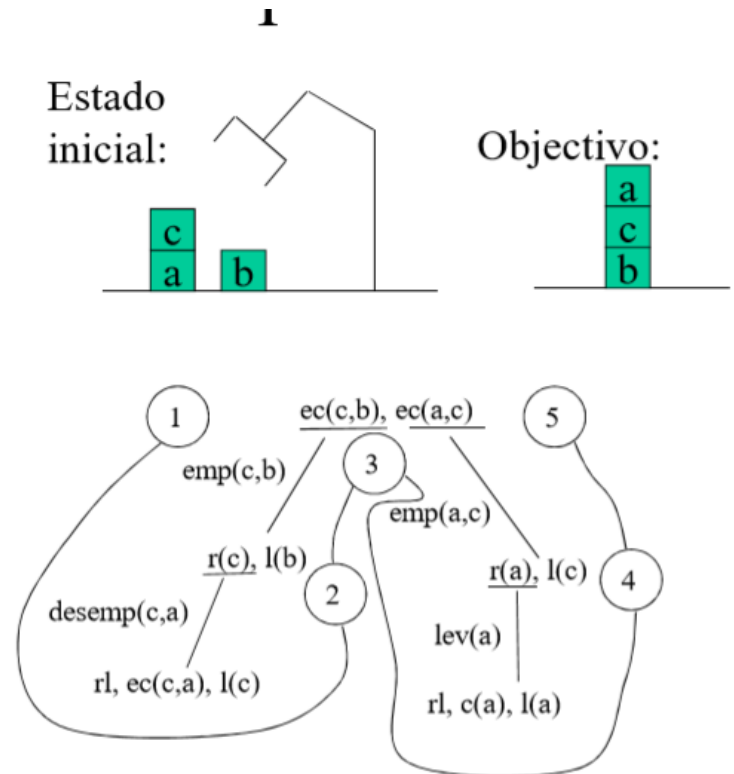
Plano:

- $desemp(c,a)$, $emp(c,b)$, $lev(a)$, $emp(a,c)$

Sucessão de estados:

- 1: $ec(c,a)$, $c(a)$, $l(c)$, $c(b)$, $l(b)$, rl
- 2: $r(c)$, $l(a)$, $c(a)$, $c(b)$, $l(b)$
- 3: $ec(c,b)$, $l(c)$, $l(a)$, $c(b)$, $c(a)$, rl
- 4: $r(a)$, $ec(c,b)$, $c(b)$, $l(c)$
- 5: $ec(a,c)$, $ec(c,b)$, $c(b)$, rl

Img 5.2– STRIPS



Dois níveis de criticalidade:

- $ec(A,B)$ - 2
- $c(B)$ - 1
- $r(X)$ - 2
- rl - 1
- $l(X)$ - 1

Plano inicial:

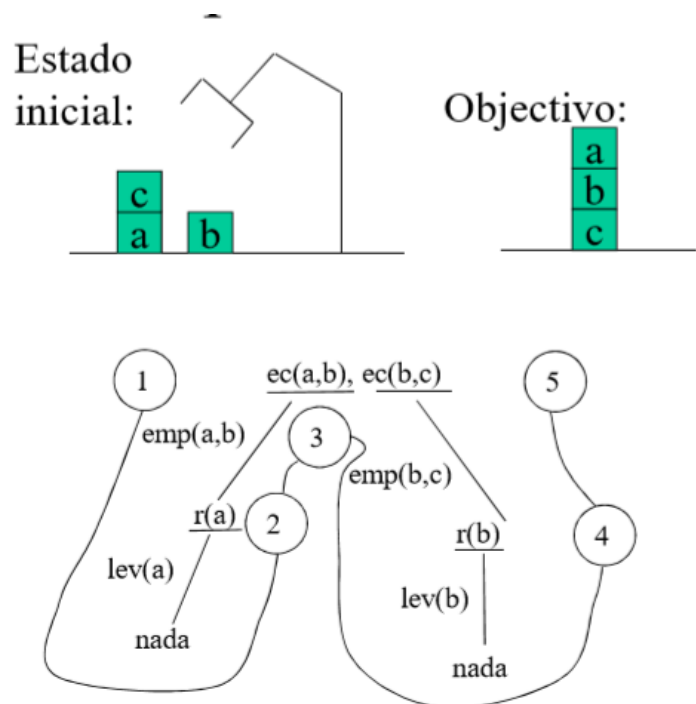
- $lev(a)$, $emp(a,b)$, $lev(b)$, $emp(b,c)$

Sucessão de estados:

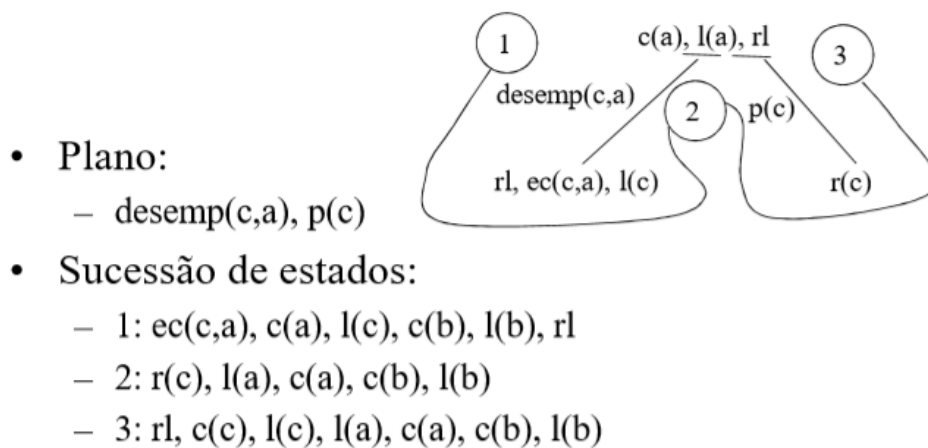
- 1: $ec(c,a)$
- 2: $ec(c,a)$, $r(a)$
- 3: $ec(c,a)$, $ec(a,b)$
- 4: $ec(c,a)$, $ec(a,b)$, $r(b)$
- 5: $ec(c,a)$, $ec(a,b)$, $ec(b,c)$

Os estados não são consistentes!

Img 5.3– ABSTRIPS com $CM = 2$



- As precondições de criticalidade 1 da primeira acção, $lev(a)$, não estão reunidas, pelo que é preciso determinar um plano para as atingir



- Plano:
 - $desemp(c,a), p(c)$
- Sucessão de estados:
 - 1: $ec(c,a), c(a), l(c), c(b), l(b), rl$
 - 2: $r(c), l(a), c(a), c(b), l(b)$
 - 3: $rl, c(c), l(c), l(a), c(a), c(b), l(b)$

Img 5.4– ABSTRIPS com CM = 1

VI. Operadores com Fórmulas Atómicas e Condicionais

O STRIPS é velho e merda...nice..

Do STRIPS nasceu a linguagem **PDDL** que é ligeiramente menos restrita que o STRIPS

PDDL – Planning Domain Definition Language, tal como o STRIPS, procura definir o estado inicial, ações disponíveis num estado, resultado de aplicar as ações e um goal test.

Ao contrario do STRIPS porém, o PDDL aceita que preconditions e goals contenham literais negativos!

Temos então:

Literal – uma formula atómica (literal positivo) ou negação de uma fórmula atómica (literal negativo)

Fórmula de aplicabilidade do operador pode ser:

- Fórmula atómica
- Negação de uma fórmula
- Conjunção de fórmulas
- Disjunção de fórmulas
- Fórmula quantificada existencialmente
- Fórmula quantificada universalmente

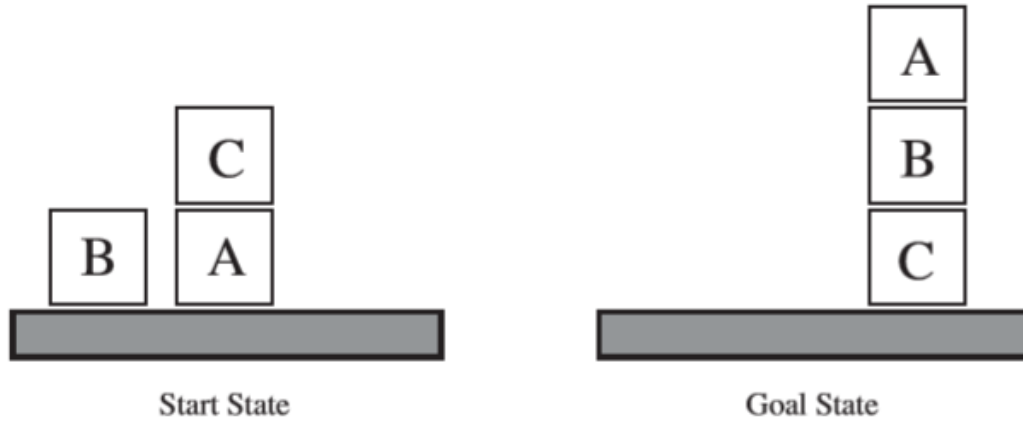
Fórmula de efeitos do operador pode ser:

- Literal
- Conjunção de literais
- Efeitos condicionais: when <fórmula de aplicabilidade> <fórmula de efeitos>
- Fórmula de efeitos quantificada universalmente
- Conjunção de fórmulas de efeitos

Img 6.1– Elementos do PDDL

```
(:action stop
:parameters (?f - floor)
:precondition (lift-at ?f)
:effect (and
  (forall (?p - passenger)
    (when (and (boarded ?p) (destin ?p ?f) )
      (and (not (boarded ?p)) (served ?p))))
  (forall (?p - passenger)
    (when (and (origin ?p ?f) (not (served ?p)))
      (boarded ?p))))))
```

Img 6.2– Exemplo da definição de uma ação – stop – em PDDL. Temos os parametros que a ação aceita, as precondições que têm que se verificar e os efeitos (que são altamente definidos).



Init(*On*(A, Table) $\wedge$ *On*(B, Table) $\wedge$ *On*(C, A)
 $\wedge$ *Block*(A) $\wedge$ *Block*(B) $\wedge$ *Block*(C) $\wedge$ *Clear*(B) $\wedge$ *Clear*(C))
Goal(*On*(A, B) $\wedge$ *On*(B, C))
Action(*Move*(b, x, y),
 PRECOND: *On*(b, x) $\wedge$ *Clear*(b) $\wedge$ *Clear*(y) $\wedge$ *Block*(b) $\wedge$ *Block*(y) $\wedge$
 (b $\neq$ x) $\wedge$ (b $\neq$ y) $\wedge$ (x $\neq$ y),
 EFFECT: *On*(b, y) $\wedge$ *Clear*(x) $\wedge$ $\neg$ *On*(b, x) $\wedge$ $\neg$ *Clear*(y))
Action(*MoveToTable*(b, x),
 PRECOND: *On*(b, x) $\wedge$ *Clear*(b) $\wedge$ *Block*(b) $\wedge$ (b $\neq$ x),
 EFFECT: *On*(b, Table) $\wedge$ *Clear*(x) $\wedge$ $\neg$ *On*(b, x))

Img 6.3— Exemplo da representação do planning problem dos blocos (simplificado) em PDDL. Neste problema temos apenas 3 blocos, A, B e C, e só podemos realizar as ações *Move*(b,x,y) que leva um bloco b que está em x até y, e *MoveToTable*(b,x) que leva o bloco b que está em x para o chão. Uma possível sequência de ações criadas por este planeador seria: [*MoveToTable*(C,A), *Move*(B,Table,C), *Move*(A,Table,B)]



The End :)